

ระบบจัดการแข่งขัน Bracket Tournament พร้อมระบบ โหวต Real-time

BattleHub: Bracket Tournament Management System with Real-time Voting

กิตติธัช เอมรัตน์

โครงงานนี้เป็นส่วนหนึ่งของการศึกษา

หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์

ภาควิชาคณิตศาสตร์ สถิติ และคอมพิวเตอร์ คณะวิทยาศาสตร์

มหาวิทยาลัยอุบลราชธานี

ปีการศึกษา 2567

ลิขสิทธิ์ของมหาวิทยาลัยอุบลราชธานี

โครงการ : ระบบจัดการแข่งขัน Bracket Tournament พร้อมระบบโหวต Real-time
BattleHub: Bracket Tournament System with Realtime Voting

โดย : นายกิตติธัช เอ็มรัตน์
อาจารย์ที่ปรึกษา : อาจารย์วาโย ปุยะติ

ระดับการศึกษา : วิทยาศาสตร์บัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรม
ซอฟต์แวร์

ปีการศึกษา : 2567

ได้รับการพิจารณาให้เป็นส่วนหนึ่งของการศึกษา
ตามหลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์
คณะกรรมการสอบประเมินความรู้โครงการของคอมพิวเตอร์

..... อาจารย์ที่ปรึกษา
(วาโย ปุยะติ)

..... กรรมการ
(ชื่อกรรมการ)

..... กรรมการ
(ชื่อกรรมการ)

..... หัวหน้าภาควิชา
(รศ.ชาญชัย ศุภอรรถกร)

วันที่/...../.....

กิตติกรรมประกาศ

โครงการฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาจาก อ.วาโย ปุยะติ อาจารย์ที่ปรึกษาโครงการ ที่ให้คำแนะนำ ข้อเสนอแนะ และข้อคิดเห็นต่างๆ ที่เป็นประโยชน์อย่างยิ่งในการดำเนินโครงการ รวมถึงการตรวจแก้ไขข้อบกพร่องต่างๆ ด้วยความเอาใจใส่

ขอขอบพระคุณคณาจารย์ภาควิชาคณิตศาสตร์ สถิติ และคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยอุบลราชธานีทุกท่าน ที่ประสิทธิ์ประสาทวิชาความรู้ตลอดหลักสูตรการศึกษา

สุดท้ายนี้ ขอขอบพระคุณครอบครัวที่คอยสนับสนุน ให้กำลังใจ และเป็นแรงผลักดันให้ผู้จัดทำสามารถทำโครงการนี้จนสำเร็จลุล่วง

นายกิตติธัช เอมรัตน์

6 กุมภาพันธ์ 2569

โครงการงาน	:	ระบบจัดการแข่งขัน Bracket Tournament พร้อมระบบโหวต Real-time BattleHub: Bracket Tournament System with Realtime Voting
โดย	:	นายกิตติธัช เอมรัตน์
อาจารย์ที่ปรึกษา	:	อาจารย์วาโย ปุยะติ
ระดับการศึกษา	:	วิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์
ปีการศึกษา	:	2567

บทคัดย่อ

โครงการนี้พัฒนาระบบ BattleHub ซึ่งเป็นเว็บแอปพลิเคชันสำหรับจัดการแข่งขันแบบ Bracket Tournament พร้อมระบบโหวต Real-time โดยมีวัตถุประสงค์เพื่อแก้ปัญหการจัดการแข่งขันแบบดั้งเดิมที่ใช้เวลานานและขาดความตื่นเต้น ระบบพัฒนาด้วย Django Framework ใช้ฐานข้อมูล PostgreSQL และ Deploy ด้วย Docker

ระบบมีความสามารถหลักได้แก่ การสร้างทัวร์นาเมนต์แบบ Single Elimination, การอัปโหลดผู้เข้าแข่งขันแบบ Bulk Upload, ห้องรอแบบ Kahoot-style ที่เข้าร่วมด้วย PIN Code 6 หลัก, ระบบโหวตแบบ Real-time พร้อม Timer ที่ Sync กับ Server, Toast Notifications แจ้งเตือนเหตุการณ์สำคัญ และการเปลี่ยนรอบอัตโนมัติเมื่อจบแมตช์

ผลการทดสอบพบว่าระบบสามารถทำงานได้ตรงตามความต้องการ ผ่านการทดสอบฟังก์ชันทั้ง 14 รายการ และสามารถ Deploy บน Docker ได้สำเร็จ

คำสำคัญ: Bracket Tournament, Real-time Voting, Django, Docker, Kahoot-style Lobby

Topic	:	BattleHub: Bracket Tournament Management System with Real-time
Author	:	Kittituch Aimrat
Advisor	:	Wayo Puyati
Degree	:	Bachelor of Science (Data Science and Software Innovation)
Academic Year	:	2024

Abstract

This project develops BattleHub, a web application for managing bracket tournaments with real-time voting system. The objective is to solve traditional competition management problems which are time-consuming and lack excitement. The system is developed using Django Framework, PostgreSQL database, and deployed with Docker.

The main features include Single Elimination tournament creation, Bulk Upload for competitors, Kahoot-style lobby with 6-digit PIN Code, Real-time voting with server-synced timer, Toast Notifications for important events, and auto-redirect when matches end.

Testing results show that the system functions as required, passing all 14 functional test cases and successfully deploying on Docker.

Keywords: Bracket Tournament, Real-time Voting, Django, Docker, Kahoot-style Lobby

สารบัญ

กิตติกรรมประกาศ	3
บทคัดย่อ	4
Abstract	5
สารบัญ	6
สารบัญภาพ	10
บทที่ 1	
บทนำ	11
1.1 ความเป็นมาและความสำคัญของปัญหา	11
1.2 วัตถุประสงค์	12
1.3 ขอบเขตการดำเนินงาน	12
1.3.1 ขอบเขตด้านฟังก์ชัน	12
1.3.2 ขอบเขตด้านเทคโนโลยี	12
1.3.3 ขอบเขตด้านผู้ใช้	12
1.4 ประโยชน์ที่คาดว่าจะได้รับ	13
1.5 เครื่องมือที่ใช้ในการพัฒนา	13
1.5.1 ฮาร์ดแวร์	13
1.5.2 ซอฟต์แวร์	13
1.6 แผนการดำเนินการ	14
บทที่ 2	
ทฤษฎีที่เกี่ยวข้อง	15
2.1 Django Framework	15
2.2 Docker และ Containerization	15
2.3 PostgreSQL Database	16
2.4 AJAX Polling และ Real-time Web	16
2.5 Tailwind CSS	17
2.6 ระบบ Bracket Tournament	17
2.7 Kahoot-style Lobby	17
2.8 งานวิจัยที่เกี่ยวข้อง	18
2.8.1 ระบบจัดการการแข่งขันออนไลน์	18
2.8.2 ระบบโหวตออนไลน์	18
บทที่ 3	
การออกแบบระบบ	19
3.1 สถาปัตยกรรมระบบ	19
3.2 ความต้องการของระบบ	21
3.2.1 ความต้องการด้านฟังก์ชัน (Functional Requirements)	21

3.2.2 ความต้องการด้านอื่นๆ (Non-Functional Requirements)	22
3.3 การออกแบบหน้าจอ (Screen Design)	22
3.3.1 หน้ารายการทัวร์นาเมนต์ (Tournament List Page)	23
3.3.2 หน้ารายละเอียดทัวร์นาเมนต์ (Tournament Detail Page)	24
3.3.3 หน้าสมัครสมาชิก (Register Page)	25
3.3.4 หน้าเข้าสู่ระบบ (Login Page)	25
3.3.5 หน้าสร้างทัวร์นาเมนต์และอัปโหลดผู้เข้าแข่งขัน (Create Tournament & Upload Competitors)	26
3.3.6 หน้าเข้าร่วมด้วย PIN (Join via PIN Page)	27
3.3.7 หน้าห้องพักรอ (Waiting Lobby Page)	28
3.3.8 หน้าโหวต (Play / Vote Page)	29
3.3.9 หน้าสรุปผลการแข่งขัน (Summary / Results Page)	30
3.3.10 หน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard Page)	31
3.4 การออกแบบแผนภาพระบบ (System Diagrams)	32
3.4.1 แผนภาพกรณีการใช้งาน (Use Case Diagram)	32
รายละเอียดกรณีการใช้งาน (Use Case Descriptions)	39
ส่วนที่ 1: Guest (ผู้เยี่ยมชม)	39
ส่วนที่ 2: Member (สมาชิก)	43
ส่วนที่ 3: Admin (ผู้ดูแลระบบ)	58
3.5 แผนภาพคลาส Class Diagram	65
3.5.1 Backend Class Diagram	66
3.5.1.2 Tournaments App	68
3.5.1.3 Custom Admin App	71
3.5.1.4 Full Backend Class Diagram (ภาพรวมความเชื่อมโยง)	73
3.5.2 Frontend Class Diagram	75
3.6 แผนภาพลำดับ (Sequence Diagram)	77
2. สัญลักษณ์ในแผนภาพลำดับ (Sequence Diagram)	77
3.6.1 Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow)	79
3.6.1.1 ส่วนการเข้าถึงข้อมูล (Browsing)	79
3.6.1.2 ส่วนการยืนยันตัวตน (Authentication)	81
3.6.2 Sequence Diagram ส่วนสมาชิก (Member Flow)	83
3.6.2.1 ส่วนการจัดการทัวร์นาเมนต์ (Management)	83
3.6.2.2 ส่วนการเข้าร่วมและรอแข่งขัน (Lobby Phase)	85
3.6.2.3 ส่วนการแข่งขันจริง (Gameplay & Real-time)	87
3.6.3 Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow)	89
3.6.3.1 ส่วนการจัดการระบบและผู้ใช้งาน (System & User Management)	89
3.6.3.2 ส่วนการจัดการเนื้อหาและตรวจสอบ (Content Moderation & Audit)	91

3.7 แบบจำลองข้อมูล (Data Model)	93
3. สัญลักษณ์ในแบบจำลองข้อมูล (ER Diagram)	93
3.7.1 Entity Relationship Diagram (ERD)	96
3.7.2 พจนานุกรมข้อมูล (Data Dictionary)	97
1. ตาราง auth_user (Users)	97
2. ตาราง tournaments_tournament (Tournaments)	98
3. ตาราง tournaments_match (Matches)	99
บทที่ 4	
การพัฒนาระบบ	100
4.1 โครงสร้างแอปพลิเคชันและการตั้งค่า (Project Configuration)	100
4.1.1 การจัดการ Settings.py สำหรับการเชื่อมต่อฐานข้อมูลและ Static Files	100
4.1.2 การออกแบบระบบ URL Routing (Urls.py) แบบแยกแอปพลิเคชัน	102
4.2 โครงสร้างไฟล์และโฟลเดอร์ของระบบ (Django Directory Structure)	103
4.2.1 โครงสร้างระดับโครงการ (Project Level: battlehub/) และหน้าที่ของไฟล์ WSGI/ASGI	103
4.2.2 โครงสร้างระดับแอปพลิเคชัน (App Level: accounts, tournaments, custom_admin)	103
4.2.3 หน้าที่ของไฟล์สำคัญในแต่ละแอปพลิเคชัน	104
4.2.4 การจัดหมวดหมู่ไฟล์เทมเพลต (Templates Hierarchy) และไฟล์สถิต (Static Files)	104
4.3 การพัฒนาฟังก์ชันการทำงานด้วย Django (Core Implementation)	105
4.3.1 การขยายความสามารถของ User Model ผ่าน Profile Model	105
4.3.2 การเขียน Logic ใน Views.py (Core Business Logic)	106
1. SD-01: การดูรายการทัวร์นาเมนต์ (Guest Browsing)	106
2. SD-02: การสมัครสมาชิก (Guest Authentication - Sign Up)	107
3. SD-03: การสร้างทัวร์นาเมนต์ (Member Tournament Management)	108
4. SD-04: ระบบห้องพักรอ (Member Lobby System)	109
5. SD-05: ระบบโหวตและการแข่งขัน (Gameplay & Voting Logic)	111
6. SD-06: การจัดการผู้ใช้โดยผู้ดูแลระบบ (Admin User Management)	112
7. SD-07: การจัดการเนื้อหาโดยผู้ดูแลระบบ (Admin Content Management)	114
4.3.3 การสร้างและจัดการฟอร์มด้วย Django Forms (Forms.py)	115
บทที่ 5	
การทดสอบระบบ	116
5.1 การทดสอบฟังก์ชันการทำงาน (Functional Testing)	117
5.2 การทดสอบพีเจียร์พิเศษ (Kahoot-style & Admin)	118
5.3 การทดสอบประสิทธิภาพและความปลอดภัย (Non-Functional Testing)	120
5.4 การยืนยันการติดตั้งระบบ (Deployment Verification)	121
5.4.1 สถานะ Docker Containers	121
5.4.2 ช่องทางเข้าใช้งานจริง (Access Points)	121
5.5 สรุปผลการทดสอบ	122

บทที่ 6	
สรุปและข้อเสนอแนะ	123
6.1 สรุปความสามารถของระบบ	123
6.2 ปัญหาและอุปสรรคในการพัฒนา	124
6.3 แนวทางในการพัฒนาต่อ	125
บรรณานุกรม	126
1) เอกสารคู่มือและเอกสารอ้างอิงของเทคโนโลยี (Official Documentation)	126
ภาคผนวก	127
ภาคผนวก ก	
การติดตั้งเครื่องมือที่ใช้พัฒนาโปรแกรม	128
ก.1 การติดตั้ง Visual Studio Code	128
ก.2 การติดตั้ง Python	129
ก.3 การติดตั้ง Docker	130
ภาคผนวก ข	
คู่มือการติดตั้งระบบ	131
ข.1 การเตรียมสภาพแวดล้อม (Prerequisites)	131
ข.2 การติดตั้งด้วย Docker Compose (แนะนำ)	131
ข.3 การใช้งานระบบผ่าน Ngrok (Public Access)	132
ข.4 การแก้ไขปัญหาเบื้องต้น (Troubleshooting)	133
ภาคผนวก ค	
คู่มือการใช้งานของระบบ	134

สารบัญภาพ

รูปที่	ชื่อภาพ	หน้า
บทที่ 3	การออกแบบระบบ	21
3.1	แผนภาพสถาปัตยกรรมระบบ (System Architecture)	22
3.2	Wireframe หน้ารายการทัวร์นาเมนต์	25
3.3	Wireframe หน้ารายละเอียดทัวร์นาเมนต์	26
3.4	Wireframe หน้าสมัครสมาชิก	27
3.5	Wireframe หน้าเข้าสู่ระบบ	27
3.6	Wireframe หน้าสร้างทัวร์นาเมนต์และอัปโหลดผู้เข้าแข่งขัน	28
3.7	Wireframe หน้าเข้าร่วมทัวร์นาเมนต์ผ่านรหัส PIN	29
3.8	Wireframe หน้าห้องพักรอ	30
3.9	Wireframe หน้าโหวต	31
3.10	Wireframe หน้าสรุปผลการแข่งขัน	32
3.11	Wireframe หน้าแดชบอร์ดผู้ดูแลระบบ	33
3.12	แผนภาพกรณีการใช้งาน (Use Case Diagram)	36
3.13	แผนภาพคลาส Backend — Accounts App	69
3.14	แผนภาพคลาส Backend — Tournaments App	73
3.15	แผนภาพคลาส Backend — Custom Admin App	74
3.16	แผนภาพคลาส Backend ภาพรวม (แสดงความเชื่อมโยงระหว่าง App)	76
3.17	แผนภาพคลาส Frontend	78
3.18	Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow Part 1) — การเข้าถึงข้อมูล	82
3.19	Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow Part 2) — การยืนยันตัวตน	84
3.20	Sequence Diagram ส่วนสมาชิก (Member Flow Part 1) — การจัดการทัวร์นาเมนต์	86
3.21	Sequence Diagram ส่วนสมาชิก (Member Flow Part 2) — การเข้าร่วมและรอแข่งขัน	88
3.22	Sequence Diagram ส่วนสมาชิก (Member Flow Part 3) — การแข่งขันจริง	90

3.23	Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow Part 1) — การจัดการระบบ	92
3.24	Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow Part 2) — การจัดการเนื้อหาและตรวจสอบ	94
3.25	แผนภาพความสัมพันธ์ของข้อมูล (Entity Relationship Diagram)	98
ภาค ผนวก ก	การติดตั้งเครื่องมือ	130
ก.1	หน้าต่างเว็บไซต์สำหรับดาวน์โหลด Visual Studio Code	130
ก.2	หน้าต่างโปรแกรม Visual Studio Code เมื่อติดตั้งเสร็จสมบูรณ์	131
ก.3	หน้าต่างตัวเลือกการติดตั้ง Python (Add Python to PATH)	131
ก.4	หน้าต่าง Command Line แสดงเวอร์ชันของ Python ที่ติดตั้งแล้ว	132
ก.5	หน้าต่างโปรแกรม Docker Desktop แสดงสถานะ Engine Running	132
ภาค ผนวก ค	คู่มือการใช้งาน	136
ค.1	หน้าสมัครสมาชิก	136
ค.2	หน้าจอเข้าร่วมผ่านรหัส PIN (Join via PIN)	137
ค.3	หน้าจอการโหวต	137
ค.4	หน้าจอการสร้างทัวร์นาเมนต์	138
ค.5	หน้าจอการอัปโหลดผู้เข้าแข่งขันแบบกลุ่ม (Bulk Upload)	139
ค.6	หน้าจอห้องพักรอ (Waiting Lobby)	140
ค.7	หน้าจอ Admin Dashboard	141

สารบัญตาราง

ตารางที่	ชื่อตาราง	หน้า
1.1	แผนการดำเนินการพัฒนาระบบ	14
3.1	คำอธิบายสัญลักษณ์แผนภาพ Use Case Diagram	32
3.2	สรุปภาพรวมกรณีการใช้งาน (Use Case Summary)	35
3.3	คำอธิบายสัญลักษณ์แผนภาพ Class Diagram	65
3.4	คำอธิบายสัญลักษณ์แผนภาพ Sequence Diagram	77
3.5	คำอธิบายสัญลักษณ์แบบจำลองข้อมูล (ER Diagram)	93
3.6	พจนานุกรมข้อมูล: ตาราง auth_user (Users)	97
3.7	พจนานุกรมข้อมูล: ตาราง tournaments_tournament	98
3.8	พจนานุกรมข้อมูล: ตาราง tournaments_match	99
5.1	ผลการทดสอบฟังก์ชันหลัก (General User Flow)	117
5.2	ผลการทดสอบระบบ Lobby และ Real-time Features	119
5.3	ผลการทดสอบระบบผู้ดูแลระบบ (Admin Features)	120
5.4	ผลการทดสอบประสิทธิภาพและความปลอดภัย	121

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบันการจัดการแข่งขันประเภท Bracket Tournament เช่น การแข่งขันโหวตภาพ การแข่งขันเกม หรือการประกวดต่างๆ มีความนิยมเพิ่มขึ้นอย่างต่อเนื่อง อย่างไรก็ตาม การจัดการแข่งขันดังกล่าวยังคงต้องอาศัยวิธีการแบบดั้งเดิม เช่น การใช้กระดาษ การนับคะแนนด้วยมือ หรือการใช้ซอฟต์แวร์ที่ไม่รองรับการโหวตแบบ Real-time ทำให้เกิดปัญหาหลายประการ ได้แก่

- 1) ความล่าช้าในการนับคะแนน ผู้จัดต้องรวบรวมคะแนนด้วยมือซึ่งใช้เวลานานและอาจเกิดข้อผิดพลาด
- 2) ขาดความตื่นเต้น ผู้ชมไม่สามารถเห็นผลคะแนนแบบ Real-time ระหว่างการแข่งขัน ทำให้ขาดความมีส่วนร่วม
- 3) ยากต่อการจัดการผู้เข้าร่วม การจัดการรายชื่อผู้เข้าแข่งขันและผู้ชมต้องทำด้วยมือ ไม่มีระบบอัตโนมัติ
- 4) ไม่รองรับการแข่งขันออนไลน์ ระบบเดิมส่วนใหญ่ออกแบบมาสำหรับการแข่งขันแบบพบหน้า ไม่รองรับผู้เข้าร่วมจากระยะไกล

ผู้พัฒนาจึงได้เล็งเห็นถึงโอกาสในการพัฒนาระบบ BattleHub ซึ่งเป็นเว็บแอปพลิเคชันสำหรับจัดการแข่งขันแบบ Knockout Tournament พร้อมระบบโหวต Real-time และ Kahoot-style Lobby ที่ทันสมัย เพื่อแก้ไขปัญหาดังกล่าวและเพิ่มประสบการณ์ที่ดีให้กับผู้ใช้งาน

1.2 วัตถุประสงค์

- 1) เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับสร้างและจัดการการแข่งขันแบบ Knockout Bracket Tournament
- 2) เพื่อพัฒนาระบบโหวตแบบ Real-time ที่แสดงผลคะแนนทันทีระหว่างการแข่งขัน
- 3) เพื่อพัฒนาระบบห้องรอ (Lobby) แบบ Kahoot-style ที่ผู้เล่นสามารถเข้าร่วมด้วย PIN Code
- 4) เพื่อพัฒนา Admin Dashboard สำหรับผู้ดูแลระบบในการจัดการทัวร์นาเมนต์และผู้ใช้

1.3 ขอบเขตการดำเนินงาน

1.3.1 ขอบเขตด้านฟังก์ชัน

- ระบบลงทะเบียนและเข้าสู่ระบบ
- ระบบสร้างทัวร์นาเมนต์ รองรับขนาด bracket 2, 4, 8, 16 คน
- ระบบอัปโหลดรูปผู้เข้าแข่งขันแบบ Bulk Upload (ลากและวาง)
- ระบบห้องรอ (Lobby) พร้อม PIN Code 6 หลัก
- ระบบโหวตแบบ Real-time พร้อม Timer นับถอยหลัง
- ระบบแจ้งเตือน (Toast Notification)
- ระบบเปลี่ยนรอบอัตโนมัติ (Auto-redirect)
- Admin Dashboard

1.3.2 ขอบเขตด้านเทคโนโลยี

- Frontend: HTML5, Tailwind CSS, JavaScript (Vanilla)
- Backend: Python 3.11, Django 5.0
- Database: PostgreSQL 15
- Deployment: Docker, Docker Compose, Nginx, Gunicorn

1.3.3 ขอบเขตด้านผู้ใช้

- Guest: ผู้เยี่ยมชมที่ยังไม่ได้ login
- Member: สมาชิกที่ลงทะเบียนแล้ว
- Admin: ผู้ดูแลระบบ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1) ผู้จัดการแข่งขันสามารถสร้างและจัดการทัวร์นาเมนต์ได้อย่างสะดวกและรวดเร็ว
- 2) ผู้เข้าร่วมสามารถโหวตและเห็นผลคะแนนแบบ Real-time เพิ่มความตื่นเต้นและมีส่วนร่วม
- 3) รองรับการแข่งขันแบบออนไลน์ ผู้เข้าร่วมไม่จำเป็นต้องอยู่ในสถานที่เดียวกัน
- 4) ลดภาระการทำงานของผู้จัด ระบบจัดการ bracket และนับคะแนนอัตโนมัติ
- 5) เป็นแนวทางในการพัฒนาเว็บแอปพลิเคชันแบบ Real-time สำหรับผู้ที่สนใจศึกษา

1.5 เครื่องมือที่ใช้ในการพัฒนา

1.5.1 ฮาร์ดแวร์

- 1) คอมพิวเตอร์ส่วนบุคคล สำหรับพัฒนาและทดสอบระบบ
 - CPU: AMD Ryzen 5 5600X
 - RAM: 16 GB
 - GPU: NVIDIA GeForce GTX 1080 Ti
 - SSD: 2 TB
- 2) เครื่อง Server สำหรับ Deploy ระบบ Production
 - สามารถใช้ Cloud Server เช่น AWS EC2, DigitalOcean หรือ Local Server

1.5.2 ซอฟต์แวร์

- 1) VS Code สำหรับเขียนและแก้ไขโค้ด
- 2) Python 3.11 Interpreter สำหรับรัน Backend
- 3) PostgreSQL 15 ระบบจัดการฐานข้อมูล
- 4) Docker และ Docker Compose สำหรับ Containerization
- 5) Nginx สำหรับ Reverse Proxy
- 6) Git สำหรับ Version Control
- 7) Google Chrome / Firefox สำหรับทดสอบหน้าเว็บ
- 8) Postman สำหรับทดสอบ API

1.6 แผนการดำเนินการ

ในการพัฒนาระบบ BattleHub มีการแบ่งการดำเนินการออกเป็น 4 ระยะ ดังนี้

ตารางที่ 1.1 แผนการดำเนินการพัฒนาระบบ

ระยะ (Phase)	กิจกรรมดำเนินงาน	ระยะเวลา	ช่วงเวลาดำเนินการ
1. การวางแผนและรวบรวมข้อมูล	ศึกษาขอบเขตของระบบจัดการแข่งขัน (Tournament), รวบรวม Requirement จากผู้ใช้งาน และศึกษาเทคโนโลยี React/Node.js	เอกสารวิเคราะห์ความต้องการ (SRS) และสรุปฟีเจอร์หลักของระบบ Battle Hub	สัปดาห์ที่ 1-2
2. การวิเคราะห์และออกแบบระบบ	ออกแบบ Use Case, Class Diagram, ER Diagram และออกแบบหน้าจอ (UI/UX) ด้วย Figma	แผนภาพวิเคราะห์ระบบ (UML), โครงสร้างฐานข้อมูล PostgreSQL และ Wireframe	สัปดาห์ที่ 3-5
3. การพัฒนาระบบสารสนเทศ	ติดตั้ง Environment (Docker), พัฒนา Backend (Node.js), พัฒนา Frontend (React) และเชื่อมต่อฐานข้อมูล	ซอฟต์แวร์ระบบ Battle Hub ที่สามารถจัดทัวร์นาเมนต์และโหวตได้จริง	สัปดาห์ที่ 6-12
4. การทดสอบและการจัดทำเอกสาร	ทดสอบระบบ (Unit & Integration Testing), แก้ไข Bug, และจัดทำคู่มือการใช้งาน/ติดตั้ง	รายงานผลการทดสอบ, เล่มโครงสร้างงานฉบับสมบูรณ์ และคู่มือผู้ใช้งาน (User Manual)	สัปดาห์ที่ 13-16

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 Django Framework

Django เป็น Web Framework ภาษา Python ที่ใช้สถาปัตยกรรมแบบ MTV (Model-Template-View) โดยมีคุณสมบัติเด่นคือ "Batteries included" หมายความว่ามาพร้อมเครื่องมือสำเร็จรูปมากมาย เช่น ระบบ Authentication, ORM, Admin Panel และ Form Validation

ในโครงการนี้ใช้ Django เวอร์ชัน 5.0 ซึ่งมีคุณสมบัติที่เหมาะสมกับการพัฒนา ได้แก่

- Django ORM สำหรับจัดการฐานข้อมูลโดยไม่ต้องเขียน SQL โดยตรง
- Django Templates สำหรับสร้างหน้าเว็บ
- Django Auth สำหรับระบบ Login/Logout
- CSRF Protection สำหรับป้องกันการโจมตีแบบ Cross-Site Request Forgery

2.2 Docker และ Containerization

Docker เป็นเทคโนโลยี Containerization ที่ช่วยให้สามารถ package แอปพลิเคชันพร้อม dependencies ทั้งหมดเป็น Container ที่สามารถรันได้ทุกเครื่องที่มี Docker โดยไม่ต้องกังวลเรื่อง environment ที่แตกต่างกัน

ข้อดีของการใช้ Docker:

- Consistency: รันได้เหมือนกันทุกเครื่อง
- Isolation: แต่ละ Container แยกกันอิสระ
- Scalability: สามารถเพิ่มจำนวน Container ได้ง่าย
- Easy Deployment: Deploy ได้รวดเร็วด้วยคำสั่งเดียว

ในโครงการนี้ใช้ Docker Compose จัดการ 3 containers:

- 1) db: PostgreSQL database
- 2) web: Django application บน Gunicorn
- 3) nginx: Reverse proxy server

2.3 PostgreSQL Database

PostgreSQL เป็น Open-source Relational Database ที่มีความเสถียรและรองรับ features ขั้นสูง เหมาะสำหรับ Production environment

ข้อดีของ PostgreSQL เมื่อเปรียบเทียบกับ SQLite:

- รองรับ Concurrent connections หลายคนพร้อมกัน
- มี ACID compliance ที่แข็งแกร่ง
- รองรับ JSON data type
- มี Extensions มากมาย

ในโครงการนี้ใช้ PostgreSQL 15 Alpine (lightweight version)

2.4 AJAX Polling และ Real-time Web

AJAX (Asynchronous JavaScript and XML) เป็นเทคนิคที่ช่วยให้หน้าเว็บสามารถสื่อสารกับ Server โดยไม่ต้อง refresh หน้าใหม่ทั้งหมด

รูปแบบ Real-time ที่ใช้ในโครงการนี้คือ Polling โดย JavaScript จะส่ง request ไปยัง Server ทุก 2 วินาที เพื่อดึงข้อมูลล่าสุด เช่น จำนวน vote และเวลาที่เหลือ

ข้อดีของ Polling:

- ง่ายต่อการ implement ไม่ต้องใช้เทคโนโลยีเพิ่มเติม
- ทำงานได้กับทุก browser
- ไม่ต้องการ persistent connection

ข้อเสียของ Polling:

- มี overhead จากการส่ง request ซ้ำๆ
- มี delay เล็กน้อย (latency ตาม interval ที่กำหนด)

2.5 Tailwind CSS

Tailwind CSS เป็น Utility-first CSS Framework ที่ให้ class เล็กๆ สำหรับ style แต่ละ property โดยตรง แทนที่จะเป็น component สำเร็จรูปเหมือน Bootstrap

ข้อดีของ Tailwind CSS:

- Highly customizable: ปรับแต่งได้ทุกส่วน
- No unused CSS: build เฉพาะ class ที่ใช้จริง
- Responsive design: มี responsive prefix เช่น md:, lg:
- เหมาะกับ Dark theme: มี class dark: สำหรับ dark mode

ในโครงการนี้ใช้ Tailwind CSS CDN เพื่อความสะดวกในการพัฒนา

2.6 ระบบ Bracket Tournament

Bracket Tournament (หรือ Knockout Tournament) เป็นรูปแบบการแข่งขันที่ผู้แพ้จะถูกคัดออกทันที ผู้ชนะจะเข้าสู่รอบถัดไปจนเหลือผู้ชนะคนสุดท้าย

โครงสร้าง Bracket แบบ Single Elimination:

- รอบที่ 1: n คน แข่งกัน ได้ผู้ชนะ $n/2$ คน
- รอบที่ 2: $n/2$ คน แข่งกัน ได้ผู้ชนะ $n/4$ คน
- รอบสุดท้าย: 2 คน แข่งกัน ได้แชมป์ 1 คน

จำนวนรอบ = $\log_2(\text{จำนวนผู้เข้าแข่งขัน})$

- 2 คน = 1 รอบ
- 4 คน = 2 รอบ
- 8 คน = 3 รอบ
- 16 คน = 4 รอบ

2.7 Kahoot-style Lobby

Kahoot เป็นแพลตฟอร์ม Game-based Learning ที่มีชื่อเสียงด้านการออกแบบ UX สำหรับการเข้าร่วมกิจกรรมแบบง่ายตาย โดยใช้ระบบ PIN Code

แนวคิดหลักของ Kahoot-style Lobby:

- 1) Host สร้าง session และได้รับ PIN Code
- 2) ผู้เล่นเข้าเว็บ → กรอก PIN → กรอก Nickname
- 3) Host เห็นรายชื่อผู้เข้าร่วมแบบ Real-time
- 4) Host กด Start เมื่อพร้อม

ในโครงงานนี้ได้นำแนวคิดดังกล่าวมาประยุกต์ใช้กับระบบ Tournament

2.8 งานวิจัยที่เกี่ยวข้อง

2.8.1 ระบบจัดการการแข่งขันออนไลน์

มีหลายแพลตฟอร์มที่ให้บริการจัดการ Tournament เช่น Challonge, Toornament อย่างไรก็ตามแพลตฟอร์มเหล่านี้ไม่ได้เน้นระบบโหวต Real-time หรือ Kahoot-style Lobby

2.8.2 ระบบโหวตออนไลน์

มีงานวิจัยหลายชิ้นที่พัฒนาระบบโหวตออนไลน์ เช่น ระบบเลือกตั้ง ระบบประเมินผล แต่ส่วนใหญ่ไม่ได้ออกแบบมาสำหรับการแข่งขันแบบ Bracket

โครงงาน BattleHub จึงพัฒนาขึ้นเพื่อรวมจุดเด่นของทั้ง Tournament Management และ Real-time Voting ไว้ในแพลตฟอร์มเดียว

บทที่ 3

การออกแบบระบบ

3.1 สถาปัตยกรรมระบบ

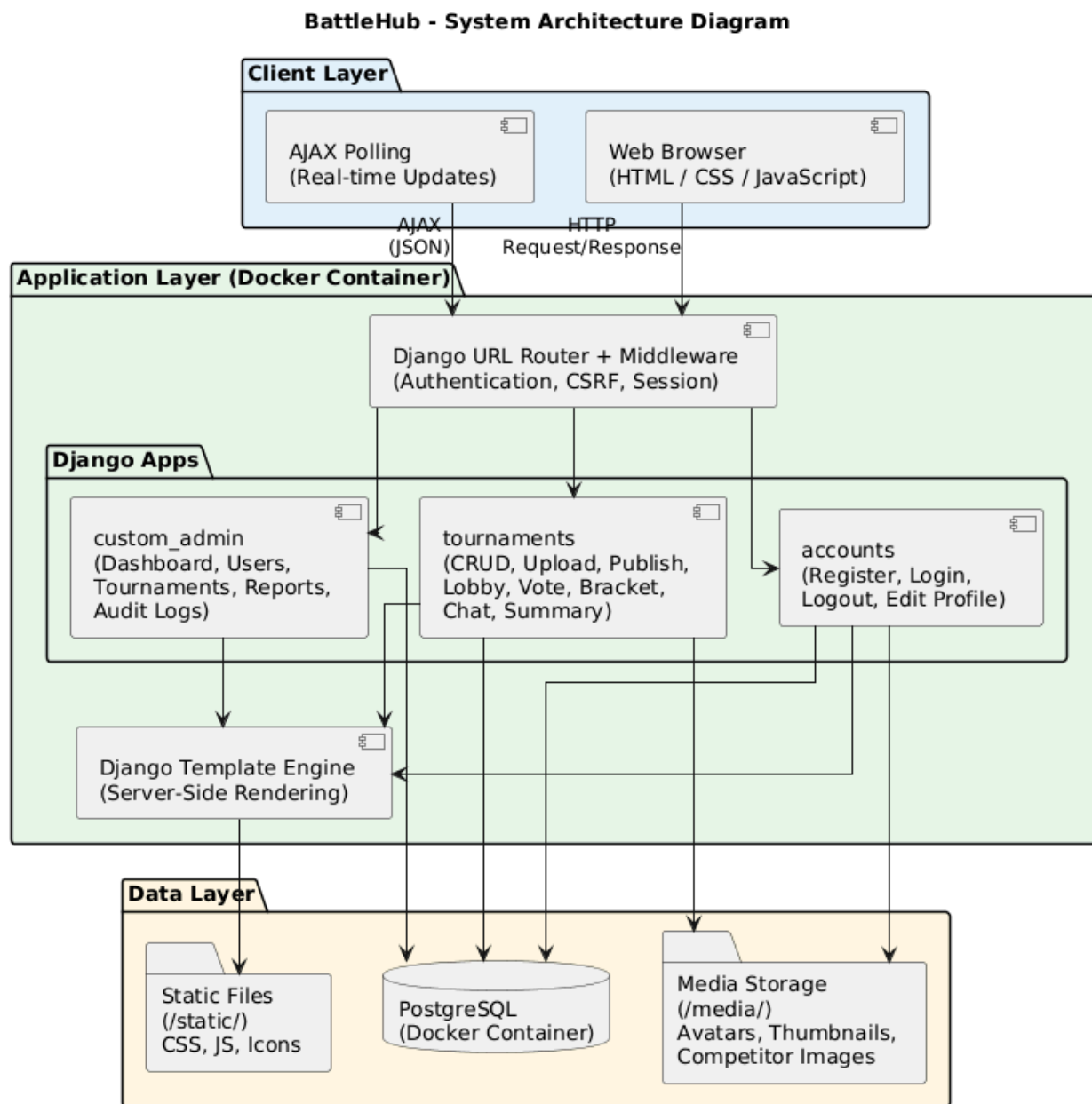
ระบบ BattleHub ใช้สถาปัตยกรรมแบบ Containerized ผ่าน Docker เพื่อให้ง่ายต่อการ deploy และ scale ในอนาคต โดยแบ่งออกเป็น 3 ส่วนหลัก ได้แก่ Web Server, Application Server และ Database Server

โครงสร้างการทำงานของระบบมีลำดับดังนี้

ผู้ใช้ (Browser) → Nginx → Docker Container (Gunicorn/Django) → PostgreSQL

รายละเอียดแต่ละ Layer มีดังนี้

- 1) Client Layer คือส่วนที่ผู้ใช้เข้าถึงผ่าน Browser โดยหน้า Frontend ใช้ HTML5 สำหรับโครงสร้าง, Tailwind CSS สำหรับจัดรูปแบบ และ JavaScript สำหรับ AJAX polling เพื่อดึงข้อมูลแบบ real-time
- 2) Web Server Layer คือ Nginx ทำหน้าที่เป็น reverse proxy รับ request ที่ port 80 และส่งต่อไปยัง application server รวมถึงให้บริการ static files และ media files
- 3) Application Layer คือ Django framework ทำงานบน Gunicorn WSGI server ภายใน Docker container ทำหน้าที่จัดการเรื่อง authentication, business logic และ API endpoints
- 4) Database Layer คือ PostgreSQL ที่ทำหน้าที่เก็บข้อมูลผู้ใช้ ทวีร์นาเมนต์ ผู้เข้าแข่งขัน และผลโหวตทั้งหมด



(รูปที่ 3.1 แผนภาพสถาปัตยกรรมระบบ)

คำอธิบายภาพ: จากภาพที่ 3.2 แสดงสถาปัตยกรรมของระบบ BattleHub ซึ่งถูกออกแบบในลักษณะ Web Application โดยแบ่งโครงสร้างการทำงานออกเป็น 3 ส่วนหลัก (Three-Tier Architecture) และทำงานอยู่ภายใต้สภาพแวดล้อมจำลอง (Containerized Environment) เพื่อความสะดวกในการติดตั้งและย้ายระบบ ดังนี้

1. ส่วนแสดงผล (Client Layer): ทำหน้าที่เป็นส่วนติดต่อกับผู้ใช้งาน (User Interface) ผ่านเว็บเบราว์เซอร์ (Web Browser) โดยใช้เทคโนโลยี HTML, CSS และ JavaScript ในการแสดงผล และมีการ

ใช้เทคนิค AJAX เพื่อรับส่งข้อมูลกับเซิร์ฟเวอร์ในรูปแบบ JSON แบบ Real-time (เช่น การอัปเดตสถานะในหน้า Lobby หรือหน้าโหวต) โดยไม่ต้องรีโหลดหน้าจอใหม่

2. ส่วนประมวลผล (Application Layer): ทำหน้าที่เป็นแกนหลักในการทำงานของระบบ พัฒนาด้วย Django Framework (ภาษา Python) ซึ่งทำงานอยู่ภายใน Docker Container โดยแบ่งโมดูลภายในตามหน้าที่การทำงาน ได้แก่
 - Accounts: จัดการเรื่องการยืนยันตัวตน (Authentication) และข้อมูลส่วนตัวผู้ใช้
 - Tournaments: จัดการตรรกะหลักของระบบ เช่น การสร้างทัวร์นาเมนต์ การจับคู่แข่งขัน และการโหวต
 - Custom Admin: ส่วนจัดการระบบสำหรับผู้ดูแล (Dashboard และ Report) นอกจากนี้ ยังมีส่วนของ Template Engine ทำหน้าที่ประมวลผลหน้าเว็บก่อนส่งกลับไปให้ส่วนแสดงผล
3. ส่วนข้อมูล (Data Layer): ทำหน้าที่จัดเก็บข้อมูลของระบบ ประกอบด้วย
 - Database: ใช้ระบบจัดการฐานข้อมูล PostgreSQL (รันบน Docker) สำหรับเก็บข้อมูลเชิงสัมพันธ์ เช่น ข้อมูลผู้ใช้, ข้อมูลการแข่งขัน และผลโหวต
 - Media Storage: พื้นที่สำหรับจัดเก็บไฟล์มัลติมีเดียที่ผู้ใช้อัปโหลด เช่น รูปโปรไฟล์ (Avatar) และรูปผู้เข้าแข่งขัน (Competitors)
 - Static Files: พื้นที่จัดเก็บไฟล์สไตล์ชีต (CSS) และสคริปต์ (JS) ขอ

บันทึกการเปลี่ยนแปลง Tech Stack

เวอร์ชัน 1.0 (มกราคม 2569) ได้ทำการเปลี่ยนจาก SQLite เป็น PostgreSQL เนื่องจาก PostgreSQL รองรับ concurrent users ได้ดีกว่าใน production environment โดยในช่วงพัฒนา local ยังคงใช้ SQLite เพื่อความสะดวก แต่ production ใช้ PostgreSQL ผ่าน Docker

3.2 ความต้องการของระบบ

3.2.1 ความต้องการด้านฟังก์ชัน (Functional Requirements)

FR-01 ระบบลงทะเบียนและเข้าสู่ระบบ

ผู้ใช้สามารถสมัครสมาชิก เข้าสู่ระบบ และออกจากระบบได้ โดยรหัสผ่านจะถูกเข้ารหัสด้วย PBKDF2

FR-02 สร้างทัวร์นาเมนต์

สมาชิกสามารถสร้างทัวร์นาเมนต์แบบ knockout ได้ โดยกำหนดขนาด bracket ได้ 2, 4, 8 หรือ 16 คน

FR-03 อัปโหลดผู้เข้าแข่งขัน

ระบบรองรับการอัปโหลดรูปหลายรูปพร้อมกัน (Bulk Upload) ผ่านการลากและวาง (Drag and Drop)

FR-04 ระบบโหวต real-time

ผู้ใช้โหวตแล้วเห็นผลทันทีผ่าน AJAX polling โดย browser จะดึงข้อมูลจาก server ทุก 2 วินาที

FR-05 ระบบ Admin Dashboard

ผู้ดูแลระบบสามารถดูสถิติภาพรวม จัดการทัวร์นาเมนต์ และจัดการผู้ใช้ได้

FR-06 ระบบ Kahoot-style Lobby

ผู้สร้างทัวร์นาเมนต์สามารถเปิดห้องรอ (Lobby) พร้อม PIN Code 6 หลัก ผู้เล่นสามารถเข้าร่วมด้วยการกรอก PIN และ Nickname โดย Host จะเห็นรายชื่อผู้เข้าร่วมแบบ Real-time

FR-07 ระบบ Timer แบบ Server-synced

ระบบมีนาฬิกานับถอยหลังที่ sync กับ Server ป้องกันผู้ใช้โกงเวลา เมื่อหมดเวลาระบบจะตัดสินผู้ชนะอัตโนมัติ

FR-08 ระบบ Toast Notifications

ระบบแจ้งเตือนแบบ Real-time เช่น "เหลือเวลา 10 วินาที", "หมดเวลา!", "มีผู้เล่นเข้าร่วม!" เพื่อเพิ่มประสบการณ์ผู้ใช้

FR-09 ระบบ Auto-redirect

เมื่อจบแมตช์หรือทัวร์นาเมนต์ ระบบจะเปลี่ยนหน้าอัตโนมัติไปยังหน้ารอบถัดไปหรือหน้าสรุปผล โดยไม่ต้องให้ผู้ใช้กด refresh

3.2.2 ความต้องการด้านอื่นๆ (Non-Functional Requirements)

NFR-01 ความเร็ว

ระบบต้องโหลดหน้าไม่เกิน 2 วินาที

NFR-02 ความปลอดภัย

ระบบต้องมี CSRF protection ทุกฟอร์ม

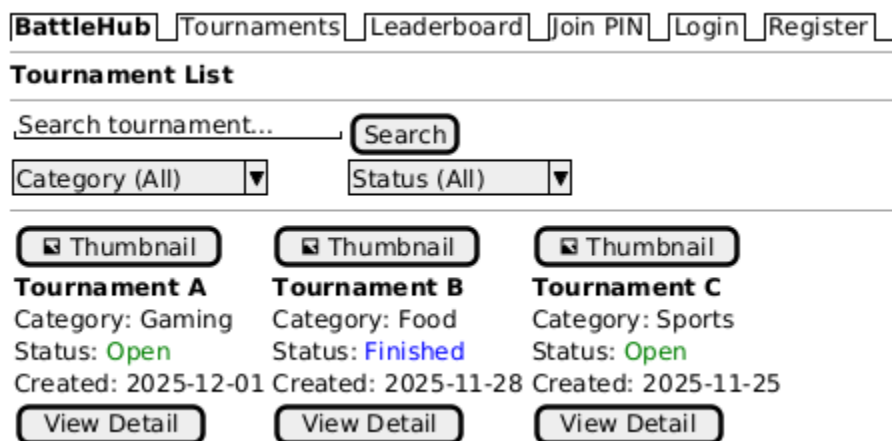
NFR-03 Responsive Design

หน้าเว็บต้องใช้งานได้ทั้ง Desktop และ Mobile

3.3 การออกแบบหน้าจอ (Screen Design)

การออกแบบหน้าจอของระบบ BattleHub ถูกจัดทำขึ้นในรูปแบบ Wireframe เพื่อแสดงโครงสร้างและองค์ประกอบของส่วนต่อประสานกับผู้ใช้ (User Interface) โดยนำเสนอเฉพาะหน้าจอหลักที่ผู้ใช้งานมีปฏิสัมพันธ์โดยตรง ดังนี้

3.3.1 หน้ารายการทัวร์นาเมนต์ (Tournament List Page)



(รูปที่ 3.2 Wireframe หน้ารายการทัวร์นาเมนต์)

คำอธิบายภาพ: จากภาพที่ 3.2 แสดง Wireframe ของหน้ารายการทัวร์นาเมนต์ ซึ่งเป็นหน้าแรกของระบบ BattleHub โดยแบ่งโครงสร้างหน้าจออกเป็น 3 ส่วนหลัก ดังนี้ ส่วนบนสุดเป็นแถบนำทาง (Navbar) ประกอบด้วยเมนูสำหรับเข้าถึงหน้าต่าง ๆ ได้แก่ Tournaments, Leaderboard, Join PIN รวมถึงปุ่ม Login และ Register สำหรับผู้เยี่ยมชม ส่วนถัดมาเป็นพื้นที่ค้นหาและตัวกรอง (Search & Filter) ประกอบด้วยช่องค้นหาด้วยคำ Dropdown สำหรับเลือกหมวดหมู่ และ Dropdown สำหรับเลือกสถานะ เพื่อให้ผู้ใช้งานสามารถค้นหาทัวร์นาเมนต์ที่ต้องการได้อย่างรวดเร็ว ส่วนหลักแสดงรายการทัวร์นาเมนต์ในรูปแบบการ์ด (Card Layout) โดยแต่ละการ์ดประกอบด้วยรูปปก ชื่อทัวร์นาเมนต์ หมวดหมู่ สถานะ วันที่สร้าง และปุ่ม "View Detail" สำหรับเข้าถึงหน้ารายละเอียด

3.3.2 หน้ารายละเอียดทัวร์นาเมนต์ (Tournament Detail Page)

[BattleHub](#)
[Tournaments](#)
[Leaderboard](#)
[Join PIN](#)
[Profile](#)
[Logout](#)

Tournament Detail

 Tournament Thumbnail

Best Anime Character 2025

Description: Vote for your favorite anime character
 Category: Anime
 Bracket Size: 8
 Vote Duration: 30 seconds
 Status: Open
 Created by: user123

[Edit](#)
[Delete](#)
[Open Lobby](#)

Competitors (8/8)

 Char1
Naruto

 Char2
Goku

 Char3
Luffy

 Char4
Ichigo

Comments

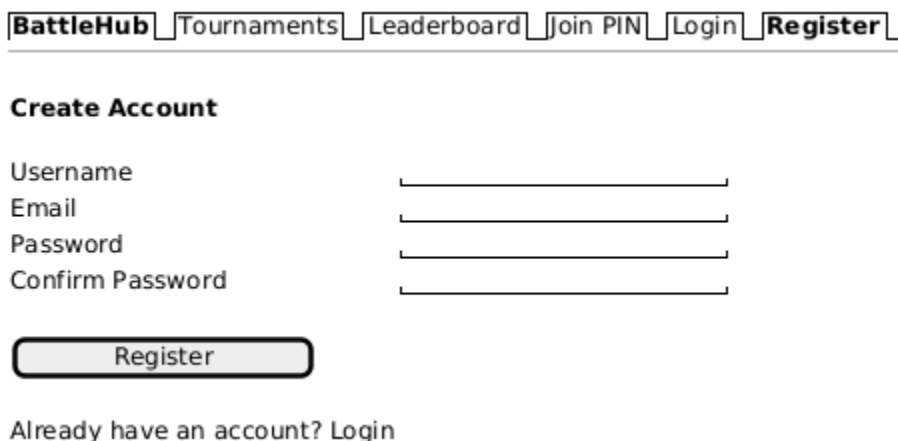
user456 - 2025-12-01
 Great tournament! [Report]

[Submit](#)

(รูปที่ 3.3 Wireframe หน้ารายละเอียดทัวร์นาเมนต์)

คำอธิบายภาพ: จากภาพที่ 3.3 แสดง Wireframe ของหน้ารายละเอียดทัวร์นาเมนต์ ซึ่งเป็นหน้าจอศูนย์กลางข้อมูลของทัวร์นาเมนต์แต่ละรายการ โดยแบ่งออกเป็น 3 ส่วนหลัก ดังนี้ ส่วนบนแสดงข้อมูลทั่วไปของทัวร์นาเมนต์ ประกอบด้วยรูปปก (Thumbnail) ทางด้านซ้าย และรายละเอียดทางด้านขวา ได้แก่ ชื่อ คำอธิบาย หมวดหมู่ ขนาด Bracket ระยะเวลาโหวต สถานะ และชื่อผู้สร้าง หากผู้ใช้งานเป็นเจ้าของทัวร์นาเมนต์ ปุ่มจัดการ (Edit, Delete, Open Lobby) จะถูกแสดงผลเพิ่มเติม ส่วนกลางแสดงรายชื่อผู้เข้าแข่งขัน (Competitors) พร้อมรูปภาพ โดยแสดงจำนวนปัจจุบันเทียบกับจำนวนที่กำหนด ส่วนล่างเป็นพื้นที่ความคิดเห็น (Comments) ผู้ใช้งานสามารถพิมพ์ความคิดเห็น กดปุ่ม Submit เพื่อส่ง และกดปุ่ม Report เพื่อรายงานความคิดเห็นที่ไม่เหมาะสมได้

3.3.3 หน้าสมัครสมาชิก (Register Page)

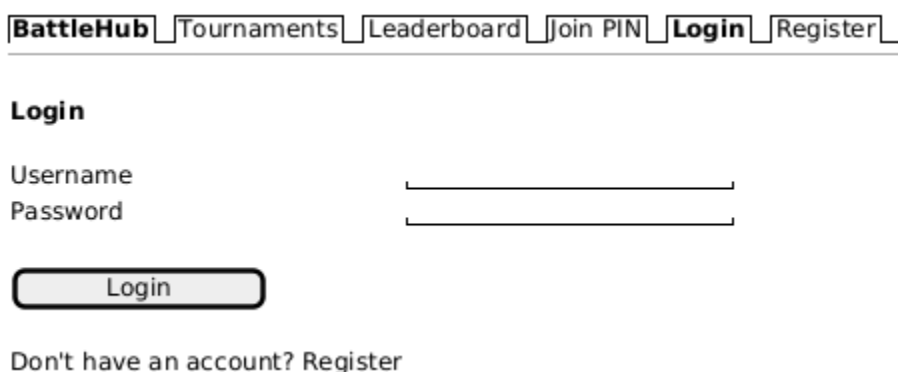


The wireframe shows a navigation bar at the top with links: BattleHub, Tournaments, Leaderboard, Join PIN, Login, and Register. Below the navigation bar is a section titled "Create Account". It contains four input fields: Username, Email, Password, and Confirm Password. Below these fields is a "Register" button. At the bottom of the section, there is a link: "Already have an account? [Login](#)".

(รูปที่ 3.4 Wireframe หน้าสมัครสมาชิก)

คำอธิบายภาพ: จากภาพที่ 3.4 แสดง Wireframe ของหน้าสมัครสมาชิก ซึ่งออกแบบให้เรียบง่ายและเน้นการใช้งานเป็นหลัก ตรงกลางหน้าจอประกอบด้วยแบบฟอร์มลงทะเบียน 4 ช่อง ได้แก่ ชื่อผู้ใช้ (Username) อีเมล (Email) รหัสผ่าน (Password) และยืนยันรหัสผ่าน (Confirm Password) ด้านล่างของแบบฟอร์มมีปุ่ม "Register" สำหรับยืนยันการสมัคร เมื่อกรอกข้อมูลครบถ้วนและข้อมูลผ่านการตรวจสอบความถูกต้อง ระบบจะสร้างบัญชีผู้ใช้และนำส่งไปยังหน้าเข้าสู่ระบบ นอกจากนี้ยังมีลิงก์ "Login" สำหรับผู้ที่มีบัญชีอยู่แล้วเพื่อดำเนินการเข้าสู่ระบบได้โดยตรง

3.3.4 หน้าเข้าสู่ระบบ (Login Page)



The wireframe shows a navigation bar at the top with links: BattleHub, Tournaments, Leaderboard, Join PIN, Login, and Register. Below the navigation bar is a section titled "Login". It contains two input fields: Username and Password. Below these fields is a "Login" button. At the bottom of the section, there is a link: "Don't have an account? [Register](#)".

(รูปที่ 3.5 Wireframe หน้าเข้าสู่ระบบ)

คำอธิบายภาพ: จากภาพที่ 3.5 แสดง Wireframe ของหน้าเข้าสู่ระบบ ประกอบด้วยแบบฟอร์มสำหรับกรอกชื่อผู้ใช้ (Username) และรหัสผ่าน (Password) ด้านล่างมีปุ่ม "Login" สำหรับยืนยันการเข้าสู่ระบบ เมื่อ

ข้อมูลถูกส่งไปยังเซิร์ฟเวอร์ ระบบจะดำเนินการตรวจสอบข้อมูลยืนยันตัวตน (Authentication) และตรวจสอบสถานะบัญชี หากข้อมูลถูกต้องและบัญชีไม่ถูกระงับ ระบบจะสร้าง Session และนำส่งผู้ใช้งานไปยังหน้ารายการทัวร์นาเมนต์ในสถานะสมาชิก หากข้อมูลไม่ถูกต้องหรือบัญชีถูกระงับ ข้อความแจ้งเตือนที่เหมาะสมจะถูกแสดงผลในส่วนแบบฟอร์ม ด้านล่างสุดมีลิงก์ "Register" สำหรับผู้ที่ยังไม่มีบัญชีเพื่อดำเนินการสมัครสมาชิก

3.3.5 หน้าสร้างทัวร์นาเมนต์และอัปโหลดผู้เข้าแข่งขัน (Create Tournament & Upload Competitors)

(รูปที่ 3.6 Wireframe หน้าสร้างทัวร์นาเมนต์และอัปโหลดผู้เข้าแข่งขัน)

คำอธิบายภาพ: จากภาพที่ 3.6 แสดง Wireframe ของหน้าสร้างทัวร์นาเมนต์และอัปโหลดผู้เข้าแข่งขัน ซึ่งเป็น 2 ขั้นตอนที่เกี่ยวข้องกันในกระบวนการสร้างทัวร์นาเมนต์ ส่วนบนเป็นแบบฟอร์มสร้างทัวร์นาเมนต์ ประกอบด้วย ช่องกรอกชื่อทัวร์นาเมนต์ ช่องกรอกคำอธิบาย Dropdown สำหรับเลือกหมวดหมู่ Dropdown สำหรับเลือกขนาด Bracket (2, 4, 8 หรือ 16 คน) ช่องกำหนดระยะเวลาโหวต (วินาที) และปุ่มอัปโหลดรูปปก เมื่อคลิกปุ่ม "Create Tournament" ทัวร์นาเมนต์จะถูกสร้างในสถานะ "Draft" ส่วนล่างเป็นหน้าอัปโหลดผู้เข้า

แข่งขัน แสดง Progress Bar สำหรับติดตามจำนวนปัจจุบันเทียบกับจำนวนที่ต้องการ รายการผู้เข้าแข่งขันที่อัปโหลดแล้วพร้อมปุ่มลบ (X) สำหรับลบรายการ พื้นที่ลากวาง (Drag & Drop) สำหรับอัปโหลดรูปภาพ ปุ่ม "Upload Competitors" สำหรับยืนยันการอัปโหลด และปุ่ม "Open Waiting Lobby" ที่จะแสดงผลเมื่อจำนวนผู้เข้าแข่งขันครบตามขนาด Bracket Size ที่กำหนด

3.3.6 หน้าเข้าร่วมด้วย PIN (Join via PIN Page)

BattleHub | Tournaments | Leaderboard | **Join PIN** | Profile | Logout

Join Tournament

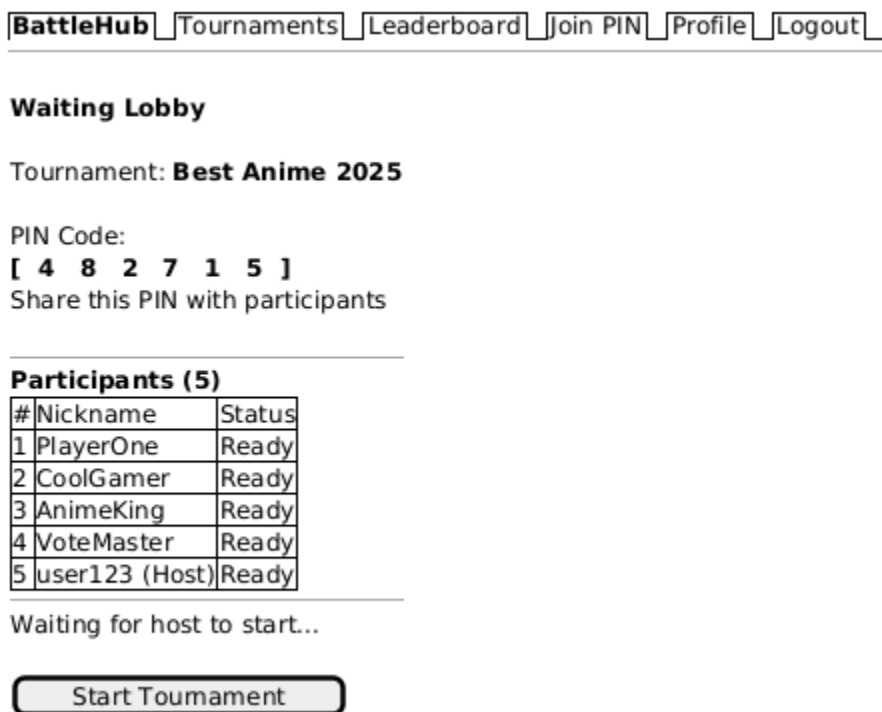
Enter the 6-digit PIN code
from the tournament host:

Join

(รูปที่ 3.7 Wireframe หน้าเข้าร่วมทัวร์นาเมนต์ผ่านรหัส PIN)

คำอธิบายภาพ: จากภาพที่ 3.7 แสดง Wireframe ของหน้าเข้าร่วมทัวร์นาเมนต์ผ่านรหัส PIN ซึ่งเป็นฟีเจอร์หลักของระบบที่ได้รับแรงบันดาลใจจาก Kahoot ออกแบบให้เรียบง่ายและเป็นศูนย์กลางหน้าจอ ประกอบด้วยข้อความคำแนะนำ "Enter the 6-digit PIN code from the tournament host" ช่องกรอกรหัส PIN 6 หลัก และปุ่ม "Join" สำหรับยืนยันการเข้าร่วม เมื่อผู้ใช้งานกรอกรหัส PIN และกดปุ่ม Join ระบบจะดำเนินการตรวจสอบรหัสกับฐานข้อมูล หากรหัสถูกต้องและทัวร์นาเมนต์อยู่ในสถานะ "Waiting" ผู้ใช้งานจะถูกนำส่งไปยังหน้ากรอกชื่อเล่น หากรหัสไม่ถูกต้องข้อความแจ้งเตือนจะถูกแสดงผล

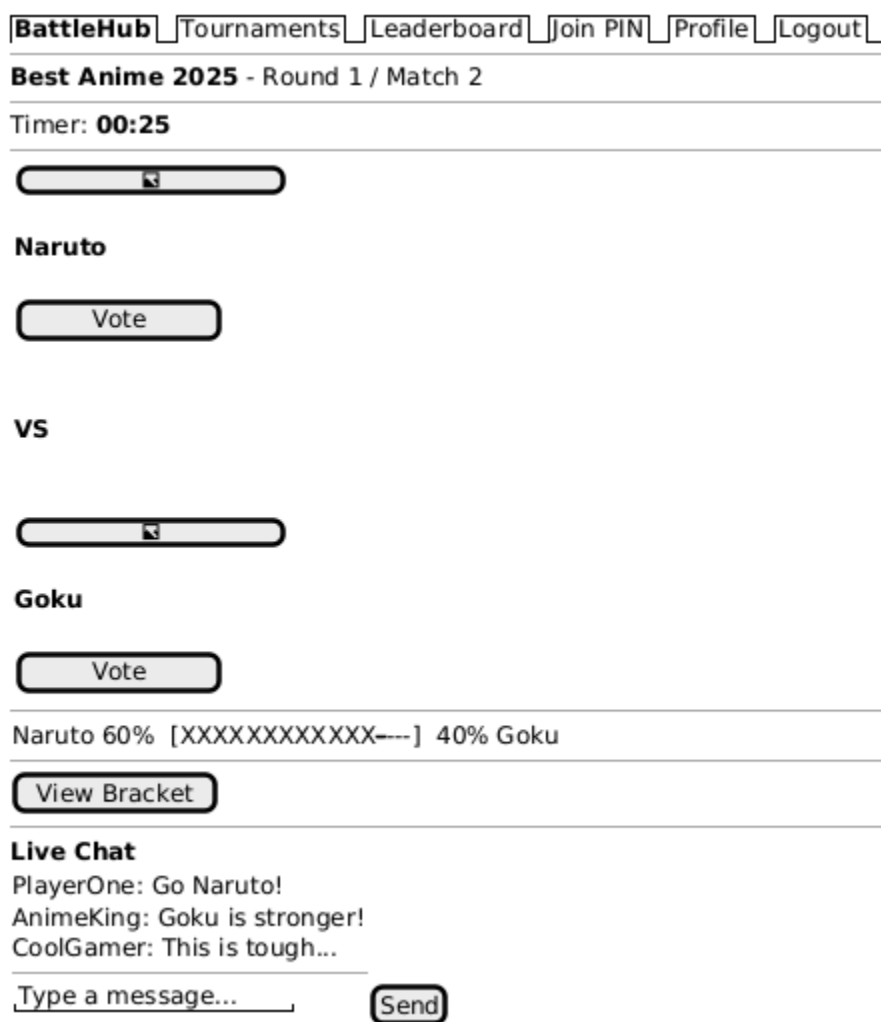
3.3.7 หน้าห้องพักรอ (Waiting Lobby Page)



(รูปที่ 3.8 Wireframe หน้าห้องพักรอ)

คำอธิบายภาพ: จากภาพที่ 3.8 แสดง Wireframe ของหน้าห้องพักรอ (Waiting Lobby) ซึ่งเป็นหน้าจอที่ผู้เข้าร่วมทุกคนจะเห็นหลังจากกรอกชื่อเล่นเรียบร้อยแล้ว โดยแบ่งออกเป็น 3 ส่วนหลัก ดังนี้ ส่วนบนแสดงชื่อทัวร์นาเมนต์และรหัส PIN 6 หลักอย่างเด่นชัด เพื่อให้เจ้าของทัวร์นาเมนต์สามารถแจกจ่ายรหัส PIN ให้ผู้เข้าร่วมคนอื่นได้สะดวก ส่วนกลางแสดงตารางรายชื่อผู้เข้าร่วม (Participants) ที่อัปเดตข้อมูลแบบ Real-time ทุก ๆ 3 วินาทีผ่าน AJAX Polling เมื่อมีผู้เข้าร่วมใหม่เข้ามา ชื่อจะถูกแสดงผลในตารางโดยอัตโนมัติ ส่วนล่างแสดงข้อความ "Waiting for host to start..." สำหรับผู้เข้าร่วมทั่วไป และปุ่ม "Start Tournament" ที่แสดงเฉพาะสำหรับเจ้าของทัวร์นาเมนต์เท่านั้น เมื่อเจ้าของกดปุ่มเริ่ม ผู้เข้าร่วมทุกคนจะถูกนำส่งไปยังหน้าโหวตโดยอัตโนมัติ

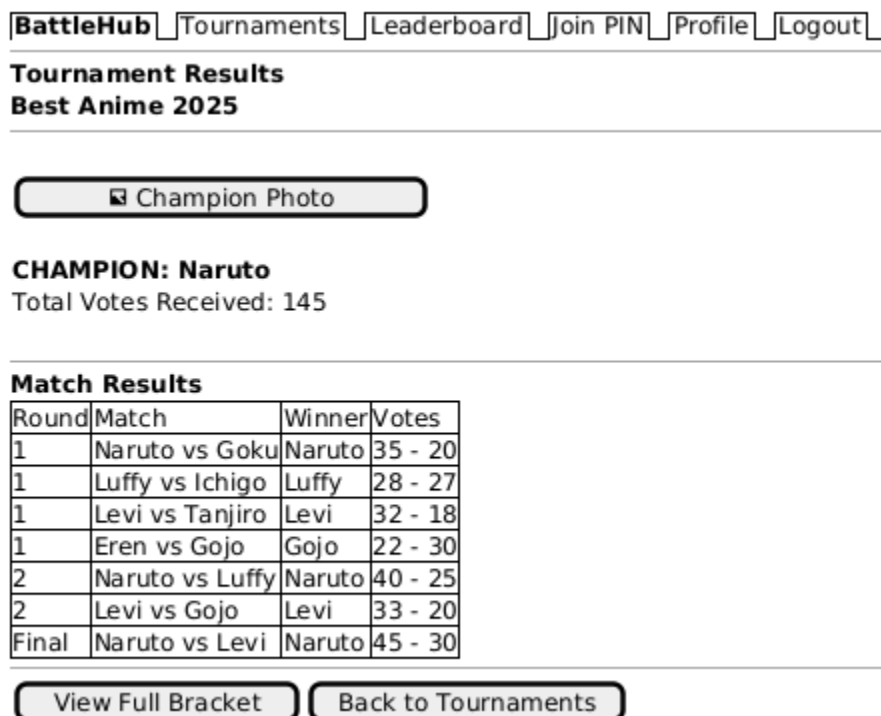
3.3.8 หน้าโหวต (Play / Vote Page)



(รูปที่ 3.9 Wireframe หน้าโหวต)

คำอธิบายภาพ: จากภาพที่ 3.9 แสดง Wireframe ของหน้าโหวต ซึ่งเป็นหน้าจอที่สำคัญที่สุดของระบบ BattleHub เนื่องจากเป็นพื้นที่ที่ผู้เข้าร่วมดำเนินการกิจกรรมหลักของระบบ โดยแบ่งออกเป็น 5 ส่วนหลัก ดังนี้ ส่วนบนสุดแสดงชื่อทัวร์นาเมนต์ รอบการแข่งขัน (Round) และลำดับแมตช์ (Match) ถัดมาเป็นตัวนับเวลาถอยหลัง (Countdown Timer) ที่แสดงเวลาคงเหลือสำหรับการโหวตในแมตช์ปัจจุบัน ส่วนกลางเป็นพื้นที่หลักแสดงรูปภาพและชื่อของผู้เข้าแข่งขัน 2 คนที่กำลังแข่งขันพร้อมปุ่ม "Vote" สำหรับลงคะแนน ด้านล่างแสดงแถบเปอร์เซ็นต์คะแนนโหวต (Vote Bar) ที่อัปเดตแบบ Real-time เพื่อให้ผู้เข้าร่วมเห็นสัดส่วนคะแนนขณะโหวต และปุ่ม "View Bracket" สำหรับดูสายการแข่งขัน ส่วนล่างสุดเป็นพื้นที่แชทสด (Live Chat) สำหรับสื่อสารระหว่างผู้เข้าร่วมในระหว่างการแข่งขัน ประกอบด้วยพื้นที่แสดงข้อความ ช่องพิมพ์ข้อความ และปุ่ม "Send"

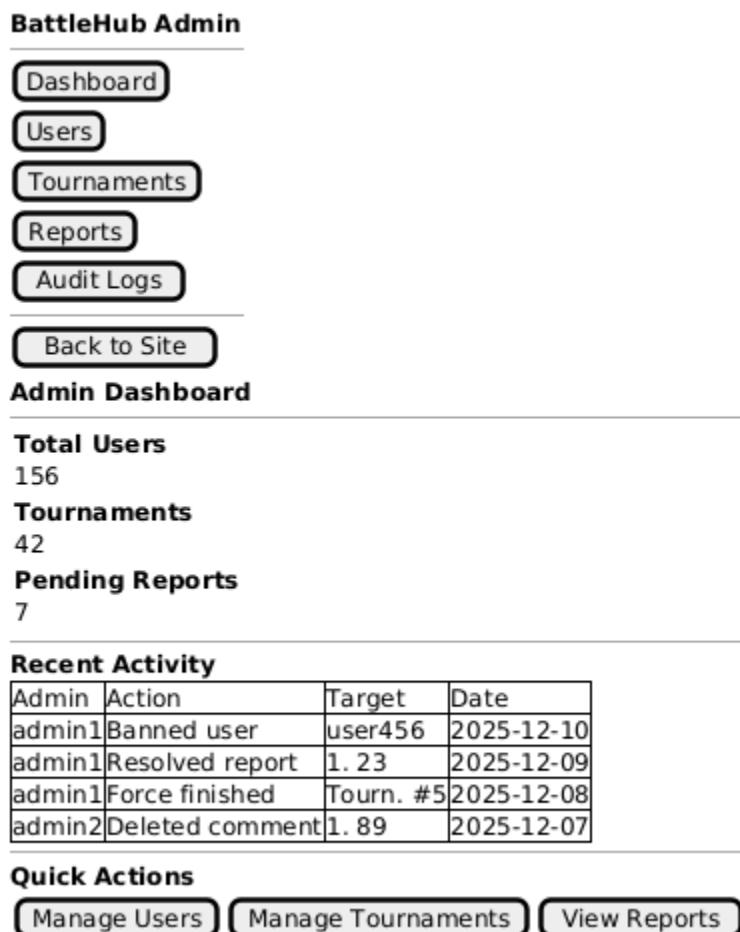
3.3.9 หน้าสรุปผลการแข่งขัน (Summary / Results Page)



(รูปที่ 3.10 Wireframe หน้าสรุปผลการแข่งขัน)

คำอธิบายภาพ: จากภาพที่ 3.10 แสดง Wireframe ของหน้าสรุปผลการแข่งขัน ซึ่งถูกแสดงผลโดยอัตโนมัติเมื่อทัวร์นาเมนต์ดำเนินการจนครบทุกรอบและเสร็จสิ้น โดยแบ่งออกเป็น 3 ส่วนหลัก ดังนี้ ส่วนบนแสดงรูปภาพและชื่อของผู้ชนะเลิศ (Champion) พร้อมจำนวนคะแนนโหวตรวมทั้งหมดที่ได้รับตลอดการแข่งขัน ส่วนกลางแสดงตารางผลการแข่งขันทุกรอบ โดยแต่ละแถวประกอบด้วย รอบ คู่แข่งขัน ผู้ชนะ และคะแนนโหวตของทั้งสองฝ่าย เพื่อให้ผู้ใช้งานสามารถทบทวนผลของทุกแมตช์ได้โดยละเอียด ส่วนล่างมีปุ่ม "View Full Bracket" สำหรับดูสายการแข่งขันเต็มในรูปแบบแผนผังต้นไม้ และปุ่ม "Back to Tournaments" สำหรับกลับไปยังหน้ารายการทัวร์นาเมนต์

3.3.10 หน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard Page)



(รูปที่ 3.11 Wireframe หน้าแดชบอร์ดผู้ดูแลระบบ)

คำอธิบายภาพ: จากภาพที่ 3.11 แสดง Wireframe ของหน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard) ซึ่งเป็นหน้าจอหลักของ Admin Panel ที่เข้าถึงได้เฉพาะผู้ดูแลระบบ (`is_staff = true`) เท่านั้น โดยจัดวาง Layout ในรูปแบบ 2 คอลัมน์ ดังนี้ ด้านซ้ายเป็นแถบเมนูด้านข้าง (Sidebar Navigation) สำหรับเข้าถึงส่วนจัดการต่าง ๆ ได้แก่ Dashboard สำหรับดูภาพรวม Users สำหรับจัดการผู้ใช้งาน Tournaments สำหรับจัดการทัวร์นาเมนต์ Reports สำหรับจัดการรายงานปัญหา Audit Logs สำหรับตรวจสอบประวัติการดำเนินการ และปุ่ม "Back to Site" สำหรับกลับไปยังหน้าเว็บไซต์หลัก ด้านขวาเป็นพื้นที่เนื้อหาหลัก แบ่งเป็น 3 ส่วน ส่วนบนแสดงการ์ดสรุปสถิติ (Stats Cards) ประกอบด้วยจำนวนผู้ใช้งานทั้งหมด จำนวนทัวร์นาเมนต์ และจำนวนรายงานที่รอดำเนินการ ส่วนกลางแสดงตารางกิจกรรมล่าสุด (Recent Activity) จาก Audit Log ส่วนล่างมีปุ่มลัด (Quick Actions) สำหรับเข้าถึงหน้าจัดการหลักได้โดยตรง

3.4 การออกแบบแผนภาพระบบ (System Diagrams)

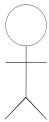
ในการวิเคราะห์และออกแบบระบบ BattleHub (ระบบจัดการแข่งขันและโทวตอนออนไลน์) ได้มีการนำแผนภาพในตระกูล Unified Modeling Language (UML) มาใช้เพื่อเป็นสื่อกลางในการอธิบายโครงสร้างและพฤติกรรมของระบบ โดยเน้นการอธิบายความสัมพันธ์ระหว่างผู้ใช้งานและหน้าที่ต่าง ๆ ของระบบผ่านแผนภาพกรณีการใช้งาน (Use Case Diagram)

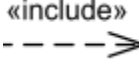
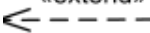
3.4.1 แผนภาพกรณีการใช้งาน (Use Case Diagram)

แผนภาพกรณีการใช้งานถูกจัดทำขึ้นเพื่อแสดงขอบเขตของระบบ (System Boundary) และปฏิสัมพันธ์ระหว่างตัวแสดงแทน (Actors) กับฟังก์ชันการทำงานต่าง ๆ โดยในระบบ BattleHub ถูกแบ่งกลุ่มผู้ใช้งานออกเป็น 3 กลุ่มหลัก ได้แก่ ผู้เยี่ยมชม (Guest), สมาชิก (Member) และผู้ดูแลระบบ (Admin)

1. สัญลักษณ์ที่ใช้ในแผนภาพ (Symbol Descriptions)
เพื่อให้เกิดความเข้าใจที่ตรงกันในการอ่านแผนภาพ จึงมีการกำหนดความหมายของสัญลักษณ์ที่ใช้ตามมาตรฐาน UML ดังนี้

ตารางที่ 3.1 คำอธิบายแผนภาพ Use Case Diagram

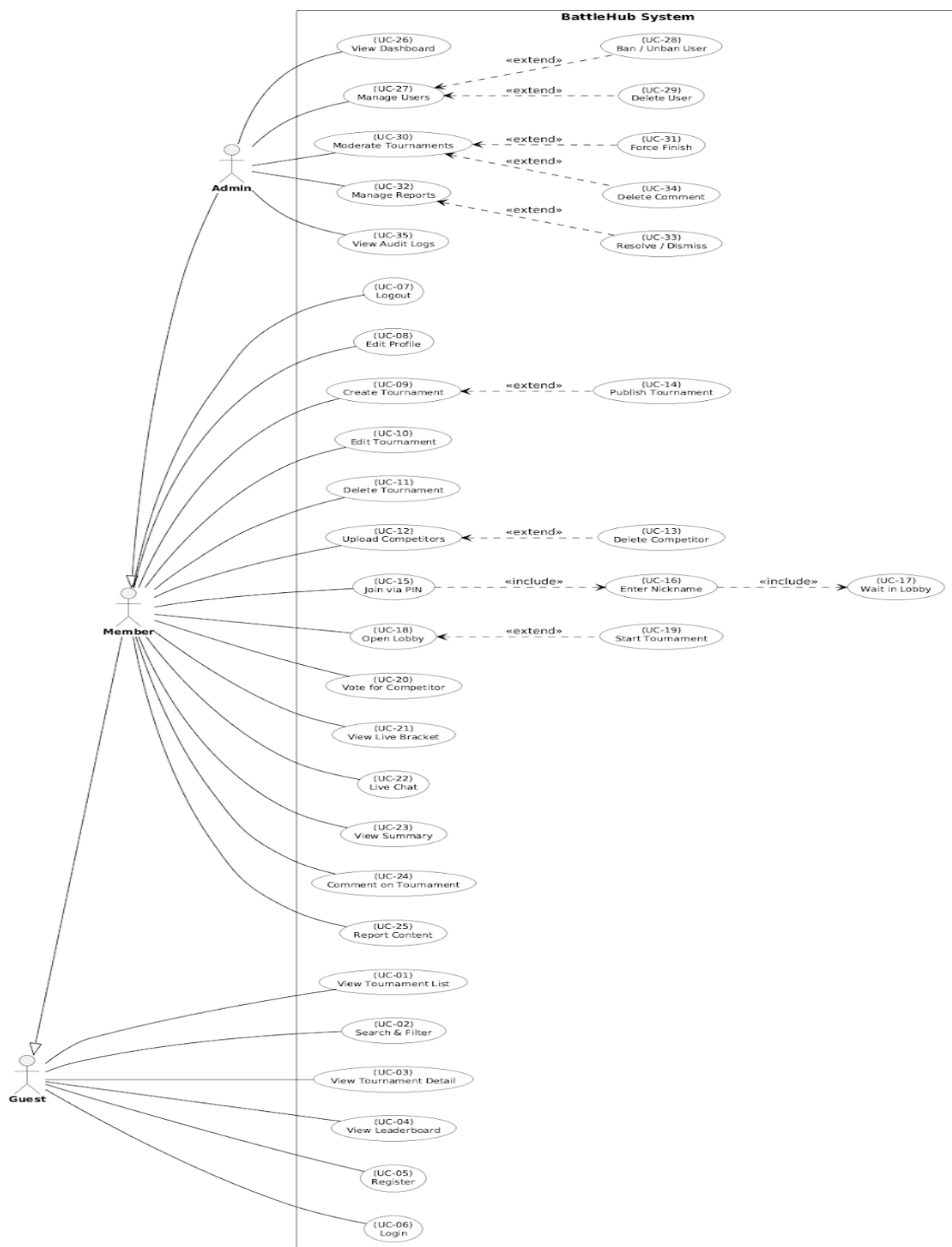
สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
	ตัวแสดงแทน (Actor)	แทนบทบาทของผู้ใช้งานภายนอกที่มีปฏิสัมพันธ์กับระบบ (Guest, Member, Admin)
	กรณีการใช้งาน (Use Case)	แทนฟังก์ชันหรือกระบวนการทำงานภายในระบบที่ให้ผลลัพธ์แก่ผู้ใช้งาน
	ขอบเขตระบบ (System Boundary)	เส้นกรอบสี่เหลี่ยมที่กำหนดขอบเขตระหว่างสิ่งที่อยู่ภายในระบบและภายนอกระบบ

	เส้นสมาคม (Association)	เส้นที่เชื่อมโยงระหว่าง Actor และ Use Case เพื่อแสดงว่ามีส่วนเกี่ยวข้องกัน
	> (Arrow Line)	การสืบทอดสิทธิ์ (Generalization)
	ความสัมพันธ์แบบรวม	แสดงกรณีการใช้งานที่ "จำเป็น" ต้องทำเป็นส่วนหนึ่งของกรณีการใช้งานหลักเสมอ
	ความสัมพันธ์แบบส่วนขยาย	แสดงกรณีการใช้งานที่เป็น "ทางเลือก" หรือส่วนเสริมจากกรณีการใช้งานหลัก

(รูปที่ 3.1 ตาราง)

2. รายละเอียดกรณีการใช้งานของระบบ (Use Case Grouping) จากการวิเคราะห์ระบบ สามารถระบุกลุ่ม Use Case หลักที่สัมพันธ์กับ Actor แต่ละประเภทได้ดังนี้:

- กลุ่มกรณีการใช้งานสำหรับผู้เยี่ยมชม (Guest Use Cases): ครอบคลุมฟังก์ชันพื้นฐานที่บุคคลทั่วไปสามารถเข้าถึงได้โดยไม่ต้องผ่านการยืนยันตัวตน ได้แก่ การเรียกดูรายการทัวร์นาเมนต์ การค้นหาและกรองข้อมูล การดูรายละเอียดการแข่งขัน การดูอันดับผู้ใช้งาน (Leaderboard) รวมถึงกระบวนการลงทะเบียนสมาชิกและการเข้าสู่ระบบเพื่อรับสิทธิ์การใช้งานที่สูงขึ้น
- กลุ่มกรณีการใช้งานสำหรับสมาชิก (Member Use Cases): ครอบคลุมฟังก์ชันการจัดการทัวร์นาเมนต์ที่ตนเองเป็นเจ้าของ ตั้งแต่การสร้าง การแก้ไข และการลบรายการแข่งขัน การนำเข้ารายชื่อผู้เข้าแข่งขัน รวมถึงการมีปฏิสัมพันธ์กับการแข่งขันในฐานะผู้มีส่วนร่วม เช่น การเข้าร่วมผ่านรหัส PIN การตั้งชื่อเล่น การโหวตผู้เข้าแข่งขัน การแชทสด และการเรียกดูผลสรุปการแข่งขัน
- กลุ่มกรณีการใช้งานสำหรับผู้ดูแลระบบ (Admin Use Cases): ครอบคลุมการกำกับดูแลความสงบเรียบร้อยของระบบในภาพรวม ผ่านการจัดการบัญชีผู้ใช้งาน (การแบนหรือปลดแบน) การตรวจสอบความเหมาะสมของเนื้อหาทัวร์นาเมนต์ การจัดการรายการแจ้งรายงาน (Reports) จากสมาชิก และการตรวจสอบบันทึกเหตุการณ์ (Audit Logs) เพื่อรักษาความปลอดภัยของระบบ



(รูปที่ 3.12 Use Case Diagram)

ในแผนภาพที่ 3.4 แสดงให้เห็นถึงขอบเขตของระบบที่ถูกแยกออกจากผู้ใช้งานอย่างชัดเจน โดยมีความสัมพันธ์แบบสืบทอด (Generalization) ระหว่าง Actor กลุ่ม Guest ไปยัง Member และ Admin เพื่อลดความซับซ้อนของเส้นความสัมพันธ์ และมีการระบุเงื่อนไขการเข้าถึงข้อมูลตามระดับสิทธิ์ที่ถูกกำหนดไว้ในระบบ (Role-based Access Control)

3. ตารางสรุปภาพรวมกรณีการใช้งาน (Use Case Summary Table) รายละเอียดหน้าที่และตัวแสดงแทนที่เกี่ยวข้องในแต่ละกรณีการใช้งาน ถูกสรุปไว้ในตารางที่ 3.2 ดังนี้:

Use Case ID	กรณีการใช้งาน	ตัวแสดงแทน (Actors)	คำอธิบาย
UC-01	ดูรายการทัวร์นาเมนต์	Guest	การเข้าชมรายการแข่งขันทั้งหมดในระบบ
UC-02	ค้นหาและกรองข้อมูล	Guest	การค้นหาการแข่งขันด้วยชื่อ หมวดหมู่ หรือสถานะ
UC-03	ดูรายละเอียดทัวร์นาเมนต์	Guest	การเข้าดูข้อมูลกติกา ผู้สมัคร และสายการแข่งขัน
UC-04	ดูอันดับผู้ใช้งาน	Guest	การเข้าชมตารางอันดับผู้สร้างทัวร์นาเมนต์ (Leaderboard)
UC-05	ลงทะเบียนสมาชิก	Guest	การสมัครบัญชีใหม่เพื่อใช้งานระบบ
UC-06	เข้าสู่ระบบ	Guest	การยืนยันตัวตนเพื่อเข้าถึงสิทธิ์ตามบทบาท
UC-07	ออกจากระบบ	Member	การยกเลิกเซสชันเพื่อออกจากระบบ

UC-08	แก้ไขข้อมูลส่วนตัว	Member	การปรับปรุงชื่อ รูปภาพ และข้อมูลแนะนำตัว
UC-09	สร้างทัวร์นาเมนต์	Member	การตั้งค่าและสร้างรายการแข่งขันใหม่ (Draft)
UC-10	แก้ไขทัวร์นาเมนต์	Member	การปรับปรุงข้อมูลการแข่งขันที่ตนเองเป็นเจ้าของ
UC-11	ลบทัวร์นาเมนต์	Member	การลบรายการแข่งขันของตนเองออกจากระบบ
UC-12	อัปโหลดผู้เข้าแข่งขัน	Member	การนำเข้ารายชื่อผู้แข่งขันเข้าสู่ทัวร์นาเมนต์
UC-13	ลบผู้เข้าแข่งขัน	Member	การนำรายชื่อผู้แข่งขันออกจากรายการ (Extend)
UC-14	เผยแพร่ทัวร์นาเมนต์	Member	การสร้างรหัส PIN และเปิดห้องรอผู้เข้าแข่งขัน (Waiting Room) เพื่อเตรียมความพร้อมก่อนเริ่มการแข่งขัน
UC-15	เข้าร่วมผ่าน PIN	Member	การเข้าสู่ห้องแข่งขันด้วยรหัสผ่าน 6 หลัก
UC-16	กรอกชื่อเล่น	Member	การระบุชื่อที่ใช้แสดงผลในการแข่ง (Include)

UC-17	รอในห้องพักรอ	Member	การรอสัญญาณเริ่มแข่งขันใน Lobby (Include)
UC-18	เปิดห้อง Lobby	Member	การเปิดระบบรับคนเข้าสู่การแข่งขัน
UC-19	เริ่มการแข่งขัน	Member	การสั่งเริ่มแข่งและเจนเนอเรต Bracket (Extend)
UC-20	โหวตผู้เข้าแข่งขัน	Member	การลงคะแนนเลือกผู้ชนะในแต่ละคู่การแข่ง
UC-21	ดูสายการแข่งขันสด	Member	การเรียกดูแผนผัง Bracket ที่อัปเดตตามจริง
UC-22	แชทสด	Member	การส่งข้อความสื่อสารระหว่างการแข่งขัน
UC-23	ดูสรุปผลการแข่งขัน	Member	การเข้าชมหน้าสรุปผู้ชนะและสถิติหลังจบงาน
UC-24	แสดงความคิดเห็น	Member	การเขียนข้อความติชมในหน้าทัวร์นาเมนต์
UC-25	รายงานเนื้อหา	Member	การแจ้งลบทัวร์นาเมนต์หรือข้อความที่ไม่เหมาะสม
UC-26	ดูแลควบคุมแอดมิน	Admin	การเข้าถึงสถิติและภาพรวมการจัดการระบบ

UC-27	จัดการผู้ใช้งาน	Admin	การตรวจสอบและบริหารจัดการบัญชีสมาชิก
UC-28	แบน / ปลดแบนผู้ใช้	Admin	การระงับหรือคืนสิทธิ์บัญชีผู้ใช้ (Extend)
UC-29	ลบผู้ใช้งาน	Admin	การลบข้อมูลบัญชีสมาชิกออกจากระบบ (Extend)
UC-30	ตรวจสอบทัวร์นาเมนต์	Admin	การกำกับดูแลรายการแข่งขันทั้งหมดในระบบ
UC-31	บังคับจบทัวร์นาเมนต์	Admin	การสั่งปิดการแข่งขันที่มีปัญหาทันที (Extend)
UC-32	จัดการรายการ รายงาน	Admin	การตรวจสอบและดำเนินการกับสิ่งที่ถูก แจ้งรายงาน
UC-33	จัดการเคสรายงาน	Admin	การอนุมัติหรือยกเลิกการแจ้งรายงาน (Extend)
UC-34	ลบความคิดเห็น	Admin	การคัดกรองและลบข้อความที่ไม่เหมาะสม (Extend)
UC-35	ดูบันทึกเหตุการณ์	Admin	การตรวจสอบ Audit Logs การทำงานของ ระบบ

รายละเอียดกรณีการใช้งาน (Use Case Descriptions)

ข้อมูลรายละเอียดกรณีการใช้งานสำหรับระบบ BattleHub ถูกจัดทำขึ้นในรูปแบบตารางเพื่ออธิบายลำดับขั้นตอนการทำงาน ปฏิสัมพันธ์ระหว่างนักแสดงแทนและระบบ รวมถึงเงื่อนไขต่าง ๆ อย่างเป็นระบบ ดังนี้

ส่วนที่ 1: Guest (ผู้เยี่ยมชม)

UC-01: ดูรายการทัวร์นาเมนต์ (View Tournament List)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-01: ดูรายการทัวร์นาเมนต์ (View Tournament List)
นักแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานสามารถเข้าถึงระบบ BattleHub ผ่านเว็บเบราว์เซอร์ได้
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้ารายการทัวร์นาเมนต์ผ่านเมนูนำทาง 2. ระบบดำเนินการดึงข้อมูลทัวร์นาเมนต์ทั้งหมดจากฐานข้อมูล พร้อมเรียงลำดับตามวันที่สร้างจากใหม่ไปเก่า 3. รายการทัวร์นาเมนต์ถูกแสดงผลในรูปแบบการ์ดประกอบด้วย รูปปก ชื่อ หมวดหมู่ สถานะ และวันที่สร้าง 4. ตัวกรองสำหรับหมวดหมู่และสถานะถูกแสดงผลเป็น Dropdown Menu พร้อมปุ่มค้นหา
เงื่อนไขหลังจบ	รายการทัวร์นาเมนต์ที่มีสถานะ "Open" หรือ "Finished" ถูกแสดงผลบนหน้าจอ

UC-02: ค้นหาและกรองทัวร์นาเมนต์ (Search & Filter)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-02: ค้นหาและกรองทัวร์นาเมนต์ (Search & Filter)
ตัวแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานอยู่ในหน้ารายการทัวร์นาเมนต์
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานกรอกคำค้นหาในช่องค้นหา หรือเลือกตัวกรองหมวดหมู่และสถานะ 2. ระบบดำเนินการประมวลผลเงื่อนไขการค้นหาที่ถูกระบุ 3. ข้อมูลทัวร์นาเมนต์ถูกกรองตามเงื่อนไขที่กำหนด และผลลัพธ์ถูกแสดงผลในรูปแบบการ์ด 4. หากไม่พบผลลัพธ์ที่ตรงกับเงื่อนไข ข้อความแจ้งว่าไม่พบข้อมูลถูกแสดงผลโดยระบบ
เงื่อนไขหลังจบ	ผลลัพธ์ที่ตรงกับเงื่อนไขการค้นหาถูกแสดงผลบนหน้าจอ

UC-03: ดูรายละเอียดทัวร์นาเมนต์ (View Tournament Detail)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-03: ดูรายละเอียดทัวร์นาเมนต์ (View Tournament Detail)
ตัวแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์ที่ต้องการดูมีอยู่ในฐานข้อมูลของระบบ

หัวข้อ	รายละเอียด
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานทำการเลือกทัวร์นาเมนต์จากหน้ารายการ 2. ระบบดำเนินการดึงข้อมูลรายละเอียดจากฐานข้อมูล 3. หน้ารายละเอียดถูกแสดงผล ประกอบด้วย ชื่อ คำอธิบาย หมวดหมู่ ขนาด Bracket สถานะ รายชื่อผู้เข้าแข่งขัน และส่วนความคิดเห็น 4. หากผู้ใช้งานเป็นเจ้าของทัวร์นาเมนต์ ปุ่มจัดการ (Edit, Delete, Open Lobby) ถูกแสดงผลเพิ่มเติม
เงื่อนไขหลังจบ	รายละเอียดทั้งหมดของทัวร์นาเมนต์ถูกแสดงผลบนหน้าจออย่างครบถ้วน

UC-04: ดูอันดับ (View Leaderboard)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-04: ดูอันดับ (View Leaderboard)
ตัวแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานสามารถเข้าถึงระบบ BattleHub ได้
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้า Leaderboard ผ่านเมนูนำทาง 2. ระบบดำเนินการดึงข้อมูลสถิติของผู้ใช้งานทั้งหมดจากฐานข้อมูล 3. ข้อมูลถูกจัดเรียงตามจำนวนทัวร์นาเมนต์ที่สร้างและคะแนน จากมากไปน้อย

หัวข้อ	รายละเอียด
	4. ตารางอันดับถูกแสดงผล ประกอบด้วย ลำดับที่ ชื่อผู้ใช้ รูปโปรไฟล์ จำนวนทัวร์นาเมนต์ และคะแนนรวม
เงื่อนไขหลังจบ	ตารางอันดับผู้สร้างทัวร์นาเมนต์ถูกแสดงผลบนหน้าจอ

UC-05: สมัครสมาชิก (Register)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-05: สมัครสมาชิก (Register)
ตัวแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานยังไม่มีบัญชีในระบบ BattleHub
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้าสมัครสมาชิกผ่านปุ่ม "Register" บนแถบนำทาง 2. ข้อมูลส่วนบุคคลถูกกรอกลงในแบบฟอร์ม ได้แก่ ชื่อผู้ใช้ (Username) อีเมล (Email) รหัสผ่าน (Password) และยืนยันรหัสผ่าน 3. ระบบดำเนินการตรวจสอบความถูกต้องของข้อมูล ได้แก่ ความซ้ำซ้อนของชื่อผู้ใช้ รูปแบบอีเมล และความปลอดภัยของรหัสผ่าน 4. ข้อมูลบัญชีใหม่ถูกบันทึกลงฐานข้อมูล และโปรไฟล์ผู้ใช้งานถูกสร้างขึ้นโดยอัตโนมัติ
เงื่อนไขหลังจบ	บัญชีผู้ใช้งานถูกสร้างขึ้น และระบบพร้อมสำหรับการเข้าสู่ระบบ

UC-06: เข้าสู่ระบบ (Login)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-06: เข้าสู่ระบบ (Login)
ตัวแสดงแทน (Actors)	ผู้เยี่ยมชม (Guest)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานมีบัญชีที่ลงทะเบียนไว้ในระบบแล้ว
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานกรอกชื่อผู้ใช้และรหัสผ่านลงในแบบฟอร์มเข้าสู่ระบบ 2. ระบบดำเนินการตรวจสอบความถูกต้องของข้อมูลยืนยันตัวตนกับฐานข้อมูล 3. ระบบตรวจสอบสถานะบัญชี — หากบัญชีถูกระงับ (Banned) การเข้าสู่ระบบถูกปฏิเสธพร้อมข้อความแจ้งเตือน 4. ผู้ใช้งานถูกนำส่งไปยังหน้ารายการทัวร์นาเมนต์ พร้อม Session สำหรับยืนยันตัวตน
เงื่อนไขหลังจบ	ผู้ใช้งานเข้าสู่สถานะการลงชื่อเข้าใช้ (Authenticated) และสามารถเข้าถึงฟังก์ชันของสมาชิกได้

ส่วนที่ 2: Member (สมาชิก)

UC-07: ออกจากระบบ (Logout)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-07: ออกจากระบบ (Logout)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบอยู่ (มี Session ที่ถูกต้อง)

หัวข้อ	รายละเอียด
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานกดปุ่ม "Logout" บนแถบนำทาง 2. ระบบดำเนินการทำลาย Session ของผู้ใช้งานจากฐานข้อมูล 3. สถานะของผู้ใช้งานถูกเปลี่ยนกลับเป็นผู้เยี่ยมชม (Guest) 4. ผู้ใช้งานถูกนำส่งไปยังหน้าแรกของระบบ พร้อมข้อความแจ้งการออกจากระบบสำเร็จ
เงื่อนไขหลังจบ	Session ถูกทำลาย และผู้ใช้งานไม่สามารถเข้าถึงฟังก์ชันของสมาชิกได้

UC-08: แก้ไขข้อมูลส่วนตัว (Edit Profile)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-08: แก้ไขข้อมูลส่วนตัว (Edit Profile)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้าแก้ไขโปรไฟล์ผ่านเมนูผู้ใช้งาน 2. แบบฟอร์มแก้ไขถูกแสดงผลพร้อมข้อมูลปัจจุบัน ได้แก่ ชื่อผู้ใช้ อีเมล ข้อความแนะนำตัว (Bio) และรูปโปรไฟล์ 3. ผู้ใช้งานทำการแก้ไขข้อมูลที่ต้องการ รวมถึงการอัปโหลดรูปโปรไฟล์ใหม่ 4. ระบบดำเนินการตรวจสอบความถูกต้องและบันทึกข้อมูลที่อัปเดตลงฐานข้อมูล

หัวข้อ	รายละเอียด
เงื่อนไขหลังจบ	ข้อมูลโปรไฟล์ของผู้ใช้งานถูกอัปเดตในฐานข้อมูลเรียบร้อยแล้ว

UC-09: สร้างทัวร์นาเมนต์ใหม่ (Create Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-09: สร้างทัวร์นาเมนต์ใหม่ (Create Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้าสร้างทัวร์นาเมนต์ผ่านปุ่ม "Create Tournament" 2. ข้อมูลถูกกรอกลงในแบบฟอร์ม ได้แก่ ชื่อทัวร์นาเมนต์ คำอธิบาย หมวดหมู่ ขนาด Bracket (2/4/8/16) ระยะเวลาโหวต และรูปปก 3. ระบบดำเนินการตรวจสอบความถูกต้องของข้อมูล 4. ข้อมูลทัวร์นาเมนต์ถูกบันทึกลงฐานข้อมูลในสถานะ "Draft" พร้อมเชื่อมโยงกับผู้สร้าง
เงื่อนไขหลังจบ	ทัวร์นาเมนต์ใหม่ถูกสร้างขึ้นในสถานะ "Draft" และผู้ใช้งานถูกนำไปยังหน้าเพิ่มผู้เข้าแข่งขัน

UC-10: แก้ไขทัวร์นาเมนต์ (Edit Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-10: แก้ไขทัวร์นาเมนต์ (Edit Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว และเป็นเจ้าของทัวร์นาเมนต์ที่ต้องการแก้ไข
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้ารายละเอียดทัวร์นาเมนต์ของตนเอง และกดปุ่ม "Edit" 2. ระบบดำเนินการตรวจสอบสิทธิ์ความเป็นเจ้าของ หากไม่ใช่เจ้าของ การเข้าถึงถูกปฏิเสธ 3. แบบฟอร์มแก้ไขถูกแสดงผลพร้อมข้อมูลปัจจุบันของทัวร์นาเมนต์ 4. ระบบดำเนินการตรวจสอบความถูกต้องและบันทึกข้อมูลที่อัปเดตลงฐานข้อมูล
เงื่อนไขหลังจบ	ข้อมูลทัวร์นาเมนต์ถูกอัปเดตและผู้ใช้งานถูกนำกลับไปยังหน้ารายละเอียดทัวร์นาเมนต์

UC-11: ลบทัวร์นาเมนต์ (Delete Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-11: ลบทัวร์นาเมนต์ (Delete Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว และเป็นเจ้าของทัวร์นาเมนต์ที่ต้องการลบ

หัวข้อ	รายละเอียด
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้ารายละเอียดทัวร์นาเมนต์ของตนเอง และกดปุ่ม "Delete" 2. กล่องยืนยันการลบถูกแสดงผลเพื่อป้องกันการลบโดยไม่ตั้งใจ 3. ผู้ใช้งานทำการยืนยันการลบ 4. ระบบดำเนินการลบทัวร์นาเมนต์และข้อมูลที่เกี่ยวข้องทั้งหมด ได้แก่ ผู้เข้าแข่งขัน แมตช์ คะแนน โหวต และความคิดเห็น ออกจากฐานข้อมูล
เงื่อนไขหลังจบ	ทัวร์นาเมนต์และข้อมูลที่เกี่ยวข้องถูกลบออกจากระบบ และข้อความแจ้งสำเร็จถูกแสดงผล

UC-12: อัปโหลดผู้เข้าแข่งขัน (Upload Competitors)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-12: อัปโหลดผู้เข้าแข่งขัน (Upload Competitors)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเป็นเจ้าของทัวร์นาเมนต์ และทัวร์นาเมนต์อยู่ในสถานะ "Draft"
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้าจัดการผู้เข้าแข่งขันของทัวร์นาเมนต์ 2. จำนวนผู้เข้าแข่งขันปัจจุบันและจำนวนที่ต้องการถูกแสดงผลในรูปแบบ Progress Bar 3. รูปภาพถูกเลือกผ่านการลากวาง (Drag & Drop) หรือปุ่มเลือกไฟล์ พร้อมกรอกชื่อสำหรับแต่ละคน

หัวข้อ	รายละเอียด
	4. ระบบดำเนินการบันทึกรูปภาพลงพื้นที่จัดเก็บไฟล์ และข้อมูลผู้เข้าแข่งขันถูกบันทึกลงฐานข้อมูล
เงื่อนไขหลังจบ	ผู้เข้าแข่งขันถูกเพิ่มเข้าในทัวร์นาเมนต์ และ Progress Bar ถูกอัปเดตตามจำนวนปัจจุบัน

UC-13: ลบผู้เข้าแข่งขัน (Delete Competitor)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-13: ลบผู้เข้าแข่งขัน (Delete Competitor)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเป็นเจ้าของทัวร์นาเมนต์ ทัวร์นาเมนต์อยู่ในสถานะ "Draft" และมีผู้เข้าแข่งขันอยู่อย่างน้อย 1 คน
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานอยู่ในหน้าจัดการผู้เข้าแข่งขัน และกดปุ่มลบ (X) บนการ์ดของผู้เข้าแข่งขันที่ต้องการ 2. กล้องยืนยันการลบถูกแสดงผลเพื่อป้องกันการลบโดยไม่ตั้งใจ 3. ผู้ใช้งานทำการยืนยันการลบ 4. ระบบดำเนินการลบข้อมูลผู้เข้าแข่งขันออกจากฐานข้อมูล และ Progress Bar ถูกอัปเดต
เงื่อนไขหลังจบ	ผู้เข้าแข่งขันที่เลือกถูกลบออกจากทัวร์นาเมนต์

UC-14: เผยแพร่ทัวร์นาเมนต์ (Publish Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-14: เผยแพร่ทัวร์นาเมนต์ (Publish Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์อยู่ในสถานะ "Draft" และจำนวนผู้เข้าแข่งขันครบตามขนาด Bracket Size ที่กำหนด
ลำดับขั้นตอนการทำงาน	1. ระบบตรวจสอบความครบถ้วนของผู้เข้าแข่งขัน (Bracket Completion Check) 2. ผู้ใช้งานกดปุ่ม "Open Waiting Lobby" เพื่อทำการเผยแพร่ (Publish) 3. ระบบยืนยันความถูกต้องและทำการ เปลี่ยนสถานะ (Change Status) ของทัวร์นาเมนต์ ในฐานข้อมูลจาก "Draft" เป็น "Waiting" 4. ระบบบันทึกเวลาการเผยแพร่และเตรียมข้อมูลสำหรับสร้าง Lobby (เข้าสู่ UC-18)
เงื่อนไขหลังจบ	ทัวร์นาเมนต์มีสถานะ "Waiting" ในฐานข้อมูล และพร้อมสำหรับการสร้างการเชื่อมต่อใน UC-18

UC-15: เข้าร่วมทัวร์นาเมนต์ผ่าน PIN (Join via PIN)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-15: เข้าร่วมทัวร์นาเมนต์ผ่าน PIN (Join via PIN)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว และทัวร์นาเมนต์ที่ต้องการเข้าร่วมมีสถานะ "Waiting"
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้า "Join via PIN" ผ่านเมนูนำทาง 2. รหัส PIN (6 หลัก) ที่ได้รับจากเจ้าของทัวร์นาเมนต์ ถูกกรอกลงในช่องกรอก 3. ระบบดำเนินการตรวจสอบรหัส PIN กับฐานข้อมูล หากไม่ถูกต้อง ข้อความแจ้งเตือนถูกแสดงผล 4. หากรหัส PIN ถูกต้อง ผู้ใช้งานถูกนำไปยังหน้า กรอกชื่อเล่น (UC-16)
เงื่อนไขหลังจบ	รหัส PIN ถูกตรวจสอบสำเร็จ และผู้ใช้งานพร้อมเข้าสู่ขั้นตอนกรอกชื่อเล่น

UC-16: กรอกชื่อเล่น (Enter Nickname)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-16: กรอกชื่อเล่น (Enter Nickname)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	รหัส PIN ถูกตรวจสอบสำเร็จแล้ว (จาก UC-15)
ลำดับขั้นตอนการทำงาน	1. หน้ากรอกชื่อเล่นถูกแสดงผลพร้อมข้อมูลชื่อทัวร์นาเมนต์ที่จะเข้าร่วม 2. ผู้ใช้งานทำการกรอกชื่อเล่นที่ต้องการใช้

หัวข้อ	รายละเอียด
	3. ระบบดำเนินการตรวจสอบว่าชื่อเล่นไม่ว่างเปล่า และตรวจสอบว่าผู้ใช้งานเคยเข้าร่วมแล้วหรือไม่ 4. ข้อมูลผู้เข้าร่วม (Participant) ถูกบันทึกลงฐานข้อมูล โดยเชื่อมโยงกับบัญชีผู้ใช้งาน
เงื่อนไขหลังจบ	ผู้ใช้งานถูกบันทึกเป็นผู้เข้าร่วมและถูกนำไปยังห้องพักรอ (UC-17)

UC-17: รอในห้องพักรอ (Wait in Lobby)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-17: รอในห้องพักรอ (Wait in Lobby)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานกรอกชื่อเล่นและเข้าร่วมทัวร์นาเมนต์สำเร็จแล้ว (จาก UC-16)
ลำดับขั้นตอนการทำงาน	1. หน้า Waiting Lobby ถูกแสดงผล ประกอบด้วยชื่อทัวร์นาเมนต์ รหัส PIN และรายชื่อผู้เข้าร่วม 2. รายชื่อผู้เข้าร่วมถูกอัปเดตแบบอัตโนมัติเป็นระยะเพื่อแสดงผู้เข้าร่วมใหม่ 3. ระบบดำเนินการตรวจสอบสถานะทัวร์นาเมนต์เป็นระยะ 4. เมื่อสถานะถูกเปลี่ยนเป็น "Open" ผู้เข้าร่วมทุกคนถูกนำไปยังหน้าการโหวตโดยอัตโนมัติ
เงื่อนไขหลังจบ	ผู้เข้าร่วมรองชนะเลิศการแข่งขันจะเริ่มต้น จากนั้นถูกนำไปยังหน้าโหวต

UC-18: เปิดห้อง Lobby (Open Lobby)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-18: เปิดห้อง Lobby (Open Lobby)
ตัวแสดงแทน (Actors)	สมาชิก (Member) — เจ้าของทัวร์นาเมนต์
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์อยู่ในสถานะ "Draft" และมีผู้เข้าแข่งขันครบตามขนาด Bracket Size ที่กำหนด
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานเข้าสู่หน้ารายละเอียดทัวร์นาเมนต์ และกดปุ่ม "Open Waiting Lobby" 2. รหัส PIN จำนวน 6 หลักถูกสร้างขึ้นแบบสุ่มโดยระบบ โดยไม่ซ้ำกับรหัสที่มีอยู่ 3. สถานะทัวร์นาเมนต์ถูกเปลี่ยนจาก "Draft" เป็น "Waiting" และรหัส PIN ถูกบันทึกลงฐานข้อมูล (ดำเนินการควบคู่กับ UC-14) 4. ผู้ใช้งานถูกนำส่งไปยังหน้า Waiting Lobby พร้อมรหัส PIN ที่แสดงอย่างเด่นชัด
เงื่อนไขหลังจบ	ห้อง Lobby ถูกเปิดพร้อมรหัส PIN สำหรับเชิญผู้เข้าร่วม และระบบพร้อมรับการเชื่อมต่อจากผู้ใช้

UC-19: เริ่มทัวร์นาเมนต์ (Start Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-19: เริ่มทัวร์นาเมนต์ (Start Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member) — เจ้าของทัวร์นาเมนต์
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์อยู่ในสถานะ "Waiting" และมีผู้เข้าร่วมอยู่ในห้อง Lobby

หัวข้อ	รายละเอียด
ลำดับขั้นตอนการทำงาน	1. รายชื่อผู้เข้าร่วมถูกแสดงแบบ Real-time ในหน้า Waiting Lobby 2. ผู้ใช้งาน (เจ้าของทัวร์นาเมนต์) กดปุ่ม "Start Tournament" 3. สถานะทัวร์นาเมนต์ถูกเปลี่ยนจาก "Waiting" เป็น "Open" และแมตช์แรกถูกเริ่มต้น 4. ผู้เข้าร่วมทุกคนถูกนำไปยังหน้าการโหวต (Play Page) โดยอัตโนมัติ
เงื่อนไขหลังจบ	ทัวร์นาเมนต์เริ่มต้นการแข่งขัน แมตช์แรกถูกดำเนินการ

UC-20: โหวตผู้เข้าแข่งขัน (Vote for Competitor)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-20: โหวตผู้เข้าแข่งขัน (Vote for Competitor)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว ทัวร์นาเมนต์อยู่ในสถานะ "Open" และมีแมตช์ที่กำลังดำเนินอยู่
ลำดับขั้นตอนการทำงาน	1. หน้าการโหวตถูกแสดงผล ประกอบด้วยรูปภาพของผู้เข้าแข่งขัน 2 คน พร้อมตัวนับเวลาถอยหลัง 2. ผู้ใช้งานทำการเลือกผู้เข้าแข่งขันที่ต้องการโหวต โดยกดที่รูปภาพ 3. ระบบดำเนินการตรวจสอบว่าผู้ใช้งานยังไม่เคยโหวตในแมตช์นี้ และบันทึกคะแนนโหวตลงฐานข้อมูล

หัวข้อ	รายละเอียด
	4. เมื่อเวลาหมดลง ผลแมตช์ถูกสรุปโดยอัตโนมัติ — ผู้ได้คะแนนมากกว่าถูกประกาศเป็นผู้ชนะ และแมตช์ถัดไปถูกเริ่มต้น
เงื่อนไขหลังจบ	คะแนนโหวตถูกบันทึก ผลแมตช์ถูกสรุป และแมตช์ถัดไปถูกเริ่มต้นโดยอัตโนมัติ

UC-21: ดูสายการแข่งขัน (View Live Bracket)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-21: ดูสายการแข่งขัน (View Live Bracket)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์อยู่ในสถานะ "Open" หรือ "Finished" และมีการจับคู่แมตช์แล้ว
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานกดปุ่ม "View Bracket" ในหน้าโหวตหรือหน้ารายละเอียดทัวร์นาเมนต์ 2. ระบบดำเนินการดึงข้อมูลแมตช์ทั้งหมดจากฐานข้อมูล และจัดกลุ่มตามรอบการแข่งขัน 3. สายการแข่งขันถูกแสดงผลในรูปแบบแผนผังต้นไม้ แมตช์ที่เสร็จสิ้นแสดงชื่อผู้ชนะ ส่วนแมตช์ที่ยังไม่ดำเนินการแสดงสถานะรอ 4. ผู้ใช้งานสามารถกลับไปยังหน้าโหวตหรือหน้ารายละเอียดทัวร์นาเมนต์ได้
เงื่อนไขหลังจบ	สายการแข่งขันพร้อมผลการแข่งขันแต่ละรอบถูกแสดงผลบนหน้าจอ

UC-22: แชทสด (Live Chat)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-22: แชทสด (Live Chat)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว ทวีร์นาเมนต์อยู่ในสถานะ "Open" และมีแมตช์กำลังดำเนินอยู่
ลำดับขั้นตอนการทำงาน	1. ส่วนแชทสดถูกแสดงผลในหน้าการโหวต 2. ผู้ใช้งานพิมพ์ข้อความและกดปุ่มส่ง 3. ข้อความถูกบันทึกลงฐานข้อมูลและแสดงผลแบบ Real-time ให้ผู้เข้าร่วมทุกคน 4. ข้อความที่ไม่เหมาะสมสามารถถูกรายงานโดยผู้ใช้งานผ่านปุ่มรายงาน
เงื่อนไขหลังจบ	ข้อความถูกบันทึกและแสดงผลในส่วนแชทสดของหน้าโหวต

UC-23: ดูสรุปผลการแข่งขัน (View Summary)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-23: ดูสรุปผลการแข่งขัน (View Summary)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ทวีร์นาเมนต์อยู่ในสถานะ "Finished" (การแข่งขันทุกรอบดำเนินการจนเสร็จสิ้น)
ลำดับขั้นตอนการทำงาน	1. ระบบตรวจสอบว่าทวีร์นาเมนต์อยู่ในสถานะ "Finished"

หัวข้อ	รายละเอียด
	2. ข้อมูลผลการแข่งขันในรอบถูกดึงจากฐานข้อมูล และจัดเรียงตามรอบ 3. หน้าสรุปผลถูกแสดงผล ประกอบด้วย รูปภาพ และชื่อผู้ชนะเลิศ (Champion) พร้อมสถิติการโหวตแต่ละรอบ 4. สายการแข่งขันที่สมบูรณ์พร้อมชื่อผู้ชนะแต่ละแมตช์ถูกแสดงผลเพิ่มเติม
เงื่อนไขหลังจบ	ผลการแข่งขันทั้งหมดและผู้ชนะเลิศถูกแสดงผลบนหน้าจอ

UC-24: แสดงความคิดเห็น (Comment on Tournament)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-24: แสดงความคิดเห็น (Comment on Tournament)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว และทัวร์นาเมนต์มีอยู่ในระบบ
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานอยู่ในหน้ารายละเอียดทัวร์นาเมนต์ ส่วนความคิดเห็นพร้อมช่องพิมพ์ถูกแสดงผล 2. ผู้ใช้งานพิมพ์ความคิดเห็นและกดปุ่มส่ง 3. ระบบดำเนินการตรวจสอบว่าข้อความไม่ว่างเปล่า และบันทึกความคิดเห็นลงฐานข้อมูล

หัวข้อ	รายละเอียด
	4. ความคิดเห็นใหม่ถูกแสดงผลในส่วนความคิดเห็น พร้อมชื่อผู้เขียนและวันเวลา
เงื่อนไขหลังจบ	ความคิดเห็นถูกบันทึกและแสดงผลในหน้ารายละเอียดทัวร์นาเมนต์

UC-25: รายงานเนื้อหาไม่เหมาะสม (Report Content)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-25: รายงานเนื้อหาไม่เหมาะสม (Report Content)
ตัวแสดงแทน (Actors)	สมาชิก (Member)
เงื่อนไขก่อนเริ่ม	ผู้ใช้งานเข้าสู่ระบบแล้ว และมีเนื้อหาที่ต้องการรายงาน
ลำดับขั้นตอนการทำงาน	1. ผู้ใช้งานกดปุ่มรายงาน (Report) บนความคิดเห็นที่ไม่เหมาะสม 2. แบบฟอร์มรายงานถูกแสดงผล ประกอบด้วยช่องระบุเหตุผลของการรายงาน 3. ผู้ใช้งานกรอกเหตุผลและกดปุ่มส่ง 4. ระบบดำเนินการบันทึกรายงานลงฐานข้อมูลในสถานะ "Pending" พร้อมข้อความแจ้งสำเร็จ
เงื่อนไขหลังจบ	รายงานปัญหาถูกสร้างขึ้นในสถานะ "Pending" รอการพิจารณาจากผู้ดูแลระบบ

ส่วนที่ 3: Admin (ผู้ดูแลระบบ)

UC-26: ดูแดชบอร์ด (View Dashboard)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-26: ดูแดชบอร์ด (View Dashboard)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีสิทธิ์ Staff
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเข้าสู่หน้า Admin Panel 2. ระบบดำเนินการดึงข้อมูลสถิติจากฐานข้อมูล ได้แก่ จำนวนผู้ใช้ จำนวนทัวร์นาเมนต์ และจำนวนรายงาน 3. ข้อมูลสถิติถูกแสดงผลในรูปแบบการ์ดสรุป (Stats Cards) พร้อมตารางกิจกรรมล่าสุด 4. เมื่อนำทางไปยังส่วนจัดการต่าง ๆ ถูกแสดงผลบนแถบด้านข้าง (Sidebar)
เงื่อนไขหลังจบ	ข้อมูลภาพรวมของระบบถูกแสดงผลบนหน้าแดชบอร์ด

UC-27: จัดการผู้ใช้งาน (Manage Users)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-27: จัดการผู้ใช้งาน (Manage Users)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีสิทธิ์ Staff

หัวข้อ	รายละเอียด
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเข้าสู่หน้าจัดการผู้ใช้งานผ่าน Admin Panel 2. ระบบดำเนินการดึงข้อมูลผู้ใช้งานทั้งหมดจากฐานข้อมูล 3. รายชื่อผู้ใช้งานถูกแสดงผล พร้อมข้อมูลสถานะวันที่สมัคร จำนวนทัวร์นาเมนต์ และตัวเลือกการดำเนินการ 4. ผู้ดูแลระบบสามารถเลือกผู้ใช้งานเพื่อดำเนินการเพิ่มเติม (แบน/ปลดแบน/ลบ) ได้
เงื่อนไขหลังจบ	รายชื่อผู้ใช้งานทั้งหมดถูกแสดงผลพร้อมตัวเลือกการดำเนินการ

UC-28: แบน/ปลดแบนผู้ใช้งาน (Ban / Unban User)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-28: แบน/ปลดแบนผู้ใช้งาน (Ban / Unban User)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบอยู่ในหน้าจัดการผู้ใช้งาน และมีผู้ใช้งานที่ต้องการดำเนินการ
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเลือกผู้ใช้งานที่ต้องการดำเนินการ 2. ปุ่ม "Ban" หรือ "Unban" ถูกแสดงผลตามสถานะปัจจุบันของผู้ใช้งาน

หัวข้อ	รายละเอียด
	3. กล่องยืนยันถูกแสดงผล และผู้ดูแลระบบทำการยืนยัน 4. ระบบดำเนินการเปลี่ยนสถานะผู้ใช้งานในฐานะข้อมูล และ Audit Log ถูกบันทึกโดยอัตโนมัติ
เงื่อนไขหลังจบ	สถานะผู้ใช้งานถูกเปลี่ยนแปลง (Active/Banned) และ Audit Log ถูกบันทึก

UC-29: ลบผู้ใช้งาน (Delete User)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-29: ลบผู้ใช้งาน (Delete User)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบอยู่ในหน้าจัดการผู้ใช้งาน และผู้ใช้งานเป้าหมายไม่ใช่ Superuser
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเลือกผู้ใช้งานที่ต้องการลบ และกดปุ่ม "Delete" 2. กล่องยืนยันการลบถูกแสดงผล 3. ผู้ดูแลระบบทำการยืนยันการลบ 4. ระบบดำเนินการลบผู้ใช้งานและข้อมูลที่เกี่ยวข้องออกจากฐานข้อมูล และ Audit Log ถูกบันทึกโดยอัตโนมัติ
เงื่อนไขหลังจบ	ผู้ใช้งานและข้อมูลที่เกี่ยวข้องถูกลบออกจากระบบ

UC-30: ดูแลทัวร์นาเมนต์ (Moderate Tournaments)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-30: ดูแลทัวร์นาเมนต์ (Moderate Tournaments)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีสิทธิ์ Staff
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเข้าสู่หน้าจัดการทัวร์นาเมนต์ผ่าน Admin Panel 2. ระบบดำเนินการดึงข้อมูลทัวร์นาเมนต์ทั้งหมดจากฐานข้อมูล 3. รายการทัวร์นาเมนต์ถูกแสดงผล พร้อมข้อมูลสถานะ เจ้าของ วันที่สร้าง และตัวเลือกการดำเนินการ 4. ผู้ดูแลระบบสามารถเลือกทัวร์นาเมนต์เพื่อดำเนินการเพิ่มเติม (ลบ/บังคับจบ) ได้
เงื่อนไขหลังจบ	รายการทัวร์นาเมนต์ทั้งหมดถูกแสดงผลพร้อมตัวเลือกการดำเนินการ

UC-31: บังคับจบทัวร์นาเมนต์ (Force Finish)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-31: บังคับจบทัวร์นาเมนต์ (Force Finish)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ทัวร์นาเมนต์อยู่ในสถานะ "Open" หรือ "Waiting"
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเลือกทัวร์นาเมนต์ที่ต้องการบังคับจบและกดปุ่ม "Force Finish"

หัวข้อ	รายละเอียด
	2. กล่องยืนยันถูกแสดงผล 3. ผู้ดูแลระบบทำการยืนยัน 4. ระบบดำเนินการเปลี่ยนสถานะทัวร์นาเมนต์เป็น "Finished" และ Audit Log ถูกบันทึกโดยอัตโนมัติ
เงื่อนไขหลังจบ	ทัวร์นาเมนต์ถูกเปลี่ยนสถานะเป็น "Finished" โดยบังคับ

UC-32: จัดการรายงานปัญหา (Manage Reports)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-32: จัดการรายงานปัญหา (Manage Reports)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีรายงานปัญหาในระบบ
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเข้าสู่หน้าจัดการรายงานปัญหาผ่าน Admin Panel 2. ระบบดำเนินการดึงข้อมูลรายงานทั้งหมดจากฐานข้อมูล 3. รายงานถูกแสดงผล พร้อมข้อมูลผู้รายงาน เป้าหมาย เหตุผล สถานะ และสถิติแยกตามสถานะ 4. ผู้ดูแลระบบสามารถกรองรายงานตามสถานะ และเลือกรายงานเพื่อดำเนินการได้
เงื่อนไขหลังจบ	รายการรายงานปัญหาถูกแสดงผลพร้อมตัวเลือกการดำเนินการ

UC-33: ดำเนินการรายงาน (Resolve / Dismiss)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-33: ดำเนินการรายงาน (Resolve / Dismiss)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบอยู่ในหน้าจัดการรายงาน และรายงานอยู่ในสถานะ "Pending"
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเลือกรายงานที่ต้องการดำเนินการ 2. ข้อมูลรายละเอียดของรายงานถูกแสดงผล 3. ผู้ดูแลระบบกดปุ่ม "Resolve" (ดำเนินการแล้ว) หรือ "Dismiss" (ยกเลิก) 4. ระบบดำเนินการเปลี่ยนสถานะรายงานในฐานข้อมูล และ Audit Log ถูกบันทึกโดยอัตโนมัติ
เงื่อนไขหลังจบ	สถานะรายงานถูกเปลี่ยนเป็น "Resolved" หรือ "Dismissed"

UC-34: ลบความคิดเห็น (Delete Comment)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-34: ลบความคิดเห็น (Delete Comment)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีความคิดเห็นที่ถูกรายงานหรือไม่เหมาะสม
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบพบความคิดเห็นที่ไม่เหมาะสมผ่านหน้าจัดการรายงานหรือนำรายละเอียดทัวร์นาเมนต์

หัวข้อ	รายละเอียด
	2. ผู้ดูแลระบบกดปุ่ม "Delete Comment" 3. กล้องยืนยันการลบถูกแสดงผล และผู้ดูแลระบบทำการยืนยัน 4. ระบบดำเนินการลบความคิดเห็นออกจากฐานข้อมูล และ Audit Log ถูกบันทึกโดยอัตโนมัติ
เงื่อนไขหลังจบ	ความคิดเห็นถูกลบออกจากระบบ และ Audit Log ถูกบันทึก

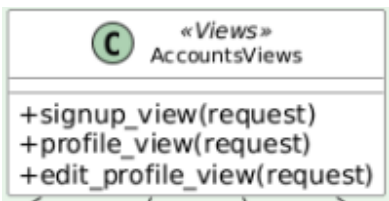
UC-35: ดูบันทึกการดำเนินการ (View Audit Logs)

หัวข้อ	รายละเอียด
ชื่อกรณีการใช้งาน	UC-35: ดูบันทึกการดำเนินการ (View Audit Logs)
ตัวแสดงแทน (Actors)	ผู้ดูแลระบบ (Admin)
เงื่อนไขก่อนเริ่ม	ผู้ดูแลระบบเข้าสู่ระบบแล้ว และมีสิทธิ์ Staff
ลำดับขั้นตอนการทำงาน	1. ผู้ดูแลระบบเข้าสู่หน้า Audit Logs ผ่าน Admin Panel 2. ระบบดำเนินการดึงข้อมูลบันทึกการดำเนินการทั้งหมดจากฐานข้อมูล 3. รายการบันทึกที่แสดงผลในรูปแบบตาราง ประกอบด้วย ผู้ดำเนินการ ประเภทการกระทำ เป้าหมาย และวันเวลา เรียงจากล่าสุดไปเก่าสุด 4. ผู้ดูแลระบบสามารถตรวจสอบประวัติการดำเนินการทั้งหมดของผู้ดูแลระบบทุกคนได้
เงื่อนไขหลังจบ	รายการบันทึกการดำเนินการถูกแสดงผลอย่างครบถ้วน

3.5 แผนภาพคลาส Class Diagram

แผนภาพคลาสของระบบ BattleHub แสดงโครงสร้างและความสัมพันธ์ระหว่างคลาสต่างๆ ภายในระบบ โดยแบ่งการแสดงผลออกเป็น 2 ส่วนหลัก ได้แก่ ฝั่ง Backend (Django Models, Views, Forms) และฝั่ง Frontend (HTML Templates, JavaScript Components) เพื่อให้การนำเสนอมีความชัดเจนและไม่ซับซ้อนจนเกินไป

ตารางที่ 3.3 สัญลักษณ์ที่ใช้ในแผนภาพคลาส

สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
	Class	กล่องแสดงคลาส ประกอบด้วย 3 ส่วน: 1. ชื่อคลาส (Class Name) 2. แอตทริบิวต์ (Attributes) 3. เมธอด (Methods)
	Generalization	การสืบทอด (Inheritance): หัวลูกศรสามเหลี่ยมโป่ง ใช้แสดงว่าคลาสลูกสืบทอดคุณสมบัติมาจากคลาสแม่ (เช่น User เป็นแม่, Admin เป็นลูก)
	Composition	องค์ประกอบ (Composition): หัวเพชรทึบ ใช้แสดงความสัมพันธ์ที่ "ขาดจากกันไม่ได้" (เช่น Tournament หาย Match ก็ต้องหายด้วย)
	Association	ความสัมพันธ์ทั่วไป (Association): เส้นทึบ

		ใช้แสดงการเชื่อมโยงระหว่างคลาสที่ทำงานร่วมกัน (เช่น User เชื่อมกับ Profile)
	Dependency	การพึ่งพา (Dependency): เส้นประหัวลูกศร ใช้แสดงว่าคลาสหนึ่งเรียกใช้อีกคลาสหนึ่งชั่วคราว (เช่น View เรียกใช้ Form)
0...1	Multiplicity	จำนวนความสัมพันธ์: ตัวเลขกำกับเส้นความสัมพันธ์ เช่น 1 (หนึ่งเดียว), 0..* (ศูนย์หรือหลาย), 1..* (หนึ่งหรือหลาย)

3.5.1 Backend Class Diagram

ระบบฝั่ง Backend พัฒนาด้วย Django Framework แบ่งออกเป็น 3 Django App ดังนี้

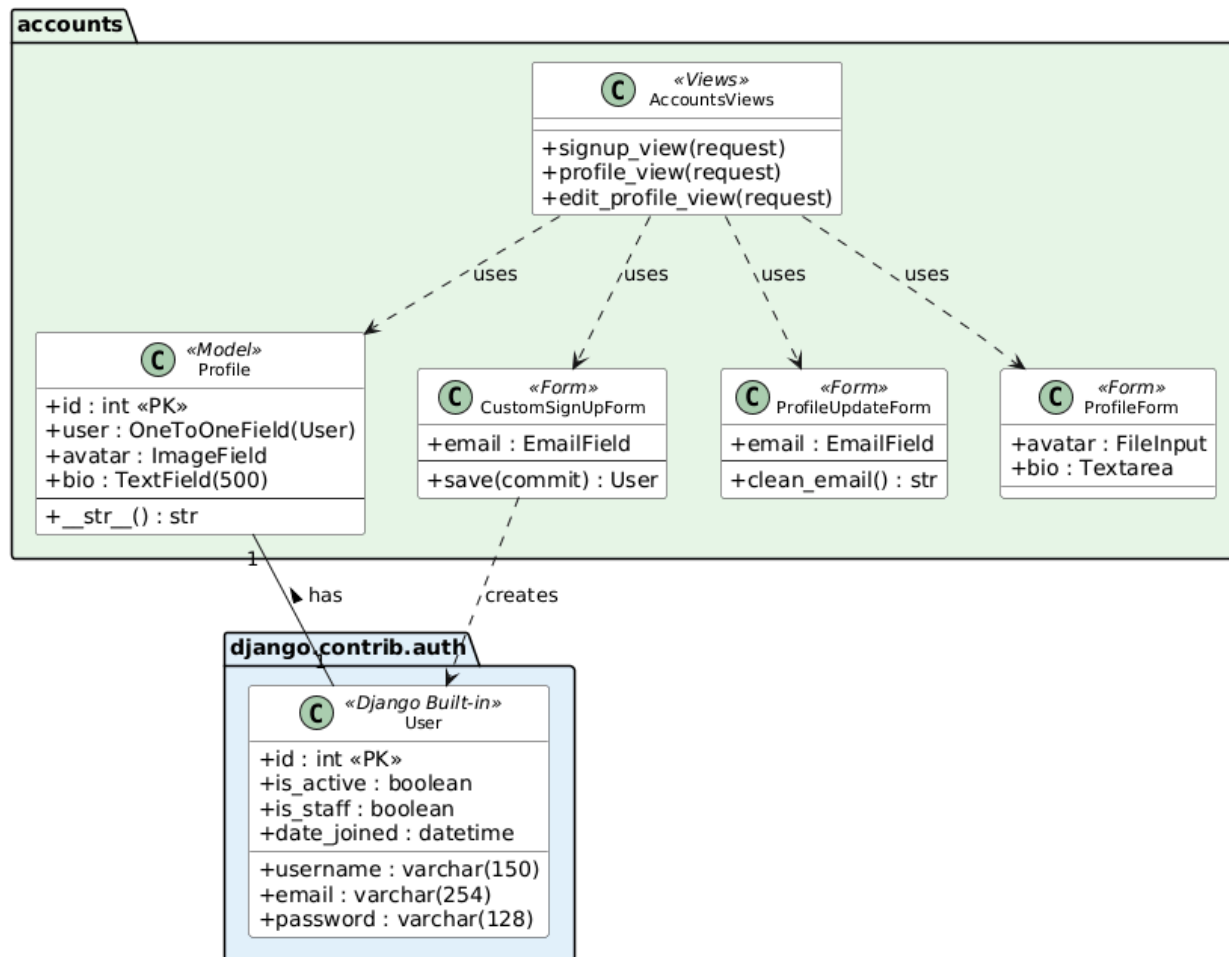
3.5.1.1 ส่วนจัดการบัญชีผู้ใช้งาน (Accounts App)

แอปพลิเคชัน Accounts รับผิดชอบกระบวนการจัดการบัญชีผู้ใช้งานทั้งหมด โดยมีรายละเอียดของคลาสที่สำคัญ ดังนี้

- User (Django Built-in): คลาสมาตรฐานของ Django สำหรับเก็บข้อมูลบัญชีผู้ใช้ ประกอบด้วยฟิลด์สำคัญ ได้แก่ id, username, email, password, is_active (สถานะบัญชี), is_staff (สิทธิ์ผู้ดูแล) และ date_joined
- Profile: คลาสสำหรับเก็บข้อมูลส่วนตัวเพิ่มเติมของผู้ใช้ มีความสัมพันธ์แบบ One-to-One กับคลาส User โดยประกอบด้วยฟิลด์ avatar (รูปโปรไฟล์) และ bio (คำแนะนำตัว) ซึ่ง Profile จะถูกสร้างขึ้นอัตโนมัติผ่าน Django post_save signal เมื่อมีการสมัครสมาชิกใหม่
- CustomSignUpForm: แบบฟอร์มสำหรับสมัครสมาชิกที่สืบทอดมาจาก UserCreationForm โดยเพิ่มช่องกรอก email

- ProfileUpdateForm: แบบฟอร์มสำหรับแก้ไขข้อมูลบัญชี (email) พร้อมเมธอด clean_email() เพื่อตรวจสอบความซ้ำซ้อนของอีเมลในระบบ
- ProfileForm: แบบฟอร์มสำหรับแก้ไขข้อมูลส่วนตัว (avatar, bio)
- AccountsViews: คลาสสำหรับการจัดการการแสดงผลและประมวลผลข้อมูล ได้แก่ signup_view (สมัครสมาชิก), profile_view (ดูโปรไฟล์) และ edit_profile_view (แก้ไขโปรไฟล์)

Backend Class Diagram — Accounts App



(รูปที่ 3.13 แผนภาพคลาส Backend — Accounts App)

คำอธิบายภาพ: จากภาพที่ 3.13 แสดงแผนภาพคลาสของ Accounts App ซึ่งรับผิดชอบระบบจัดการบัญชีผู้ใช้งาน ประกอบด้วย Model "User" ที่เป็นคลาสพื้นฐานของ Django สำหรับเก็บข้อมูลบัญชี (username, email, password, สิทธิ์) และ Model "Profile" ที่มีความสัมพันธ์แบบ One-to-One กับ User สำหรับเก็บข้อมูลเพิ่มเติม ได้แก่ รูป Avatar และ Bio ฝั่ง Form มี 3 คลาส ได้แก่ CustomSignUpForm สำหรับสมัครสมาชิก (สืบทอดจาก UserCreationForm เพิ่มช่อง email), ProfileUpdateForm สำหรับแก้ไข email

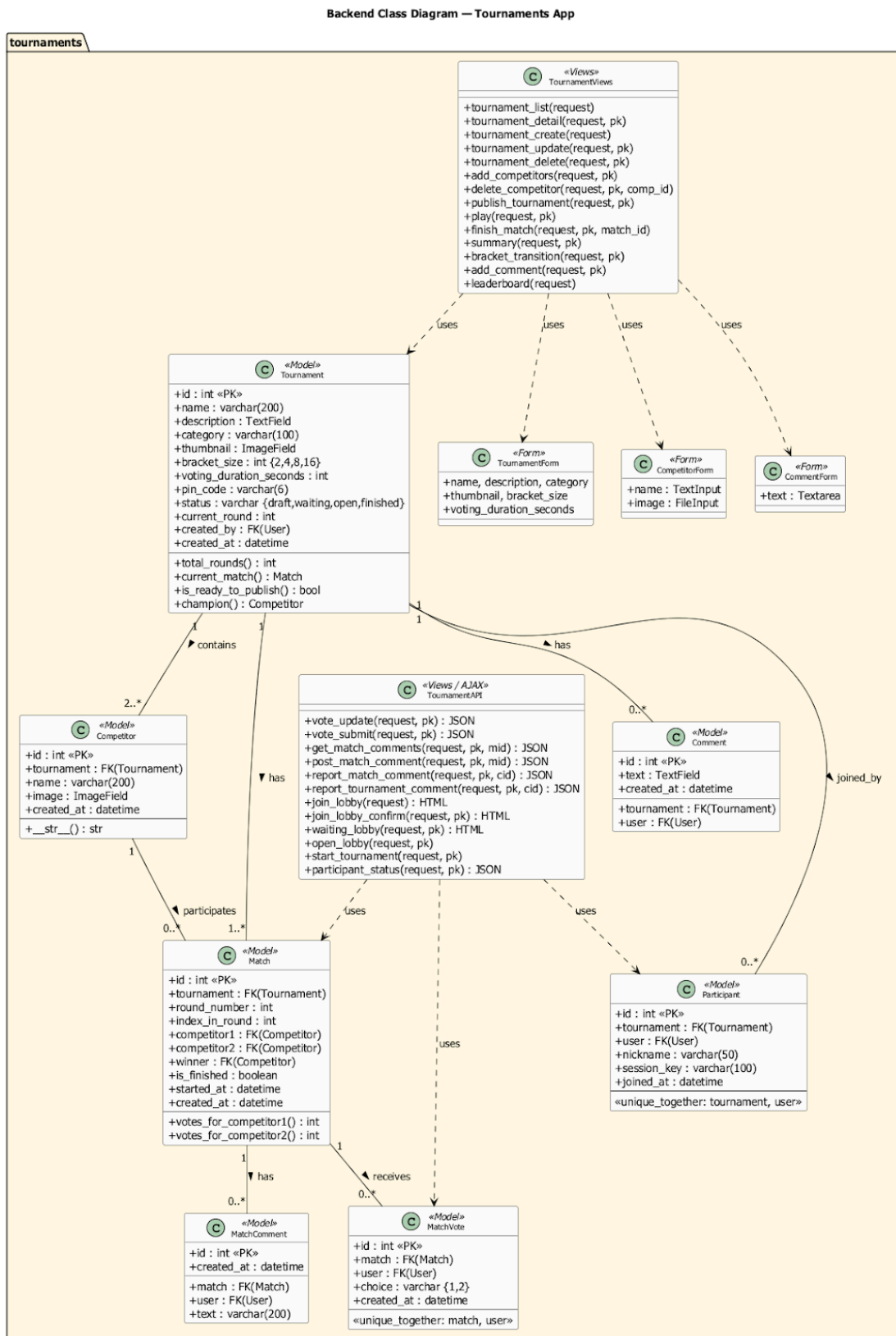
และ ProfileForm สำหรับแก้ไข avatar และ bio ทั้งหมดถูกเรียกใช้งานผ่าน AccountsViews ซึ่งมี 3 ฟังก์ชัน ได้แก่ signup_view, profile_view และ edit_profile_view

3.5.1.2 Tournaments App

เป็น App หลักและใหญ่ที่สุดของระบบ รับผิดชอบฟังก์ชันการทำงานทั้งหมดของทัวร์นาเมนต์ ประกอบด้วย

- Tournament เก็บข้อมูลทัวร์นาเมนต์ ประกอบด้วย id, name, description, category, thumbnail (ImageField), bracket_size (2, 4, 8 หรือ 16), voting_duration_seconds, pin_code (6 หลัก), status (draft, waiting, open, finished), current_round, created_by (FK → User) มีเมธอด total_rounds(), current_match(), is_ready_to_publish(), champion()
- Competitor เก็บข้อมูลผู้เข้าแข่งขัน ประกอบด้วย id, tournament (FK), name, image (ImageField) มีความสัมพันธ์แบบ Many-to-One กับ Tournament
- Match เก็บข้อมูลแมตช์ 1v1 ประกอบด้วย id, tournament (FK), round_number, index_in_round, competitor1 (FK), competitor2 (FK), winner (FK, nullable), is_finished, started_at มีเมธอด votes_for_competitor1(), votes_for_competitor2()
- MatchVote เก็บข้อมูลการโหวต ประกอบด้วย id, match (FK), user (FK), choice ('1' หรือ '2'), created_at มี unique_together constraint (match, user) ป้องกันโหวตซ้ำ แต่อนุญาตให้เปลี่ยนใจได้
- Comment เก็บความคิดเห็นในหน้ารายละเอียด ประกอบด้วย id, tournament (FK), user (FK), text (TextField), created_at
- MatchComment เก็บข้อความแชทระหว่างโหวต ประกอบด้วย id, match (FK), user (FK), text (varchar 200), created_at จำกัด 200 ตัวอักษร
- Participant เก็บข้อมูลผู้เข้าร่วม Lobby ประกอบด้วย id, tournament (FK), user (FK, nullable), nickname (varchar 50), session_key, joined_at มี unique_together constraint (tournament, user)
- TournamentForm ใช้สร้างและแก้ไขทัวร์นาเมนต์ (name, description, category, thumbnail, bracket_size, voting_duration_seconds)

- CompetitorForm ใช้อัปโหลดผู้เข้าแข่งขัน (name, image)
- CommentForm ใช้เขียนความคิดเห็น (text)
- TournamentViews จัดการ Server-Side Rendering 14 ฟังก์ชัน ได้แก่ tournament_list, tournament_detail, tournament_create, tournament_update, tournament_delete, add_competitors, delete_competitor, publish_tournament, play, finish_match, summary, bracket_transition, add_comment, leaderboard
- TournamentAPI จัดการ AJAX Endpoint 12 ฟังก์ชัน ได้แก่ vote_update, vote_submit, get_match_comments, post_match_comment, report_match_comment, report_tournament_comment, join_lobby, join_lobby_confirm, waiting_lobby, open_lobby, start_tournament, participant_status



(รูปที่ 3.14 แผนภาพคลาส Backend — Tournaments App)

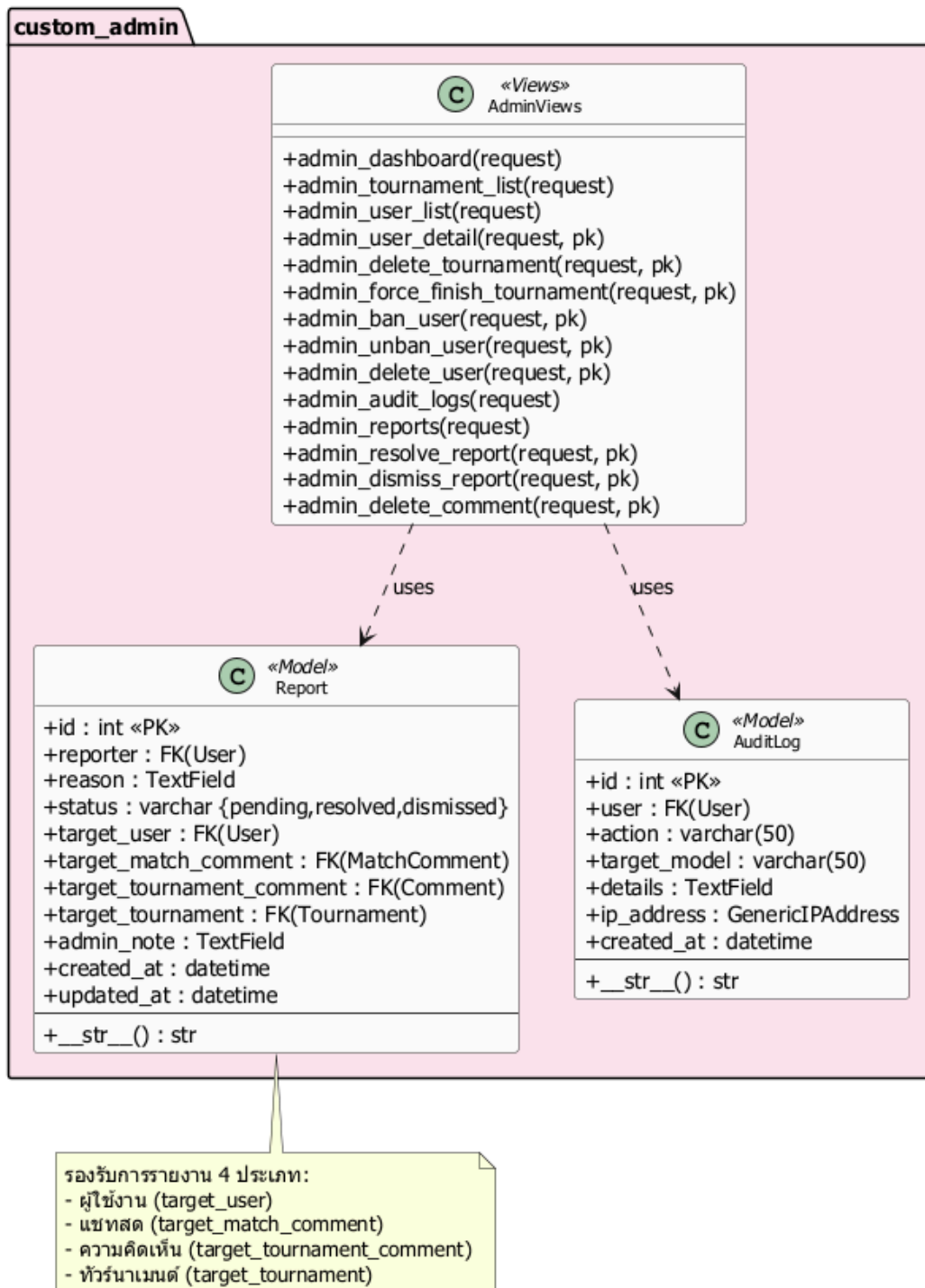
คำอธิบายภาพ: จากภาพที่ 3.14 แสดงแผนภาพคลาสของ Tournaments App ซึ่งเป็น App หลักและใหญ่ที่สุดของระบบ รับผิดชอบฟังก์ชันการทำงานทั้งหมดของทัวร์นาเมนต์ ประกอบด้วย Model 7 ตัว ดังนี้ (1) Tournament เก็บข้อมูลทัวร์นาเมนต์ มี 4 สถานะ (draft, waiting, open, finished) และเก็บ PIN 6 หลัก สำหรับระบบ Lobby (2) Competitor เก็บข้อมูลผู้เข้าแข่งขัน มีความสัมพันธ์ Many-to-One กับ Tournament (3) Match เก็บข้อมูลแมตช์ 1v1 ประกอบด้วย เลขรอบ ลำดับแมตช์ คู่แข่งขัน และผู้ชนะ (4) MatchVote เก็บคะแนนโหวต มี unique_together constraint ป้องกันโหวตซ้ำ (5) Comment เก็บความคิดเห็นในหน้ารายละเอียดทัวร์นาเมนต์ (6) MatchComment เก็บข้อความแชทระหว่างโหวต จำกัด 200 ตัวอักษร (7) Participant เก็บข้อมูลผู้เข้าร่วม Lobby มี nickname และ session_key ฝั่ง Form มี 3 คลาส และฝั่ง Views แบ่งเป็น TournamentViews สำหรับ Server-Side Rendering (14 ฟังก์ชัน) และ TournamentAPI สำหรับ AJAX Endpoint (12 ฟังก์ชัน)

3.5.1.3 Custom Admin App

รับผิดชอบระบบจัดการสำหรับผู้ดูแลระบบ ประกอบด้วย

- **Report** เก็บข้อมูลการรายงานปัญหา ประกอบด้วย id, reporter (FK → User), reason (TextField), status (pending, resolved, dismissed) รองรับ 4 ประเภทเป้าหมายผ่าน Nullable Foreign Key ได้แก่ target_user, target_match_comment, target_tournament_comment, target_tournament มี admin_note สำหรับบันทึกภายใน created_at และ updated_at
- **AuditLog** เก็บประวัติการดำเนินการของผู้ดูแลระบบ ประกอบด้วย id, user (FK → Admin), action (varchar 50 เช่น BAN, DELETE, FORCE_FINISH), target_model (varchar 50), details (TextField), ip_address (GenericIPAddress), created_at ทุกการกระทำของ Admin จะถูกบันทึกอัตโนมัติเพื่อความโปร่งใสและตรวจสอบย้อนหลังได้
- **AdminViews** ประกอบด้วย 14 ฟังก์ชัน แบ่งเป็น 4 กลุ่มหลัก ได้แก่ (1) Dashboard: admin_dashboard (2) จัดการทัวร์นาเมนต์: admin_tournament_list, admin_delete_tournament, admin_force_finish_tournament (3) จัดการผู้ใช้: admin_user_list, admin_user_detail, admin_ban_user, admin_unban_user, admin_delete_user (4) จัดการรายงาน: admin_reports, admin_resolve_report, admin_dismiss_report, admin_delete_comment, admin_audit_logs ทุกฟังก์ชันถูกป้องกันด้วย @admin_required decorator ที่ตรวจสอบ is_staff = True

Backend Class Diagram — Custom Admin App

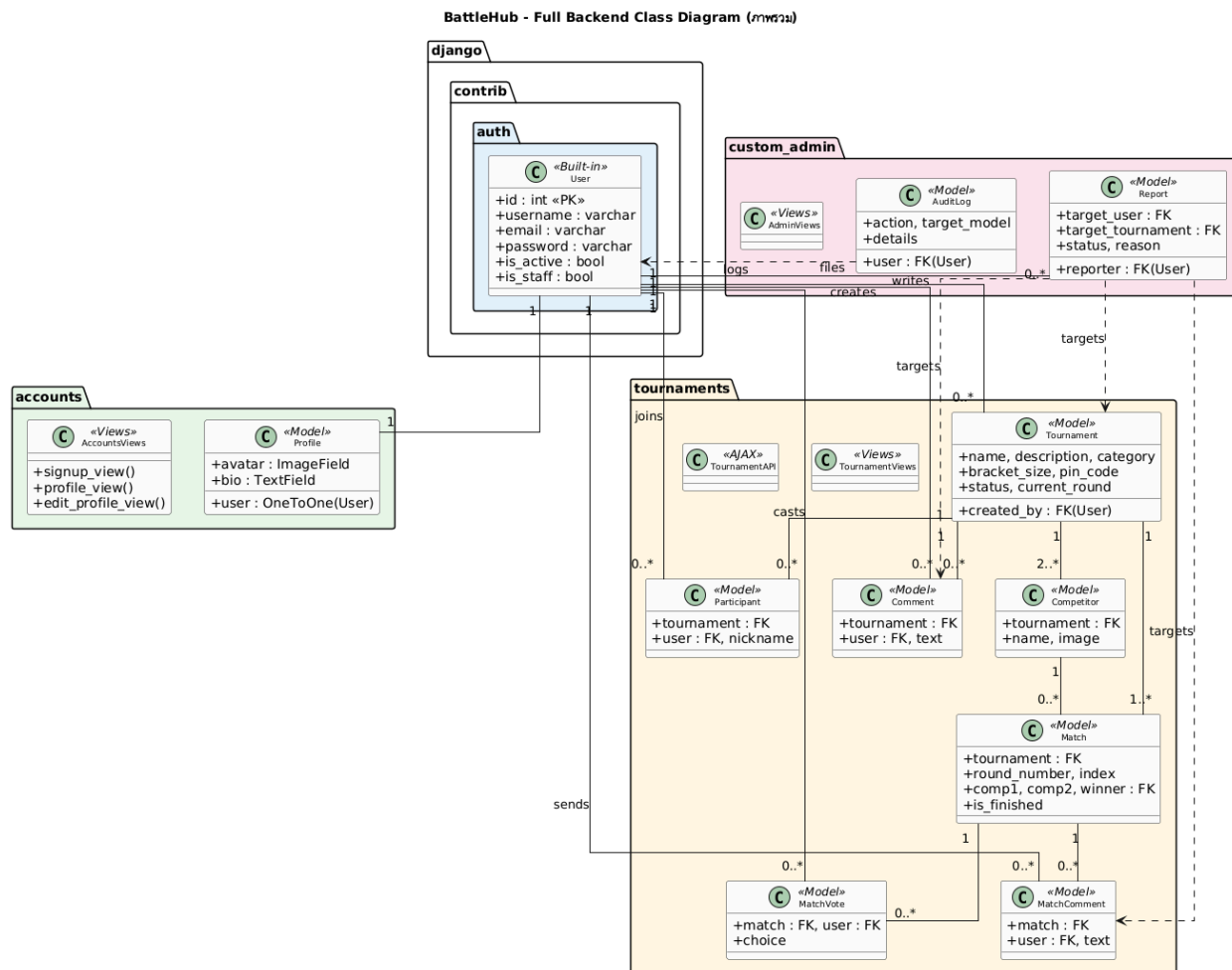


(รูปที่ 3.15 แผนภาพคลาส Backend — Custom Admin App)

คำอธิบายภาพ: จากภาพที่ 3.15 แสดงแผนภาพคลาสของ Custom Admin App ซึ่งรับผิดชอบระบบจัดการสำหรับผู้ดูแลระบบ ประกอบด้วย Model 2 ตัว ดังนี้ (1) Report เก็บข้อมูลการรายงานปัญหาจากผู้ใช้งาน รองรับ 4 ประเภทเป้าหมาย ได้แก่ ผู้ใช้งาน แชนสด ความคิดเห็น และทัวร์นาเมนต์ โดยใช้ Nullable Foreign Key เพื่อความยืดหยุ่น มี 3 สถานะ (pending, resolved, dismissed) และช่อง admin_note สำหรับบันทึกภายใน (2) AuditLog เก็บประวัติการดำเนินการของผู้ดูแลระบบทุกครั้ง ประกอบด้วย ผู้ดำเนินการ ประเภทการกระทำ (BAN, DELETE, FORCE_FINISH ฯลฯ) เป้าหมาย รายละเอียด และ IP Address ฝั่ง Views มี AdminViews ที่มี 14 ฟังก์ชัน ครอบคลุมการจัดการผู้ใช้ ทัวร์นาเมนต์ รายงาน และ Audit Log ทุกฟังก์ชันถูกป้องกันด้วย @admin_required decorator ที่ตรวจสอบ is_staff = True

3.5.1.4 Full Backend Class Diagram (ภาพรวมความเชื่อมโยง)

แผนภาพนี้รวมทุก App เข้าด้วยกัน เพื่อแสดงความเชื่อมโยงข้ามโมดูล โดย User จาก django.contrib.auth เป็นศูนย์กลางที่เชื่อมต่อกับทุก App มีความสัมพันธ์ One-to-One กับ Profile (accounts), One-to-Many กับ Tournament, MatchVote, Comment, MatchComment, Participant (tournaments) และ Report (custom_admin) ส่วน Report เชื่อมโยงกลับไปยัง Tournament, Comment, MatchComment ผ่าน Nullable Foreign Key ทำให้สามารถรายงานเนื้อหาจากหลายแหล่งได้ AuditLog บันทึกการกระทำของ Admin ที่ส่งผลต่อ Model ต่าง ๆ ข้ามทุก App



(รูปที่ 3.16 แผนภาพคลาส Backend ภาพรวม — แสดงความเชื่อมโยงระหว่าง App)

คำอธิบายภาพ: จากภาพที่ 3.16 แสดงแผนภาพคลาส Backend ภาพรวมที่รวมทุก App เข้าด้วยกัน เพื่อแสดงความเชื่อมโยงข้ามโมดูล User จาก django.contrib.auth เป็นศูนย์กลางที่เชื่อมต่อกับทุก App โดยมีความสัมพันธ์ One-to-One กับ Profile ใน accounts, One-to-Many กับ Tournament, MatchVote, Comment, MatchComment และ Participant ใน tournaments และ One-to-Many กับ Report ใน custom_admin Report ใน custom_admin เชื่อมโยงกลับไปยัง Tournament, Comment และ MatchComment ใน tournaments ผ่าน Nullable Foreign Key ทำให้สามารถรายงานเนื้อหาจากหลายแหล่งได้ AuditLog บันทึกการกระทำของ Admin ที่ส่งผลต่อ Model ต่าง ๆ ข้ามทุก App

3.5.2 Frontend Class Diagram

ระบบฝั่ง Frontend แบ่งออกเป็น 2 ส่วน ดังนี้

ส่วน Templates (HTML Pages)

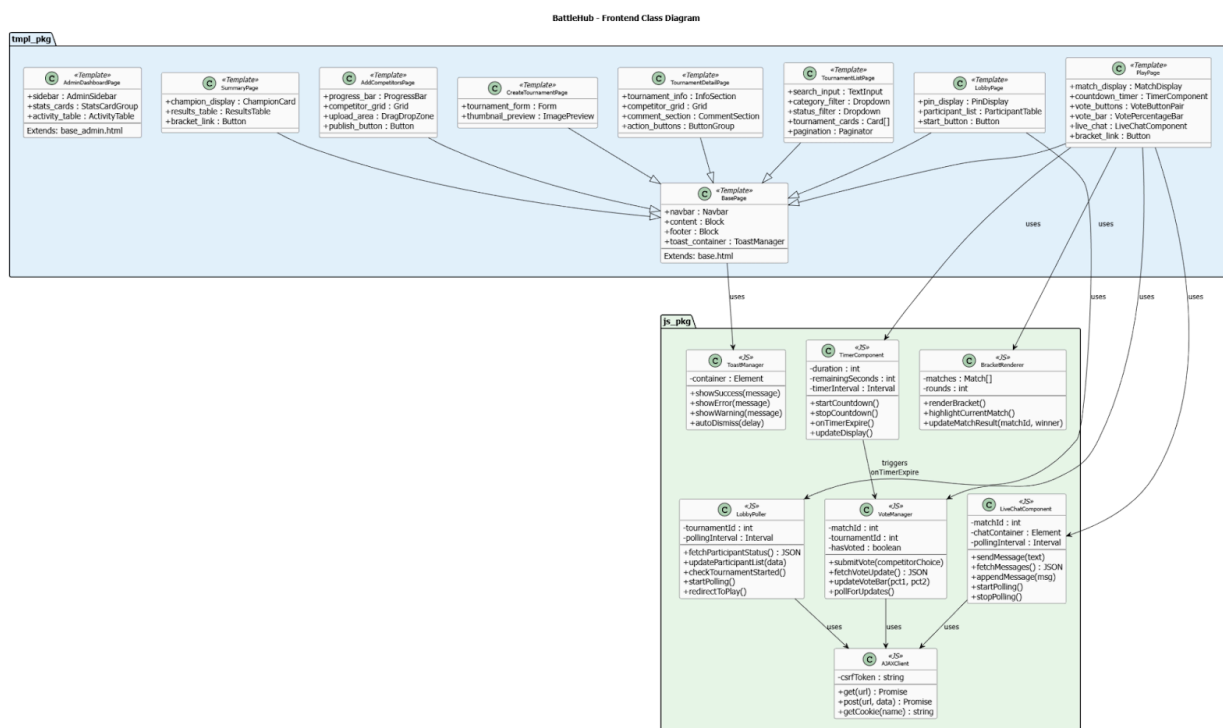
- BasePage เป็น Template หลักที่ทุกหน้าสืบทอด ประกอบด้วย Navbar, Content Block, Footer และ Toast Container
- TournamentListPage หน้ารายการทัวร์นาเมนต์ มีช่องค้นหา ตัวกรองหมวดหมู่ ตัวกรองสถานะ และ Pagination
- TournamentDetailPage หน้ารายละเอียด แสดงข้อมูลทัวร์นาเมนต์ ตารางผู้เข้าแข่งขัน และส่วนความคิดเห็น
- CreateTournamentPage หน้าสร้างทัวร์นาเมนต์ มี Form และ Preview รูปปก
- AddCompetitorsPage หน้าอัปโหลดผู้เข้าแข่งขัน มี Progress Bar, Upload Area และปุ่ม Open Waiting Lobby
- PlayPage หน้าโหวต เป็นหน้าที่ซับซ้อนที่สุด ใช้ JavaScript Component 4 ตัว ได้แก่ TimerComponent, VoteManager, LiveChatComponent, BracketRenderer
- LobbyPage หน้าห้องพักรอ แสดง PIN, รายชื่อผู้เข้าร่วม และปุ่มเริ่มแข่ง
- SummaryPage หน้าสรุปผล แสดงผู้ชนะเลิศและตารางผลทุกแมตช์
- AdminDashboardPage หน้าแดชบอร์ดผู้ดูแลระบบ ใช้ Template แยก (base_admin.html) มี Sidebar, Stats Cards และ Activity Table

ส่วน JavaScript Components

- TimerComponent จัดการตัวนับเวลาถอยหลัง มีเมธอด startCountdown(), stopCountdown(), onTimerExpire(), updateDisplay()
- VoteManager จัดการการลงคะแนนและอัปเดตแถบเปอร์เซ็นต์แบบ Real-time มีเมธอด submitVote(), fetchVoteUpdate(), updateVoteBar(), pollForUpdates()
- LiveChatComponent จัดการแชทระหว่างโหวต มีเมธอด sendMessage(), fetchMessages(), appendMessage(), startPolling(), stopPolling()

- LobbyPoller ตรวจสอบสถานะห้องพักรอทุก 3 วินาที มีเมธอด fetchParticipantStatus(), updateParticipantList(), checkTournamentStarted(), redirectToPlay()
- AJAXClient คลาสกลางสำหรับเรียก API ผ่าน HTTP มีเมธอด get(), post(), getCookie() จัดการ CSRF Token อัตโนมัติ
- ToastManager จัดการข้อความแจ้งเตือนแบบ Pop-up มีเมธอด showSuccess(), showError(), showWarning(), autoDismiss()
- BracketRenderer วาดสายการแข่งขันในรูปแบบแผนผัง Tournament Bracket มีเมธอด renderBracket(), highlightCurrentMatch(), updateMatchResult()

ทุก Component ที่สื่อสารกับ Backend ใช้ AJAXClient เป็นตัวกลาง TimerComponent มีความสัมพันธ์เชิง Trigger กับ VoteManager เมื่อเวลาหมดลง



(รูปที่ 3.17 แผนภาพคลาส Frontend)

คำอธิบายภาพ: จากภาพที่ 3.17 แสดงแผนภาพคลาสฝั่ง Frontend ของระบบ BattleHub โดยแบ่งออกเป็น 2 Package หลัก ดังนี้ Package "Templates" ประกอบด้วย 9 คลาสที่แทนหน้าจอ HTML หลัก ทุกหน้าสืบทอด (Inherit) จาก BasePage ซึ่งกำหนดโครงสร้างพื้นฐาน ได้แก่ Navbar, Content Block, Footer

และ Toast Container ยกเว้น AdminDashboardPage ที่ใช้ Template แยกต่างหาก (base_admin.html) หน้าที่ซับซ้อนที่สุดคือ PlayPage ที่ใช้ JavaScript Component ถึง 4 ตัว Package "JavaScript Components" ประกอบด้วย 7 คลาส ได้แก่ TimerComponent จัดการตัวนับเวลาถอยหลังและเรียก onTimerExpire เมื่อหมดเวลา, VoteManager จัดการการลงคะแนนและอัปเดตแถบเปอร์เซ็นต์แบบ Real-time, LiveChatComponent จัดการแชทสดด้วย Polling ทุก 2 วินาที, LobbyPoller ตรวจสอบสถานะห้องพักรอทุก 3 วินาที, AJAXClient เป็นคลาสมiddlewareสำหรับเรียก API ผ่าน HTTP, ToastManager จัดการข้อความแจ้งเตือนแบบ Pop-up และ BracketRenderer วาดสายการแข่งขันในรูปแบบแผนผัง ทุกคลาสที่สื่อสารกับ Backend ใช้ AJAXClient เป็นตัวกลาง โดย TimerComponent มีความสัมพันธ์เชิง Trigger กับ VoteManager เมื่อเวลาหมดลง

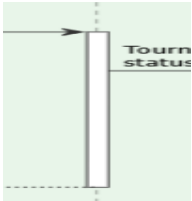
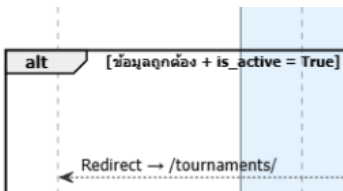
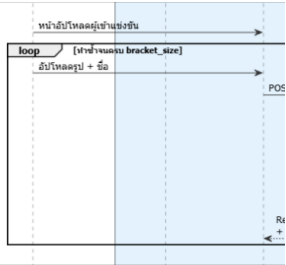
3.6 แผนภาพลำดับ (Sequence Diagram)

แผนภาพลำดับของระบบ BattleHub แสดงลำดับขั้นตอนการทำงานของระบบเมื่อผู้ใช้งานดำเนินการกิจกรรมต่าง ๆ โดยแบ่งตามบทบาทของผู้ใช้งาน 3 กลุ่ม ได้แก่ ผู้เยี่ยมชม (Guest), สมาชิก (Member) และผู้ดูแลระบบ (Admin) เพื่อให้การนำเสนอมีความชัดเจนและไม่ซับซ้อนจนเกินไป จึงได้แบ่งแผนภาพออกเป็นส่วยย่อยตามกลุ่มฟังก์ชันการทำงาน

2. สัญลักษณ์ในแผนภาพลำดับ (Sequence Diagram)

ตารางที่ 3.4 สัญลักษณ์ที่ใช้ใน Sequence Diagram

สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
	Actor	ผู้กระทำ ผู้ใช้งานหรือระบบ ภายนอกที่มีปฏิสัมพันธ์กับระบบ (เช่น ผู้เยี่ยมชม, สมาชิก, ผู้ดูแลระบบ)
 (เส้นตรงแนวตั้ง)	Lifeline	เส้นชีวิต เส้นประแนวตั้งแสดงถึงช่วงเวลาที่ออบเจกต์หรือคลาสมีตัวตนอยู่ในกระบวนการ

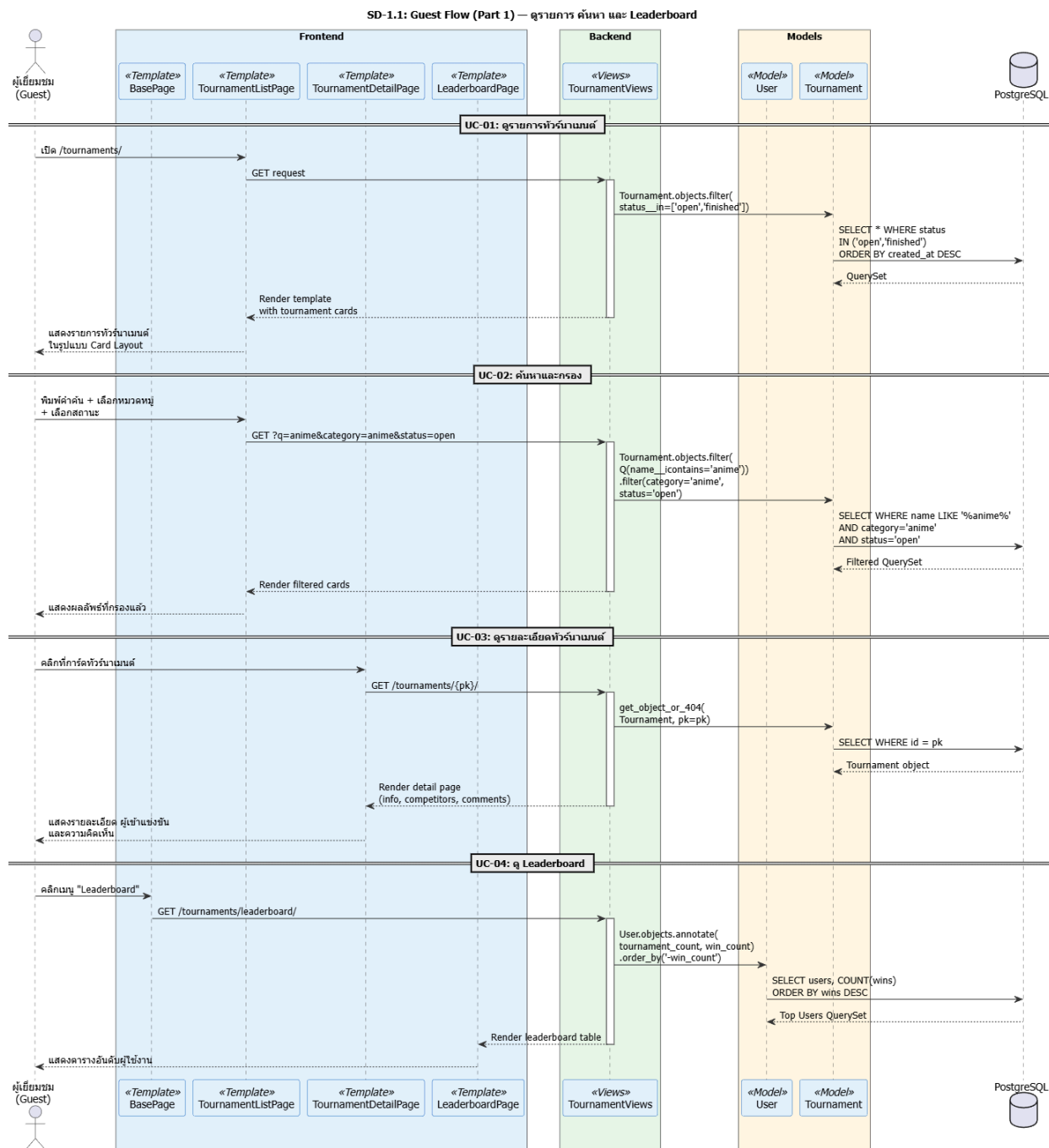
สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
 (Activation bar)	Activation Bar	แถบการทำงาน แท่งสี่เหลี่ยมบน Lifeline แสดงช่วงเวลาที่ย่อจบเจ กต์กำลังประมวลผล (Active)
 (หัวลูกศรทึบ)	Synchronous Message	ข้อความแบบซิงโครนัส การส่งคำสั่งแบบรอการตอบกลับ (เส้นทึบ หัวลูกศรทึบ) เช่น การเรียกใช้ View หรือ API
 (เส้นประ)	Return Message	ข้อความตอบกลับ การส่งผลลัพธ์หรือข้อมูลกลับไปยังผู้เรียก (เส้นประ หัวลูกศรเปิด) เช่น การส่ง Template กลับไปแสดงผล
 (Alt / else)	Alternative Fragment	ทางเลือก (Alternative) กรอบแสดงเงื่อนไขการทำงาน (If-Else) เช่น การตรวจสอบว่า Login สำเร็จหรือไม่
 (Loop)	Loop Fragment	การวนซ้ำ (Loop) กรอบแสดงการทำงานซ้ำ เช่น การวนลูปดึงข้อมูลผู้เข้าแข่งขันมาแสดงในหน้าเว็บ

3.6.1 Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow)

แบ่งการทำงานออกเป็น 2 ส่วน คือ ส่วนการเข้าถึงข้อมูล (Browsing) และส่วนการยืนยันตัวตน (Authentication)

3.6.1.1 ส่วนการเข้าถึงข้อมูล (Browsing)

แสดงขั้นตอนการดูรายการทัวร์นำเมนต์ การค้นหา การดูรายละเอียด และการดูตารางอันดับ



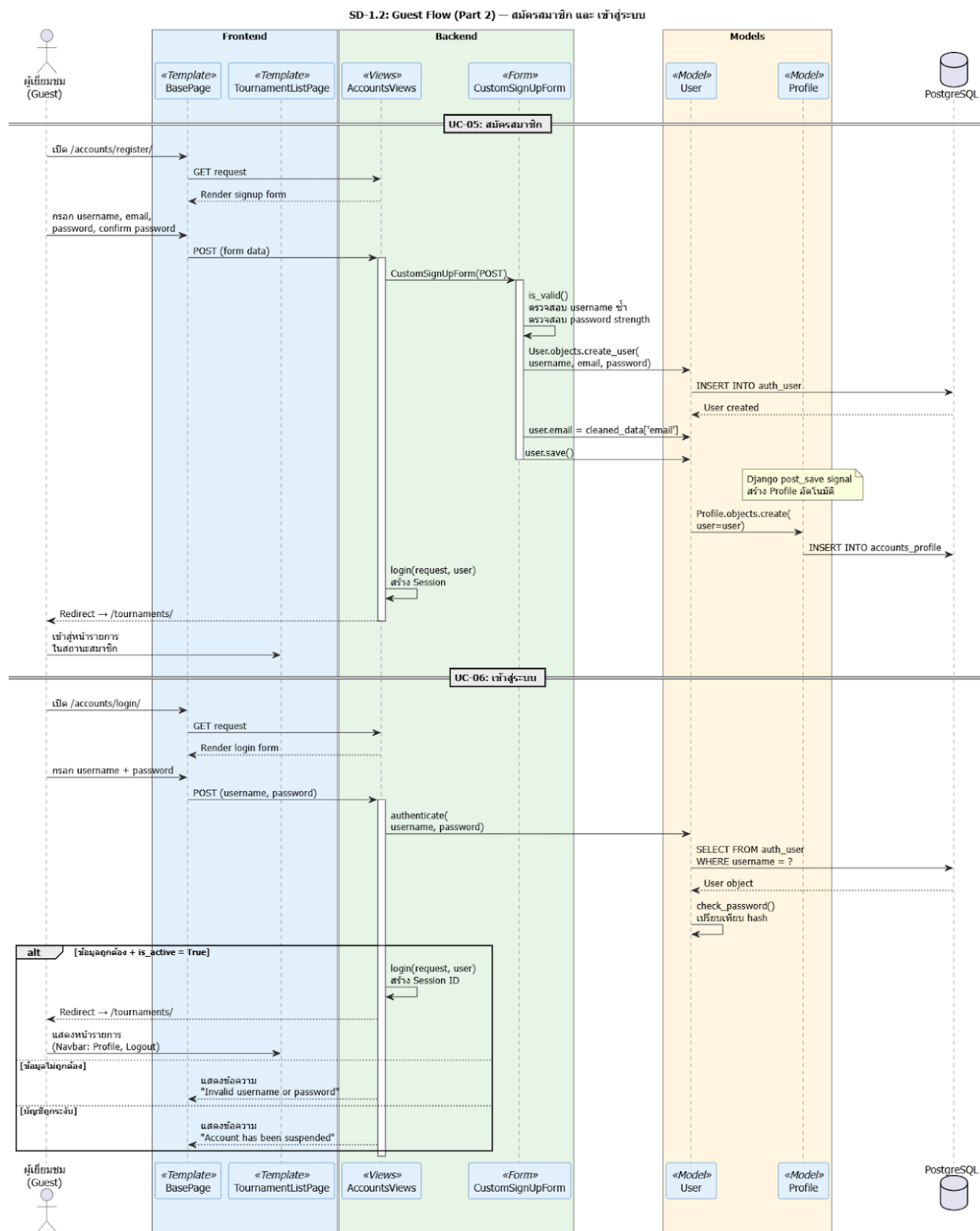
(รูปที่ 3.18 Sequence Diagram ส่วนผู้ใช้งานทั่วไป - Part 1 การเข้าถึงข้อมูล)

คำอธิบายภาพ: จากภาพที่ 3.18 แสดง Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow Part 1) ครอบคลุมการใช้งานพื้นฐานที่ไม่ต้องเข้าสู่ระบบ ได้แก่ การดูรายการทัวร์นาเมนต์ (UC-01) การค้นหาและ

กรองข้อมูล (UC-02) การดูรายละเอียดทัวร์นาเมนต์ (UC-03) และการดู Leaderboard (UC-04) ซึ่งทั้งหมดเป็นการดึงข้อมูล (GET Request) จากฐานข้อมูลมาแสดงผล

3.6.1.2 ส่วนการยืนยันตัวตน (Authentication)

แสดงขั้นตอนการสมัครสมาชิกและการเข้าสู่ระบบ



(รูปที่ 3.19 Sequence Diagram ส่วนผู้ใช้งานทั่วไป - Part 2 การยืนยันตัวตน)

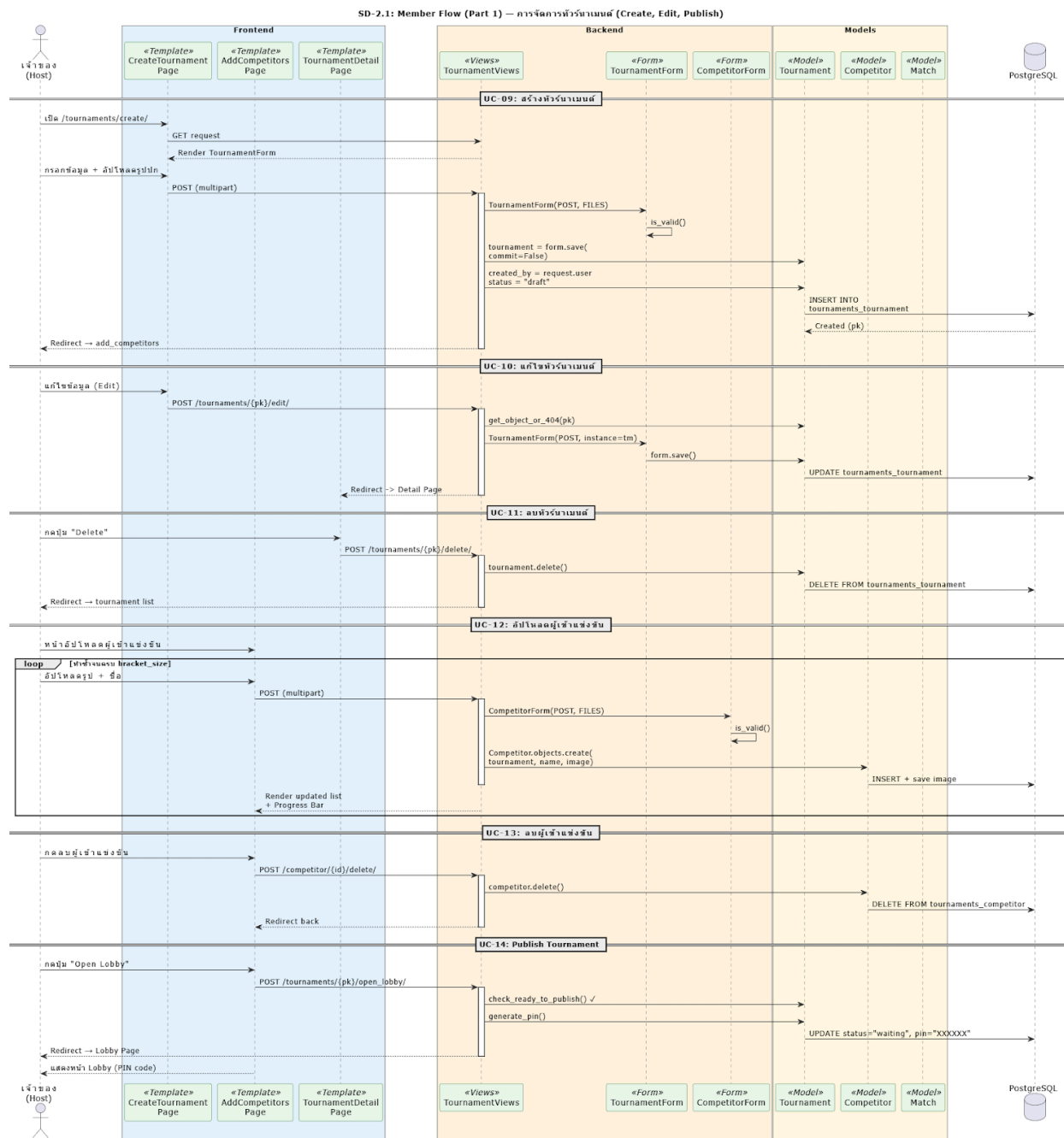
คำอธิบายภาพ: จากภาพที่ 3.19 แสดง Sequence Diagram ส่วนผู้ใช้งานทั่วไป (Guest Flow Part 2) เน้นกระบวนการยืนยันตัวตน เริ่มจากการสมัครสมาชิก (UC-05) ที่มีการสร้าง User และ Profile อัตโนมัติผ่าน Django Signal และการเข้าสู่ระบบ (UC-06) ที่มีการตรวจสอบความถูกต้องของรหัสผ่านและสถานะบัญชี (is_active)

3.6.2 Sequence Diagram ส่วนสมาชิก (Member Flow)

เนื่องจากกระบวนการทำงานของสมาชิกมีความซับซ้อนและมีรายละเอียดจำนวนมาก จึงแบ่งการนำเสนอออกเป็น 2 ส่วน คือ ส่วนการจัดการทัวร์นาเมนต์ (Management) และส่วนการแข่งขัน (Gameplay)

3.6.2.1 ส่วนการจัดการทัวร์นาเมนต์ (Management)

แสดงลำดับขั้นตอนตั้งแต่การสร้างทัวร์นาเมนต์ การแก้ไข การจัดการผู้เข้าแข่งขัน และการเปิดห้องรอแข่งขัน (Open Waiting Lobby)



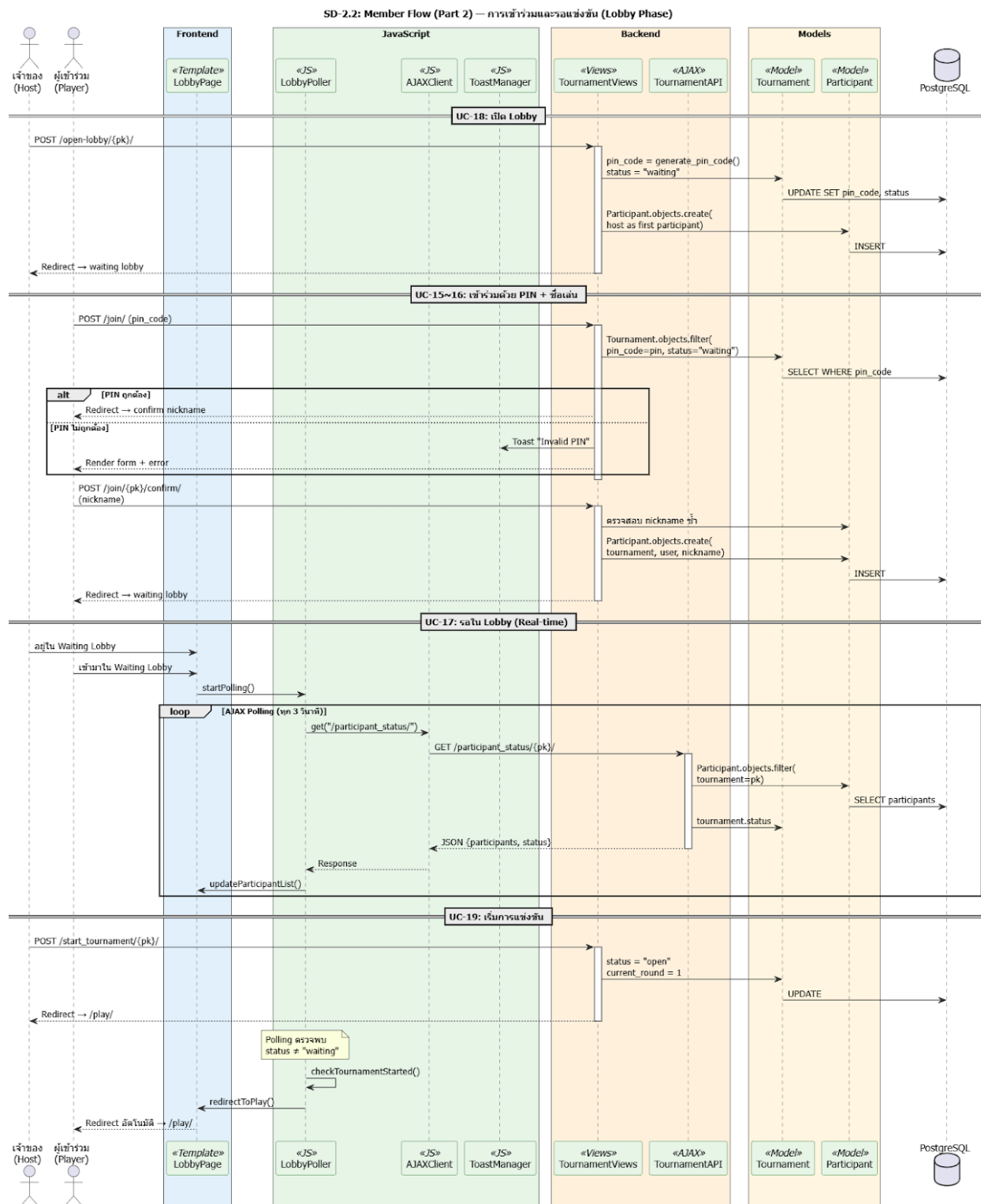
(รูปที่ 3.20 Sequence Diagram ส่วนสมาชิก - Part 1 การจัดการทัวร์นาเมนต์)

คำอธิบายภาพ: จากภาพที่ 3.20 แสดง Sequence Diagram ส่วนสมาชิก (Member Flow Part 1) เน้นกระบวนการจัดการทัวร์นาเมนต์ เริ่มตั้งแต่การสร้างทัวร์นาเมนต์ (UC-09) ซึ่งสถานะเริ่มต้นจะเป็น 'Draft' ผู้ใช้งานสามารถแก้ไข (UC-10) หรือลบ (UC-11) ทัวร์นาเมนต์ได้ ต่อมาคือขั้นตอนการจัดการผู้เข้าแข่งขัน (UC-12, UC-13) ซึ่งต้องอัปโหลดให้ครบตามจำนวน Bracket Size และสุดท้ายคือการเปิดห้องรอแข่งขัน (Open

Waiting Lobby) (UC-14) ซึ่งระบบจะทำการสุ่มจับคู่และสร้าง Match ทั้งหมดเตรียมไว้สำหรับการแข่งขัน และเปลี่ยนสถานะทัวร์นาเมนต์เป็น 'Waiting'

3.6.2.2 ส่วนการเข้าร่วมและรอแข่งขัน (Lobby Phase)

แสดงลำดับขั้นตอนการเปิดห้อง Lobby การเข้าร่วมด้วย PIN และการรอในห้องพัก (Waiting Room)



(รูปที่ 3.21 Sequence Diagram ส่วนสมาชิก - Part 2 การเข้าร่วมและรอแข่งขัน)

คำอธิบายภาพ: จากภาพที่ 3.21 แสดง Sequence Diagram ส่วนสมาชิก (Lobby Phase) เริ่มจากการเปิด Lobby (UC-18) และการเข้าร่วมของสมาชิก (UC-15, UC-16) ที่มีการใช้ PIN Code และชื่อเล่น ระบบ Waiting Lobby (UC-17) ใช้ AJAX Polling เพื่ออัปเดตผู้เข้าร่วมแบบ Real-time และเมื่อ Host กดเริ่มแข่งขัน (UC-19) ผู้เล่นทุกคนจะถูก Redirect ไปยังหน้า PlayPage โดยอัตโนมัติ

3.6.2.3 ส่วนการแข่งขันจริง (Gameplay & Real-time)

เน้นการทำงานระหว่างแข่งขัน ได้แก่ การโหวต การแชทสด ระบบจับเวลา และการสรุปผล ซึ่งต้องมีการตอบสนองแบบ Real-time สูง

(รูปที่ 3.22 Sequence Diagram ส่วนสมาชิก - Part 3 การแข่งขันจริง)

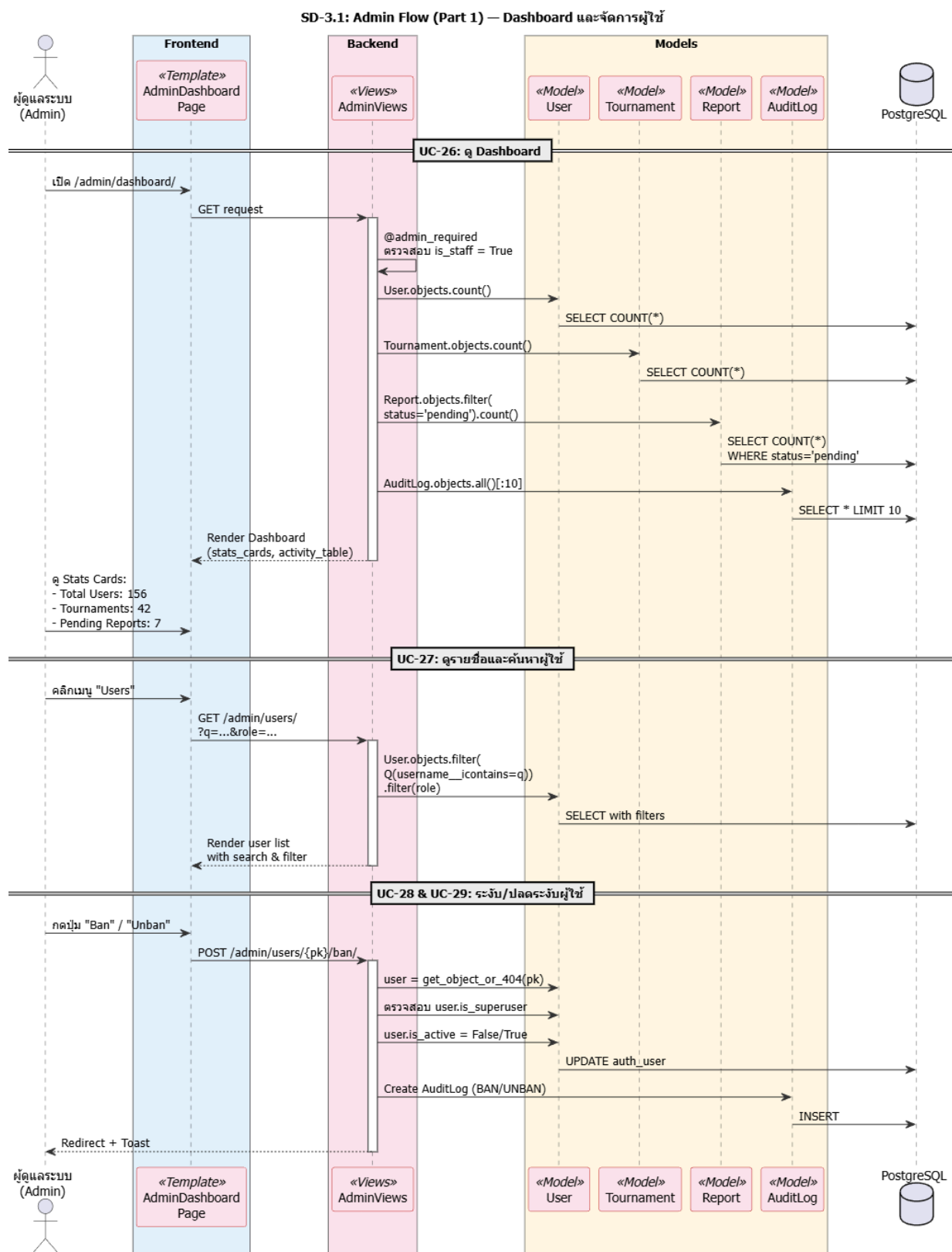
คำอธิบายภาพ: จากภาพที่ 3.22 แสดง Sequence Diagram ส่วนสมาชิก (Gameplay Phase) ครอบคลุมการทำงานขณะแข่งขัน ได้แก่ การโหวต (UC-20) และแชทสด (UC-22) ซึ่งทำงานแบบ Real-time ผ่าน AJAX และ Polling System ระบบจับเวลาจะควบคุมจังหวะการแข่งขันและเปลี่ยนรอบอัตโนมัติเมื่อหมดเวลา สุดท้ายคือหน้าสรุปผล (UC-23) รวมถึงการแสดงความคิดเห็น (UC-24) และการรายงานปัญหา (UC-25)

3.6.3 Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow)

แบ่งการทำงานออกเป็น 2 ส่วน คือ ส่วนการจัดการระบบและผู้ใช้งาน (System & User Management) และส่วนการจัดการเนื้อหาและตรวจสอบ (Content Moderation & Audit)

3.6.3.1 ส่วนการจัดการระบบและผู้ใช้งาน (System & User Management)

แสดงขั้นตอนการดู Dashboard และการจัดการบัญชีผู้ใช้งาน

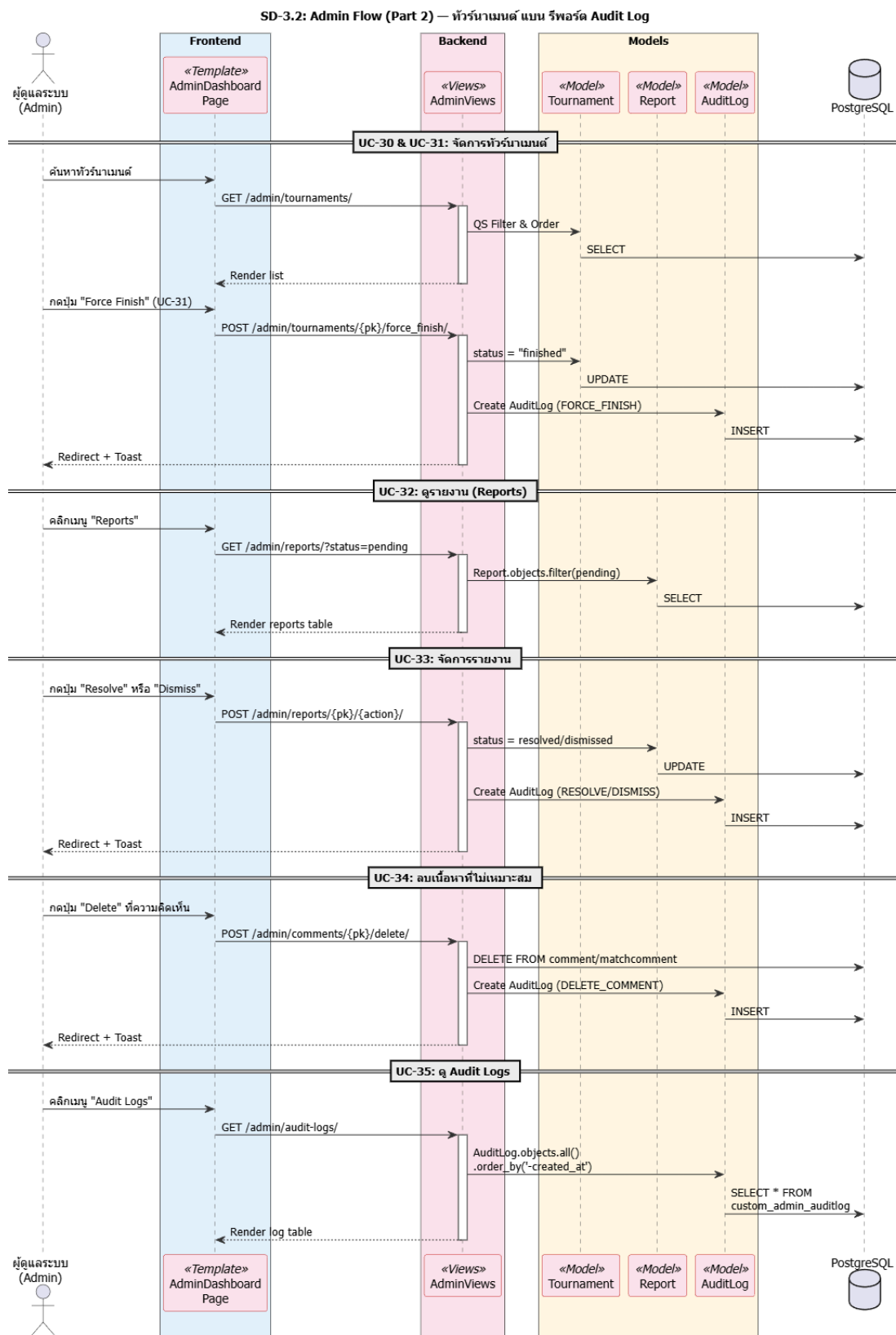


(รูปที่ 3.23 Sequence Diagram ส่วนผู้ดูแลระบบ - Part 2 การจัดการระบบ)

คำอธิบายภาพ: จากภาพที่ 3.23 แสดง Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow Part 1) เน้นการจัดการผู้ใช้งาน เริ่มจากหน้า Dashboard (UC-26) ที่แสดงภาพรวมสถิติระบบ การดูรายชื่อและค้นหาผู้ใช้งาน (UC-27) รวมถึงการระงับ (Ban) และปลดระงับ (Unban) บัญชีผู้ใช้งาน (UC-28, UC-29) ซึ่งมีการตรวจสอบสิทธิ์และบันทึก Audit Log

3.6.3.2 ส่วนการจัดการเนื้อหาและตรวจสอบ (Content Moderation & Audit)

แสดงขั้นตอนการจัดการทวิร์นาเมนต์ การจัดการรายงานปัญหา การลบเนื้อหา และการตรวจสอบบันทึกการใช้งาน (Audit Logs)



(รูปที่ 3.24 Sequence Diagram ส่วนผู้ดูแลระบบ - Part 2 การจัดการเนื้อหาและตรวจสอบ)

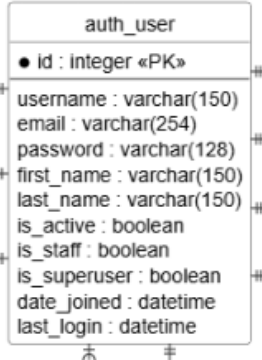
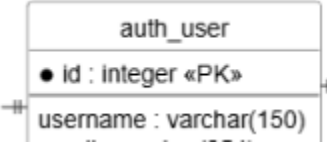
คำอธิบายภาพ: จากภาพที่ 3.24 แสดง Sequence Diagram ส่วนผู้ดูแลระบบ (Admin Flow Part 2) เน้นการจัดการเนื้อหาและความโปร่งใส ได้แก่ การจัดการทัวร์นาเมนต์ (UC-30, UC-31) การจัดการรายงาน (Reports) (UC-32, UC-33) การลบความคิดเห็นที่ไม่เหมาะสม (UC-34) และสุดท้ายคือการดู Audit Logs (UC-35) เพื่อตรวจสอบประวัติการทำงานของผู้ดูแลระบบทุกคน

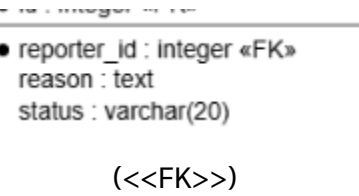
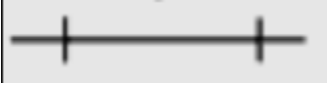
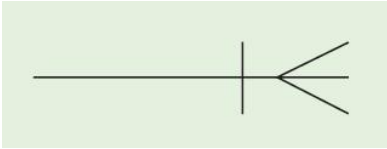
3.7 แบบจำลองข้อมูล (Data Model)

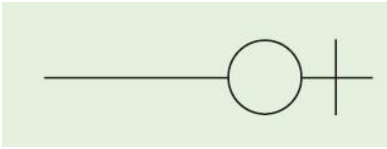
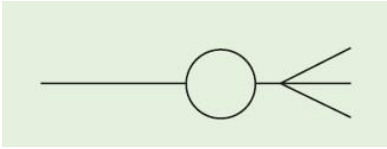
แบบจำลองข้อมูลแสดงโครงสร้างการจัดเก็บข้อมูลในระบบจัดการฐานข้อมูล (Database Schema) โดยระบบ BattleHub ใช้ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ในการจัดเก็บข้อมูล ซึ่งประกอบด้วย ตาราง (Entities) และความสัมพันธ์ (Relationships) ดังแสดงในแผนภาพ Entity Relationship Diagram (ERD)

3. สัญลักษณ์ในแบบจำลองข้อมูล (ER Diagram)

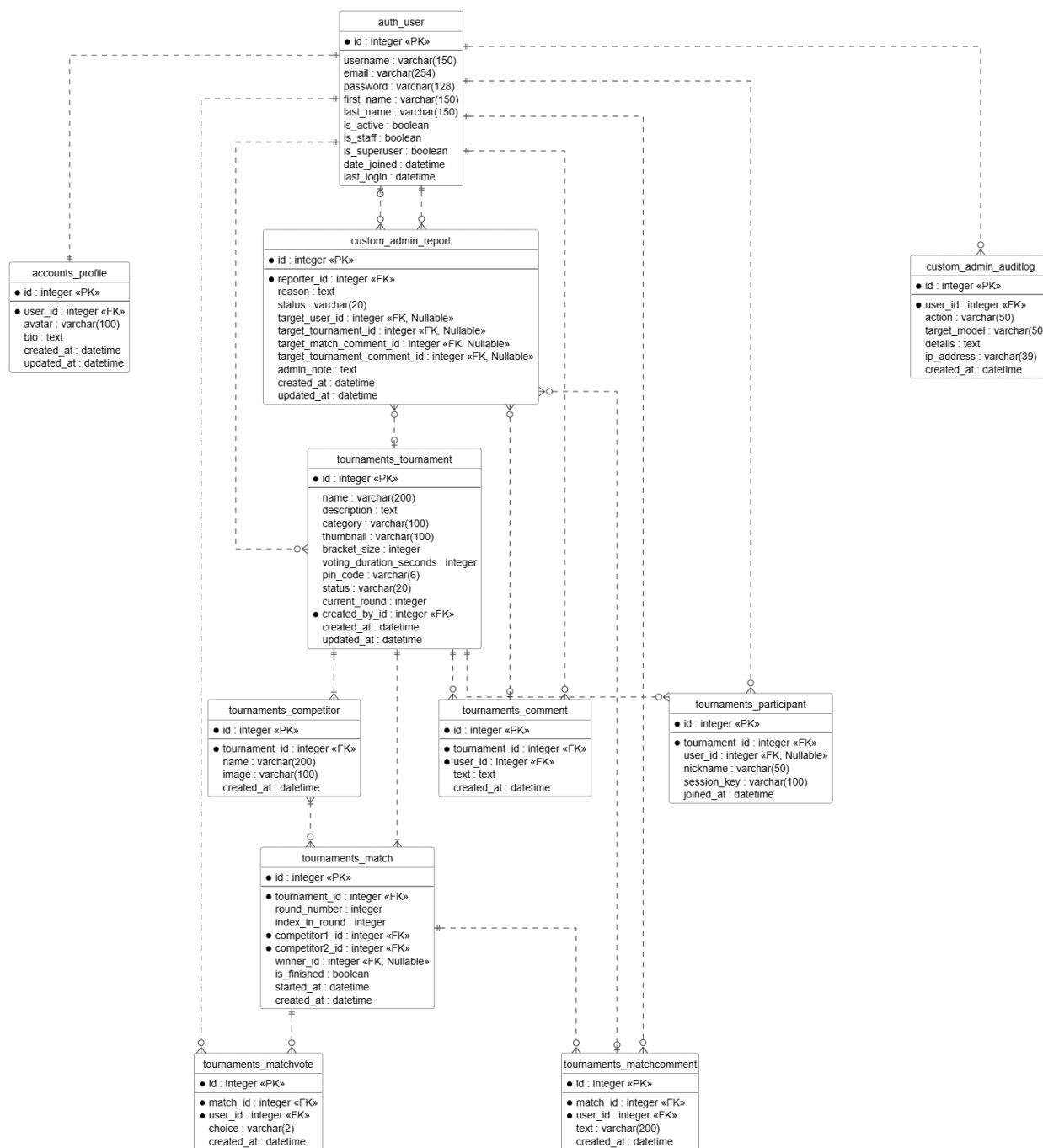
ตารางที่ 3.5 คำอธิบายสัญลักษณ์ในแบบจำลองข้อมูล (Entity Relationship Diagram)

สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
	Entity	เอนทิตี (ตาราง) กล่องสี่เหลี่ยม แสดงชื่อตารางและรายชื่อฟิลด์ (Attributes) ภายในตาราง
 (<<PK>>)	Primary Key	คีย์หลัก ฟิลด์ที่ใช้ระบุความแตกต่างของข้อมูลในแต่ละแถว ไม่สามารถซ้ำกันได้

สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
 ● reporter_id : Integer «FK» reason : text status : varchar(20) («FK»)	Foreign Key	คีย์นอก ฟิลด์ที่ใช้อ้างอิงไปยัง PK ของตารางอื่น เพื่อสร้างความสัมพันธ์
	One-to-One	หนึ่งต่อหนึ่ง ข้อมูล 1 แถวในตาราง A สัมพันธ์กับข้อมูล 1 แถวในตาราง B (เช่น User กับ Profile)
	One-to-Many	หนึ่งต่อกลุ่ม ข้อมูล 1 แถวในตาราง A สัมพันธ์กับข้อมูลหลายแถวในตาราง B (เช่น User สร้างได้หลาย Tournament)
	Many-to-One	กลุ่มต่อหนึ่ง ข้อมูลหลายแถวในตาราง A สัมพันธ์กับข้อมูล 1 แถวในตาราง B (เช่น หลาย Match อยู่ใน 1 Tournament)
	Mandatory One	ต้องมีหนึ่งเท่านั้น ขีดสองขีดแนวตั้ง แสดงว่าความสัมพันธ์ด้านนั้นต้องมีข้อมูลอย่างน้อย 1 และมากที่สุด 1
	Mandatory Many	ต้องมีอย่างน้อยหนึ่ง ขีดแนวตั้งและตีเินกา แสดงว่าความสัมพันธ์ด้านนั้นต้องมีข้อมูลอย่างน้อย 1 (Many)

สัญลักษณ์ (Symbol)	ชื่อเรียก (Name)	ความหมาย (Description)
	Optional One	มีศูนย์หรือหนึ่ง วงกลมและขีดแนวตั้ง แสดงว่าความสัมพันธ์ด้านนั้นอาจไม่มีข้อมูลเลย หรือมีได้แค่ 1 (Nullable)
	Optional Many	มีศูนย์หรือหลาย วงกลมและติ่งกา แสดงว่าความสัมพันธ์ด้านนั้นอาจไม่มีข้อมูลเลย หรือมีได้หลายข้อมูล (Zero or Many)

3.7.1 Entity Relationship Diagram (ERD)



(รูปที่ 3.25 แผนภาพความสัมพันธ์ของข้อมูล Entity Relationship Diagram)

คำอธิบายแผนภาพ: จากภาพที่ 3.25 แสดงโครงสร้างความสัมพันธ์ของตารางในฐานข้อมูล โดยมีตาราง auth_user เป็นศูนย์กลาง เชื่อมโยงกับตารางอื่น ๆ ดังนี้:

1. Users & Profiles: auth_user มีความสัมพันธ์แบบ One-to-One กับ accounts_profile เพื่อเก็บข้อมูลเพิ่มเติมของผู้ใช้
2. Tournaments System:
 - tournaments_tournament ถูกสร้างโดย User และประกอบด้วย tournaments_competitor (ผู้เข้าแข่งขัน)
 - tournaments_match เก็บข้อมูลการจับคู่แข่งขัน เชื่อมโยงกับ Competitor 2 คน (คู่แข่ง) และ 1 คน (ผู้ชนะ)
 - tournaments_matchvote เก็บคะแนนโหวต เชื่อมโยง User กับ Match (User 1 คน โหวตได้ 1 ครั้งต่อ Match)
 - tournaments_participant เก็บข้อมูลผู้เข้าร่วม Lobby
3. Social Interactions:
 - tournaments_comment เก็บความคิดเห็นในทัวร์นาเมนต์
 - tournaments_matchcomment เก็บแชทสดในแมตช์
4. Admin & Moderation:
 - custom_admin_report เก็บข้อมูลการรายงาน เชื่อมโยงกับ User (ผู้แจ้ง) และเป้าหมายปลายทาง (User, Tournament, Comment, Chat) ผ่าน Nullable Foreign Keys
 - custom_admin_auditlog บันทึกประวัติการกระทำของผู้ดูแลระบบ

3.7.2 พจนานุกรมข้อมูล (Data Dictionary)

รายละเอียดโครงสร้างตาราง (Table Schema) ของระบบ BattleHub มีดังนี้

1. ตาราง auth_user (Users)

ตารางที่ 3.6 จัดเก็บข้อมูลบัญชีผู้ใช้งานระบบ (Default Django User Model)

Field Name	Data Type	Key	Description
id	Integer	PK	รหัสประจำตัวผู้ใช้งาน (Auto Increment)
username	Varchar(150)	UQ	ชื่อผู้ใช้งาน (Unique)
email	Varchar(254)		อีเมลผู้ใช้งาน

Field Name	Data Type	Key	Description
password	Varchar(128)		รหัสผ่าน (Hashed)
is_active	Boolean		สถานะบัญชี (True=ปกติ, False=ระงับการใช้งาน)
is_staff	Boolean		สิทธิ์ผู้ดูแลระบบ (True=Admin)
date_joined	Datetime		วันที่สมัครสมาชิก

2. ตาราง tournaments_tournament (Tournaments)

ตารางที่ 3.7 จัดเก็บข้อมูลทัวร์นาเมนต์

Field Name	Data Type	Key	Description
id	Integer	PK	รหัสทัวร์นาเมนต์
name	Varchar(200)		ชื่อทัวร์นาเมนต์
category	Varchar(100)		หมวดหมู่ (Anime, Game, etc.)
bracket_size	Integer		จำนวนผู้เข้าแข่งขัน (2, 4, 8, 16)
status	Varchar(20)		สถานะ (draft, waiting, open, finished)
pinning_code	Varchar(6)		รหัส PIN 6 หลักสำหรับเข้าร่วม Lobby

Field Name	Data Type	Key	Description
created_by_id	Integer	FK	รหัสผู้สร้างทัวร์นาเมนต์ (Ref: auth_user)
created_at	Datetime		วันที่สร้าง

3. ตาราง tournaments_match (Matches)

ตารางที่ 3.8 จัดเก็บข้อมูลแมตช์การแข่งขัน

Field Name	Data Type	Key	Description
id	Integer	PK	รหัสแมตช์
tournament_id	Integer	FK	รหัสทัวร์นาเมนต์ (Ref: tournaments_tournament)
round_number	Integer		รอบที่แข่งขัน (1, 2, 3...)
index_in_round	Integer		ลำดับแมตช์ในรอบนั้น
competitor1_id	Integer	FK	ผู้เข้าแข่งขันฝ่ายที่ 1 (Ref:tournaments_competitor)
competitor2_id	Integer	FK	ผู้เข้าแข่งขันฝ่ายที่ 2 (Ref:tournaments_competitor)
winner_id	Integer	FK	ผู้ชนะในแมตช์นั้น (Nullable)
is_finished	Boolean		สถานะจบการแข่งขัน

บทที่ 4

การพัฒนาระบบ

ในบทนี้จะกล่าวถึงรายละเอียดการพัฒนาระบบ BattleHub ซึ่งพัฒนาด้วย Django Framework โดยแบ่งเนื้อหาออกเป็นโครงสร้างแอปพลิเคชัน, การตั้งค่าระบบ, การจัดการไฟล์, การพัฒนาฟังก์ชันหลักตาม Use Case และการจัดการส่วนติดต่อผู้ใช้ (UI/UX)

4.1 โครงสร้างแอปพลิเคชันและการตั้งค่า (Project Configuration)

4.1.1 การจัดการ Settings.py สำหรับการเชื่อมต่อฐานข้อมูลและ Static Files

ไฟล์ settings.py เป็นหัวใจหลักในการกำหนดค่าการทำงานของ Django Project ในระบบ BattleHub ได้มีการกำหนดค่าสำคัญดังนี้:

1. การลงทะเบียนแอปพลิเคชัน (Installed Apps) เพื่อให้ Django รู้จักและสามารถทำงานร่วมกับแอปพลิเคชันที่สร้างขึ้นใหม่ จำเป็นต้องเพิ่มชื่อ App เข้าไปในตัวแปร INSTALLED_APPS โดยระบบนี้ได้เพิ่มแอปพลิเคชันหลัก 3 ตัว ได้แก่:

Python

```
INSTALLED_APPS = [
    # Django Default Apps...
    'django.contrib.admin',
    'django.contrib.auth',

    # BattleHub Apps
    'accounts',      # จัดการ User & Profile
    'tournaments',  # จัดการระบบแข่งขันและ Lobby
    'custom_admin',  # ระบบ Dashboard ของ Admin (แยกจาก Django Admin ปกติ)

    # Third-party Apps
    'tailwind',      # สำหรับจัดการ CSS Framework
```

```
'theme',      # แอปพลิเคชันสำหรับ Theme ของ Tailwind
'django_cleanup', # ช่วยลบไฟล์รูปภาพเมื่อมีการลบข้อมูลใน DB
]
```

2. การเชื่อมต่อฐานข้อมูล (Database Configuration) ระบบเลือกใช้ PostgreSQL เป็นฐานข้อมูลหลักสำหรับ Production เนื่องจากมีประสิทธิภาพสูงและรองรับข้อมูลเชิงสัมพันธ์ที่ซับซ้อน (โดยในสภาพแวดล้อม Local Development อาจใช้ SQLite เพื่อความสะดวก)

Python

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'battlehub_db',
        'USER': 'postgres',
        'PASSWORD': 'password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

3. การจัดการไฟล์สถิตและมีเดีย (Static & Media Files) กำหนดเส้นทางสำหรับเก็บไฟล์ Static (CSS, JS) และ Media (รูปภาพที่ผู้สื้อปโหลด เช่น รูปปกทัวรนาเมนต์):

Python

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

4.1.2 การออกแบบระบบ URL Routing (Urls.py) แบบแยกแอปพลิเคชัน

เพื่อให้ระบบมีความเป็นระเบียบและง่ายต่อการขยายผลในอนาคต โครงการได้ออกแบบ URL Routing แบบกระจาย (Decentralized Routing) โดยใช้ฟังก์ชัน `include()` ในไฟล์ `battlehub/urls.py` เพื่อเชื่อมต่อไปยัง URL ของแต่ละแอปพลิเคชันแยกกัน ดังนี้:

ไฟล์: `battlehub/urls.py` (Project Level)

Python

```
from django.contrib import admin
from django.urls import path, include
from django.views.generic import RedirectView

urlpatterns = [
    # 1. Redirect Root URL ไปยังหน้า Tournament List ทันที เพื่อ UX ที่ดี
    path("", RedirectView.as_view(pattern_name="tournaments:tournament_list",
    permanent=False)),

    # 2. Django Admin (Backend System)
    path("admin/", admin.site.urls),

    # 3. Include URL ของแต่ละ App
    path("accounts/", include(("accounts.urls", "accounts"), namespace="accounts")),
    path("tournaments/", include(("tournaments.urls", "tournaments"),
    namespace="tournaments")),
    path("admin-panel/", include(("custom_admin.urls", "custom_admin"),
    namespace="custom_admin")),
]
```

การกำหนด namespace (เช่น `namespace="tournaments"`) ช่วยให้สามารถอ้างอิง URL ใน Template ได้อย่างถูกต้องและไม่ซ้ำซ้อน เช่น `{% url 'tournaments:play' pk=tournament.id %}`

4.2 โครงสร้างไฟล์และโฟลเดอร์ของระบบ (Django Directory Structure)

4.2.1 โครงสร้างระดับโครงการ (Project Level: battlehub/) และหน้าที่ของไฟล์ WSGI/ASGI

โครงสร้างไฟล์ของ Django Project ถูกจัดวางตามมาตรฐาน MVT (Model-View-Template) โดยมีโฟลเดอร์ battlehub/ ทำหน้าที่เป็นศูนย์กลางการตั้งค่า (Configuration Center) ของระบบ ประกอบด้วยไฟล์สำคัญดังนี้:

- manage.py: เครื่องมือ Command-line สำหรับจัดการโปรเจกต์ (เช่น runserver, makemigrations, migrate, createsuperuser)
- battlehub/settings.py: ไฟล์ตั้งค่าหลักของระบบ (Database, Installed Apps, Static Files, Middleware)
- battlehub/urls.py: ประตูทางเข้าหลัก (Main Gateway) ของ URL Routing ทั้งหมด
- battlehub/wsgi.py: (Web Server Gateway Interface) สำหรับการ Deploy บน Web Server มาตรฐาน (Synchronous)
- battlehub/asgi.py: (Asynchronous Server Gateway Interface) รองรับการทำงานแบบ Asynchronous สำหรับพีเจียร์ Real-time ในอนาคต

4.2.2 โครงสร้างระดับแอปพลิเคชัน (App Level: accounts, tournaments, custom_admin)

ระบบ BattleHub ถูกแบ่งออกเป็น 3 แอปพลิเคชันหลักตามหน้าที่การทำงาน (Modular Design) ดังนี้:

1. accounts/: รับผิดชอบระบบสมาชิกทั้งหมด
 - จัดการ Sign Up, Login, Logout
 - จัดการ User Profile (Avatar, Bio)
2. tournaments/: หัวใจหลักของระบบ
 - จัดการข้อมูล Tournament, Competitor, Match
 - ระบบการแข่งขัน, การโหวต, และ Lobby
3. custom_admin/: ระบบหลังบ้านสำหรับผู้ดูแล
 - Dashboard สรุปสถิติ
 - เครื่องมือจัดการ Users และ Tournaments (Ban, Delete, Force Finish)

4.2.3 หน้าที่ของไฟล์สำคัญในแต่ละแอปพลิเคชัน

ในแต่ละแอปพลิเคชัน จะประกอบด้วยไฟล์มาตรฐานที่ทำหน้าที่เฉพาะเจาะจงตามหลักการ

Separation of Concerns:

- models.py: กำหนดโครงสร้างข้อมูล (Database Schema) โดยใช้ Django ORM
- views.py: เขียน Logic การทำงานหลัก (Business Logic) รับ Request ประมวลผล และส่ง Response กลับ
- forms.py: สร้างแบบฟอร์มสำหรับการรับข้อมูลจากผู้ใช้ ตรวจสอบความถูกต้อง (Validation) และบันทึกลง Model
- urls.py: กำหนดเส้นทาง URL ภายในแอปพลิเคชันนั้นๆ
- admin.py: ลงทะเบียน Model เพื่อให้บริหารจัดการได้ผ่าน Django Admin Interface
- apps.py: ตั้งค่าและลงทะเบียน Config ของแอปพลิเคชัน

4.2.4 การจัดหมวดหมู่ไฟล์เทมเพลต (Templates Hierarchy) และไฟล์สถิต (Static Files)

ระบบแยกไฟล์ HTML และไฟล์ Static (CSS, JS, Images) ไว้อย่างชัดเจนเพื่อความเป็นระเบียบ:

- templates/: เก็บไฟล์ HTML โดยแบ่งโฟลเดอร์ย่อยตามชื่อ App
 - templates/base.html: ไฟล์โครงร่างหลัก (Layout)
 - templates/accounts/: (login.html, signup.html, profile.html)
 - templates/tournaments/: (tournament_list.html, play.html, summary.html)
 - templates/custom_admin/: (dashboard.html, reports.html)
- static/: เก็บไฟล์ CSS, JavaScript และรูปภาพที่ใช้ร่วมกัน
 - static/css/style.css: ไฟล์ CSS หลัก
 - static/images/: โลโก้และรูปภาพไอคอน
 - (Bootstrap/Tailwind CDN ถูกเรียกใช้ผ่าน Base Template โดยตรง)

4.3 การพัฒนาฟังก์ชันการทำงานด้วย Django (Core Implementation)

4.3.1 การขยายความสามารถของ User Model ผ่าน Profile Model

เพื่อให้ระบบสามารถเก็บข้อมูลเพิ่มเติมของผู้ใช้ เช่น รูปประจำตัว (Avatar) และคำแนะนำตัว (Bio) โดยไม่กระทบกับโครงสร้างหลักของ Django User (auth_user) จึงได้ใช้เทคนิค One-to-One Link สร้าง Model Profile เชื่อมต่อกับ User

ไฟล์: accounts/models.py

Python

```
class Profile(models.Model):
    # เชื่อมต่อกับ User แบบ 1-ต่อ-1 (1 User มี 1 Profile เท่านั้น)
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)
    bio = models.TextField(max_length=500, blank=True)

    def __str__(self):
        return f'{self.user.username} Profile'
```

การสร้าง Profile อัตโนมัติด้วย Signals (Signals.py) เพื่อความสะดวกและป้องกันข้อมูลไม่ครบถ้วนระบบใช้ Django Signals (post_save) เพื่อดักจับเหตุการณ์ "เมื่อ User ถูกสร้าง" ให้ทำการ "สร้าง Profile" ให้ทันทีโดยอัตโนมัติ

ไฟล์: accounts/signals.py

Python

```
@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

@receiver(post_save, sender=User)
def save_profile(sender, instance, **kwargs):
    instance.profile.save()
```

4.3.2 การเขียน Logic ใน Views.py (Core Business Logic)

ส่วนนี้จะอธิบายขั้นตอนการทำงานของระบบ (Business Logic) โดยละเอียด ซึ่งสอดคล้องกับแผนภาพลำดับ (Sequence Diagram) ที่ได้ออกแบบไว้ในบทที่ 3 โดยแบ่งออกเป็น 3 ส่วนหลัก ได้แก่ ส่วนผู้ใช้งานทั่วไป (Guest Flow), ส่วนสมาชิก (Member Flow), และส่วนผู้ดูแลระบบ (Admin Flow)

1. SD-01: การดูรายการทัวร์นาเมนต์ (Guest Browsing)

- ฟังก์ชัน (View): TournamentListView
- คำอธิบาย (Goal): ทำหน้าที่ดึงข้อมูลรายการทัวร์นาเมนต์ทั้งหมดที่มีสถานะ "เปิดรับสมัคร" (Open) หรือ "จบการแข่งขันแล้ว" (Finished) มาแสดงผลให้ผู้ใช้ทั่วไปได้รับทราบ
- ขั้นตอนการทำงาน (Logic):
 1. รับ Request: ระบบรับ HTTP GET Request จากผู้ใช้งาน
 2. Query Data: ทำการดึงข้อมูลจาก Database ผ่าน Django ORM โดยใช้คำสั่ง `Tournament.objects.filter()` โดยกรองเฉพาะทัวร์นาเมนต์ที่มีสถานะเป็น open หรือ finished เท่านั้น
 3. Filter & Search: ตรวจสอบ Parameter จาก URL (Query String) หากมีการระบุคำค้นหา หรือหมวดหมู่ ระบบจะทำการกรองผลลัพธ์เพิ่มเติม
 4. Pagination: หากผลลัพธ์มีจำนวนมาก ระบบจะแบ่งหน้า (Pagination) เพื่อไม่ให้โหลดข้อมูลหนักเกินไป
 5. Render Template: ส่งข้อมูล Context (รายการทัวร์นาเมนต์) ไปยังไฟล์เทมเพลต `tournament_list.html` เพื่อแสดงผล
- ตัวอย่างโค้ด (Implementation):

Python

```
def tournament_list(request):
    # รับค่า Search Query และ Filter จาก URL
    search_query = request.GET.get("search", "")
    selected_status = request.GET.get("status", "")

    # Base Query: เรียงตามวันที่ล่าสุด
    tournaments = Tournament.objects.all().order_by('-created_at')
```

```

# 1. Logic การค้นหา (Search)
if search_query:
    tournaments = tournaments.filter(
        Q(name__icontains=search_query) |
        Q(description__icontains=search_query)
    )

# 2. Logic การกรองสถานะ (Filter)
if selected_status:
    tournaments = tournaments.filter(status=selected_status)

# ... (Pagination & Context setup) ...

return render(request, "tournaments/tournament_list.html", context)

```

2. SD-02: การสมัครสมาชิก (Guest Authentication - Sign Up)

- ฟังก์ชัน (View): signup_view
- คำอธิบาย (Goal): จัดการกระบวนการลงทะเบียนสมาชิกใหม่ พร้อมทั้งสร้างข้อมูลส่วนตัว (Profile) อัตโนมัติ
- ขั้นตอนการทำงาน (Logic):
 1. Validate Request: ตรวจสอบว่าเป็น HTTP POST หรือไม่ หากเป็น POST จะรับข้อมูลจากแบบฟอร์ม CustomSignUpForm
 2. Form Validation: ตรวจสอบความถูกต้องของข้อมูล (เช่น รูปแบบอีเมล, รหัสผ่านที่ตรงกัน, ชื่อผู้ใช้ซ้ำ)
 3. Save User: หากข้อมูลถูกต้อง จะบันทึกข้อมูลลงตาราง auth_user
 4. Signal Trigger: ระบบ Django Signal (post_save) จะทำงานอัตโนมัติเพื่อสร้างข้อมูลในตาราง accounts_profile ที่ผูกกับ User นั้นๆ
 5. Login Immediately: ทำการล็อกอินผู้ใช้ทันทีหลังสมัครเสร็จสิ้น (Auto-login)

6. Redirect: ส่งผู้ใช้งานไปยังหน้าแรก (Homepage)

- ตัวอย่างโค้ด (Implementation):

Python

```
def signup_view(request):
    if request.method == "POST":
        form = CustomSignUpForm(request.POST)
        # ตรวจสอบความถูกต้องของข้อมูล
        if form.is_valid():
            user = form.save()
            # เหตุ: Signal (post_save) ใน models.py จะทำงานอัตโนมัติเพื่อสร้าง Profile

            # Login ทันทีหลังสมัครเสร็จ
            login(request, user)
            return redirect("tournaments:tournament_list")
        else:
            form = CustomSignUpForm()
            return render(request, "accounts/signup.html", {"form": form})
```

3. SD-03: การสร้างทัวร์นาเมนต์ (Member Tournament Management)

- ฟังก์ชัน (View): create_tournament_view
- คำอธิบาย (Goal): สำหรับสมาชิกที่เข้าสู่ระบบแล้ว ใช้ในการสร้างทัวร์นาเมนต์ใหม่
- ขั้นตอนการทำงาน (Logic):
 1. Authentication Check: ใช้ Decorator @login_required ตรวจสอบสิทธิ์ว่าผู้ใช้ล็อกอินแล้วหรือไม่
 2. Process Form: รับข้อมูลจาก TournamentForm (ชื่อ, รายละเอียด, รูปภาพปก)
 3. Assign Creator: กำหนดให้ฟิลด์ created_by เป็น User ปัจจุบัน (request.user)
 4. Set Initial Status: กำหนดสถานะเริ่มต้นเป็น draft (ฉบับร่าง)
 5. Save to DB: บันทึกข้อมูลลงฐานข้อมูล
 6. Redirect: เปลี่ยนเส้นทางไปยังหน้าอัปโหลดผู้เข้าแข่งขัน

- ตัวอย่างโค้ด (Implementation):

Python

```
@login_required
def tournament_create(request):
    if request.method == "POST":
        form = TournamentForm(request.POST, request.FILES)
        if form.is_valid():
            # ยังไม่ Save ลง DB ทันที เพราะต้องเติมข้อมูลผู้สร้างก่อน
            tournament = form.save(commit=False)
            tournament.created_by = request.user
            tournament.status = "draft" # กำหนดสถานะเริ่มต้น
            tournament.current_round = 1
            tournament.save()

            return redirect("tournaments:add_competitors", pk=tournament.pk)
        else:
            form = TournamentForm()
            return render(request, "tournaments/tournament_form.html", {"form": form})
```

4. SD-04: ระบบห้องพักรอ (Member Lobby System)

- ฟังก์ชัน (View & API): join_lobby_view และ lobby_status_api
- คำอธิบาย (Goal): จัดการการเข้าร่วม Lobby แบบ Real-time ผู้เล่นสามารถเข้าร่วมด้วย PIN Code และ Host สามารถเห็นรายชื่อผู้เข้าร่วมทันที
- ขั้นตอนการทำงาน (Logic):
 1. Join Lobby (Validation): ผู้เล่นกรอก PIN และชื่อเล่น ระบบตรวจสอบ PIN ว่าตรงกับทัวร์นาเมนต์หรือไม่ และตรวจสอบชื่อซ้ำ
 2. Create Participant: หากถูกต้อง ระบบสร้าง Object Participant ผูกกับ Session ของผู้เล่น
 3. Polling Mechanism: ฝั่ง Frontend (JavaScript) จะส่ง AJAX Request ไปยัง API ทุกๆ 3 วินาที

4. Return Status: API ตรวจสอบสถานะทัวร์นาเมนต์และคืนค่า JSON ที่ประกอบด้วยรายชื่อผู้เข้าร่วมปัจจุบัน และสถานะ (waiting หรือ started)
- ตัวอย่างโค้ด (Implementation):

Python

```
def join_lobby(request):
    """ตรวจสอบ PIN เพื่อเข้าร่วม Lobby"""
    if request.method == "POST":
        pin = request.POST.get('pin', "").strip()
        # ค้นหาทัวร์นาเมนต์จาก PIN
        tournament = Tournament.objects.filter(pin_code=pin).first()

        if tournament.status not in ['waiting', 'open']:
            messages.error(request, "This tournament is not accepting participants.")
            return redirect('tournaments:join_lobby')

        # ส่งไปหน้ากรอกชื่อเล่น (Nickname)
        return redirect('tournaments:join_lobby_confirm', pk=tournament.pk)
    return render(request, 'tournaments/join_lobby_pin.html')

def participant_status(request, pk):
    """API สำหรับ Polling รายชื่อผู้เข้าร่วม"""
    tournament = get_object_or_404(Tournament, pk=pk)
    participants = tournament.participants.all()

    # ส่งข้อมูลกลับเป็น JSON
    return JsonResponse({
        'status': tournament.status,
        'participant_count': participants.count(),
```



```
'participants': participants_data, # List of nicknames
'redirect_url': redirect_url, # URL สำหรับ Redirect เมื่อเริ่มเกม
})
```

5. SD-05: ระบบโหวตและการแข่งขัน (Gameplay & Voting Logic)

- ฟังก์ชัน (API): vote_view และ next_match_api
- คำอธิบาย (Goal): จัดการการลงคะแนนแบบ Real-time และคำนวณผลแพ้ชนะเมื่อหมดเวลา
- ขั้นตอนการทำงาน (Logic):
 1. Receive Vote (AJAX): รับค่า choice ('1' หรือ '2') ผ่าน AJAX POST
 2. Constraint Check: ตรวจสอบ unique_together ในตาราง MatchVote เพื่อป้องกันการโหวตซ้ำ (ถ้าโหวตแล้วจะเป็นการอัปเดตแทน)
 3. Update Score: บันทึกคะแนนลงฐานข้อมูล
 4. Timer Logic: เมื่อเวลาหมด Frontend จะเรียก API เพื่อขอข้อมูลแมตช์ถัดไป
 5. Determine Winner: Server คำนวณคะแนนรวม อัปเดตผู้ชนะ (winner_id) ในตาราง Match และส่งข้อมูล JSON ของแมตช์คู่ถัดไปกลับมา
- ตัวอย่างโค้ด (Implementation):

Python

```
@require_POST
def vote_submit(request, pk):
    """บันทึกคะแนนโหวต (AJAX)"""
    # ... (Get tournament & match logic) ...

    # ใช้ update_or_create เพื่อป้องกันการโหวตซ้ำ (ถ้ามีแล้ว = แก้ไข)
    vote_obj, created = MatchVote.objects.update_or_create(
        match=current_match,
        user=request.user,
        defaults={'choice': choice})
```

```

    )

    return JsonResponse({'success': True, 'created': created, 'choice': choice})

def vote_update(request, pk):
    """API ตรวจสอบสถานะแมตช์และเวลา (Polling)"""
    # ... (Calculate votes & percentages) ...

    # คำนวณเวลาถอยหลัง (Server-side Timer)
    if current_match.started_at:
        elapsed = (timezone.now() - current_match.started_at).total_seconds()
        time_remaining = max(0, tournament.voting_duration_seconds - int(elapsed))

        # AUTO-FINISH Logic: เมื่อเวลาหมดและแมตช์ยังไม่จบ
        if time_remaining <= 0 and not current_match.is_finished:
            # คำนวณหาผู้ชนะจากผลโหวตสูงสุด
            winner = current_match.competitor1 if votes_c1 > votes_c2 else
current_match.competitor2

            current_match.winner = winner
            current_match.is_finished = True
            current_match.save()

        return JsonResponse({'status': 'finished', ...})

    return JsonResponse({ 'match': { ... }, 'time_remaining': time_remaining })

```

6. SD-06: การจัดการผู้ใช้โดยผู้ดูแลระบบ (Admin User Management)

- ฟังก์ชัน (View): admin_ban_user
- คำอธิบาย (Goal): ผู้ดูแลระบบสามารถระงับการใช้งานบัญชีสมาชิกที่ทำผิดกฎ

- ขั้นตอนการทำงาน (Logic):
 1. Permission Check: ตรวจสอบสิทธิ์ด้วย Decorator @admin_required (ต้องเป็น is_staff=True)
 2. Retrieve User: ดึงข้อมูล User เป้าหมายจาก ID
 3. Update Status: แก้ไขสถานะ is_active เป็น False (ทำให้ล็อกอินไม่ได้)
 4. Log Action: สร้างบันทึกในตาราง AuditLog ระบุว่าใครเป็นคนแบน และแบนใคร
 5. Redirect: กลับไปยังหน้า Dashboard
- ตัวอย่างโค้ด (Implementation):

Python

```
@admin_required
def admin_ban_user(request, pk):
    if request.method == 'POST':
        user = get_object_or_404(User, pk=pk)

        # ป้องกันการแบน Superuser
        if user.is_superuser:
            return redirect('custom_admin:user_list')

        # ปรับสถานะเป็น Inactive
        user.is_active = False
        user.save()

        # บันทึก Audit Log เพื่อตรวจสอบย้อนหลัง
        AuditLog.objects.create(
            user=request.user,
            action='BAN',
            target_model='User',
            details=f'Banned user: {user.username} (ID: {pk})',
```

```

        ip_address=request.META.get('REMOTE_ADDR')
    )

    messages.success(request, f'User "{user.username}" has been banned.')

    return redirect('custom_admin:user_list')

```

7. SD-07: การจัดการเนื้อหาโดยผู้ดูแลระบบ (Admin Content Management)

- ฟังก์ชัน (View): admin_delete_tournament
- คำอธิบาย (Goal): ผู้ดูแลระบบสามารถลบทัวร์นาเมนต์ที่ไม่เหมาะสมออกจากระบบ
- ขั้นตอนการทำงาน (Logic):
 1. Permission Check: ตรวจสอบสิทธิ์ Admin
 2. Retrieve & Delete: ดึงข้อมูล Tournament และสั่ง delete()
 3. Cascade Delete: ระบบ Django จะลบข้อมูลที่เกี่ยวข้องทั้งหมด (Matches, Comments, Competitors) ให้อัตโนมัติ (CASCADE)
 4. Cleanup Files: Library django-cleanup จะลบไฟล์รูปภาพออกจาก Server เพื่อคืนพื้นที่
 5. Log Action: บันทึกการลบลงใน AuditLog
- ตัวอย่างโค้ด (Implementation):

Python

```

@admin_required
def admin_delete_tournament(request, pk):
    if request.method == 'POST':
        tournament = get_object_or_404(Tournament, pk=pk)
        name = tournament.name

        # บันทึก Audit Log ก่อนลบ
        AuditLog.objects.create(
            user=request.user,

```

```

        action='DELETE',
        target_model='Tournament',
        details=f'Deleted tournament: {name} (ID: {pk})',
        ip_address=request.META.get('REMOTE_ADDR')
    )

    # ลบ Tournament (Django Cascade จะลบข้อมูลลูกให้อัตโนมัติ)
    tournament.delete()

    messages.success(request, f'Tournament "{name}" has been deleted.')

    return redirect('custom_admin:tournament_list')

```

4.3.3 การสร้างและจัดการฟอร์มด้วย Django Forms (Forms.py)

Django Forms ช่วยลดความซับซ้อนในการจัดการ HTML Form และการตรวจสอบข้อมูล (Validation) โดยในระบบนี้ใช้ ModelForm เพื่อเชื่อมโยง Form เข้ากับ Model โดยตรง

ตัวอย่าง 1: TournamentForm (tournaments/forms.py) ใช้สำหรับสร้างและแก้ไขทัวร์นาเมนต์ โดยมี การกำหนด Widget เพื่อปรับแต่ง CSS Classes ให้เข้ากับธีม

Python

```

class TournamentForm(forms.ModelForm):
    class Meta:
        model = Tournament
        fields = ["name", "description", "category", "language", "thumbnail",
            "bracket_size", "voting_duration_seconds"]
        widgets = {

```

```

        'name': forms.TextInput(attrs={'class': 'bh-input', 'placeholder': 'Enter
tournament name'}),
        'description': forms.Textarea(attrs={'class': 'bh-textarea', 'rows': 4}),
        # ...
    }

```

ตัวอย่าง 2: CustomSignUpForm (accounts/forms.py) ขยายความสามารถจาก UserCreationForm เดิมของ Django เพื่อเพิ่มการรับค่า Email (ซึ่งปกติ Django UserCreationForm ไม่บังคับ)

Python

```

class CustomSignUpForm(UserCreationForm):
    # เพิ่ม Field Email เข้ามาในฟอร์มสมัครสมาชิก
    email = forms.EmailField(required=True, widget=forms.EmailInput(attrs={'class':
'form-control'}))

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

```

บทที่ 5

การทดสอบระบบ

การทดสอบระบบ (System Testing) มีวัตถุประสงค์เพื่อตรวจสอบความถูกต้องของการทำงานในส่วนต่าง ๆ ของเว็บแอปพลิเคชัน BattleHub เพื่อให้มั่นใจว่าระบบทำงานได้ตรงตามขอบเขตและโจทย์ที่กำหนดไว้ โดยครอบคลุมทั้งการทดสอบฟังก์ชันการทำงานหลัก (Functional Testing), พีเจอร์พิเศษแบบ Real-time (Kahoot-style), การทดสอบที่ไม่ใช่ฟังก์ชัน (Non-Functional Testing) และการตรวจสอบสถานะการติดตั้งบน Server (Deployment Verification)

5.1 การทดสอบฟังก์ชันการทำงาน (Functional Testing)

เป็นการทดสอบกระบวนการทำงานพื้นฐานของผู้ใช้งานทั่วไปและสมาชิก เพื่อยืนยันว่าระบบสามารถรองรับ User Journey หลักได้ถูกต้อง

ตารางที่ 5.1 ผลการทดสอบฟังก์ชันหลัก (General User Flow)

รหัสการทดสอบ	รายการทดสอบ (Test Case)	เงื่อนไขที่คาดหวัง (Expected Result)	ผลลัพธ์จริง (Actual Result)	สถานะ
TC-01	การสมัครสมาชิก (Register)	ระบบบันทึกข้อมูลสมาชิกใหม่และเข้าสู่ระบบให้อัตโนมัติ	บันทึกข้อมูลสำเร็จและ Redirect ไปยังหน้าแรก	ผ่าน
TC-02	การเข้าสู่ระบบ (Login)	ผู้ใช้สามารถเข้าสู่ระบบด้วย Username/Password ที่ถูกต้อง	เข้าสู่ระบบได้และแสดงชื่อบน Navbar	ผ่าน
TC-03	การสร้างทัวร์นาเมนต์	ระบบบันทึกข้อมูลทัวร์นาเมนต์สถานะ Draft	บันทึกข้อมูลลงฐานข้อมูลถูกต้อง	ผ่าน

รหัสการทดสอบ	รายการทดสอบ (Test Case)	เงื่อนไขที่คาดหวัง (Expected Result)	ผลลัพธ์จริง (Actual Result)	สถานะ
TC-04	การเพิ่มผู้เข้า แข่งขัน (Bulk Upload)	อัปโหลดรูปภาพ 4 รูปและสร้าง Competitors ได้ ครบถ้วน	ระบบสร้าง Competitor ครบ 4 คนตามจำนวน	ผ่าน
TC-05	การโหวต (Voting)	เมื่อกดโหวต คะแนนของ Competitor ต้อง เพิ่มขึ้น +1	คะแนนเพิ่มขึ้น ทันทีในฐานข้อมูล	ผ่าน
TC-06	การป้องกันโหวต ซ้ำ	ผู้ใช้คนเดิมไม่ สามารถโหวตซ้ำให้ คะแนนเพิ่มเกิน 1 ได้	ระบบเปลี่ยนเป้า หมายการโหวต แทนการนับเพิ่ม	ผ่าน
TC-07	การแสดงผล Real-time	คะแนนและกราฟ แสดงผลอัปเดต โดยไม่ต้องรีเฟรช หน้าจอ	หน้าจออัปเดต ข้อมูลใหม่ทุก 2 วินาที (AJAX)	ผ่าน
TC-08	การทำงานบน Docker	Container ทั้งหมด ต้องมีสถานะ Up และเชื่อมต่อกันได้	Web, DB, Nginx ทำงานและสื่อสาร กันได้ปกติ	ผ่าน

5.2 การทดสอบพีเจอร์ทิพิเศษ (Kahoot-style & Admin)

ส่วนนี้เน้นการทดสอบจุดเด่นของระบบ BattleHub คือระบบ Lobby, การจับเวลา (Timer), และระบบจัดการของผู้ดูแลระบบ (Admin)

ตารางที่ 5.2 ผลการทดสอบระบบ Lobby และ Real-time Features

รหัสการทดสอบ	รายการทดสอบ	เงื่อนไขที่คาดหวัง	ผลลัพธ์จริง	สถานะ
TC-09	การเข้าร่วมผ่าน PIN (Join via PIN)	ผู้เข้าร่วมสามารถกรอก PIN 6 หลักเพื่อเข้ามารรอในห้องได้	เข้าสู่หน้า Waiting Lobby สำเร็จเมื่อ PIN ถูกต้อง	ผ่าน
TC-10	ระบบ Waiting Lobby	Host เห็นรายชื่อผู้เข้าร่วมปรากฏขึ้นทันที (Real-time)	รายชื่อ Update ทันทีเมื่อมีคน Join เข้ามา	ผ่าน
TC-11	ระบบจับเวลา (Server Timer)	เวลานับถอยหลังสัมพันธ์กันทุกเครื่อง (Server-side Sync)	เวลาบนหน้าจอผู้เล่นทุกคนตรงกัน (Sync ทุก 2 วิ)	ผ่าน
TC-12	การแจ้งเตือน (Toast Notification)	แสดง Alert แจ้งเตือนเมื่อเวลาเหลือ 10 วินาทีสุดท้าย	Toast สีเหลืองปรากฏเมื่อเวลาเหลือ 10 วินาที	ผ่าน
TC-13	การเปลี่ยนแมตช์อัตโนมัติ (Auto-redirect)	เมื่อหมดเวลาหรือจบแมตช์ ระบบพาไปหน้าถัดไปอัตโนมัติ	Redirect ไปยังหน้า Bracket/Next Match อัตโนมัติ	ผ่าน
TC-14	การจบการแข่งขัน (Tournament Finish)	เมื่อแข่งครบทุกรอบ ระบบสรุปผลผู้ชนะเลิศ	แสดงหน้า Summary พร้อม	ผ่าน

รหัสการทดสอบ	รายการทดสอบ	เงื่อนไขที่คาดหวัง	ผลลัพธ์จริง	สถานะ
			ชื่อ Champion ถูก ต้อง	

ตารางที่ 5.3 ผลการทดสอบระบบผู้ดูแลระบบ (Admin Features)

รหัสการทดสอบ	รายการทดสอบ	เงื่อนไขที่คาดหวัง	ผลลัพธ์จริง	สถานะ
TC-15	การแบนผู้ใช้งาน (Ban User)	Admin สามารถ ระงับสิทธิ์การใช้ งาน User ได้	User สถานะ เปลี่ยนเป็น Inactive และ Login ไม่ได้	ผ่าน
TC-16	การลบทัวร์นาเม้นต์ (Delete Content)	Admin ลบทัวร์นา เมนต์และข้อมูลที่เกี่ยวข้อง ทั้งหมด	ข้อมูล Tournament, Match, Comments ถูกลบ เกลี้ยง	ผ่าน
TC-17	การดู Audit Logs	ระบบบันทึกการก กระทำสำคัญของ Admin ได้	มีรายการ Log ระบุ User, Action, IP Address ครบถ้วน	ผ่าน

5.3 การทดสอบประสิทธิภาพและความปลอดภัย (Non-Functional Testing)

ตารางที่ 5.4 ผลการทดสอบประสิทธิภาพและความปลอดภัย

หัวข้อการทดสอบ	เป้าหมาย / เกณฑ์	ผลลัพธ์จากการทดสอบ	สถานะ
Response Time	เวลาโหลดหน้าเว็บไม่เกิน 2 วินาที	เฉลี่ย 1.2 วินาที (ทดสอบบน Local Network)	ผ่าน
CSRF Protection	ทุกฟอร์มต้องมี Token ป้องกันการปลอมแปลง	ตรวจพบ {% csrf_token %} ในทุกแบบฟอร์ม	ผ่าน
Responsive Design	ใช้งานได้ดีบนอุปกรณ์มือถือและแท็บเล็ต	Layout ปรับตัวอัตโนมัติ (Grid/Flex) บน Mobile	ผ่าน
Password Hashing	รหัสผ่านต้องไม่ถูกเก็บเป็น Plain Text	ใช้ PBKDF2 ตามมาตรฐาน Django Auth	ผ่าน

5.4 การยืนยันการติดตั้งระบบ (Deployment Verification)

เพื่อยืนยันว่าระบบสามารถทำงานได้จริงในสภาพแวดล้อม Production (จำลอง) ได้ทำการตรวจสอบสถานะของ Container ผ่าน Docker Compose

5.4.1 สถานะ Docker Containers

ตรวจสอบด้วยคำสั่ง docker-compose ps พบว่าบริการหลักทั้ง 3 ส่วนทำงานปกติ

ตารางที่ 5.5 สถานะ Containers | Container Name | Service | Status | Port Mapping | | :--- | :--- | :--- | :--- | | battlehub_db | PostgreSQL Database | Up (healthy) | 5432 | | battlehub_nginx | Web Server / Proxy | Up | 0.0.0.0:80 → 80 | | battlehub_web | Django Application | Up | 8000 |

5.4.2 ช่องทางเข้าใช้งานจริง (Access Points)

เนื่องจากการทดสอบระบบผ่าน Ngrok (Free Tier) จะได้รับ URL แบบสุ่มที่เปลี่ยนแปลงทุกครั้งเมื่อเริ่มการทำงานใหม่ (Dynamic URL) ตัวอย่างเส้นทางเข้าใช้งานมีดังนี้:

- Web Application: <https://c271a6310f28.ngrok-free.app> (สำหรับผู้ใช้งานทั่วไป)
- Django Admin: <https://c271a6310f28.ngrok-free.app/admin> (สำหรับจัดการข้อมูลดิบ)
- Admin Dashboard: <https://c271a6310f28.ngrok-free.app/admin-panel> (สำหรับผู้ดูแลระบบ)

5.5 สรุปผลการทดสอบ

จากการทดสอบระบบตามกรณีทดสอบ (Test Cases) ทั้งหมด 17 รายการ (Functional & Non-Functional) พบว่า:

- ผ่านเกณฑ์ (Passed): 17 รายการ (100%)
- ไม่ผ่านเกณฑ์ (Failed): 0 รายการ

สรุปภาพรวม: ระบบ BattleHub สามารถทำงานได้ถูกต้องตามความต้องการระบบ (System Requirements) ที่กำหนดไว้ ฟังก์ชันสำคัญอย่างการสร้างทัวร์นาเมนต์และการโหวตแบบ Real-time ทำงานได้เสถียร รวมถึงระบบมีความปลอดภัยพื้นฐานและการตอบสนองที่รวดเร็วเพียงพอต่อการใช้งานจริง

บทที่ 6

สรุปและข้อเสนอแนะ

การพัฒนาโครงการพิเศษเรื่อง "ระบบจัดการการแข่งขันและโหวตแบบเรียลไทม์ (BattleHub)" มีวัตถุประสงค์เพื่อสร้างเว็บแอปพลิเคชันที่เป็นศูนย์กลางสำหรับการจัดทัวร์นาเมนต์ออนไลน์ ที่เน้นการมีส่วนร่วมของผู้ชมผ่านระบบโหวต โดยคณะผู้จัดทำได้ดำเนินการวิเคราะห์ ออกแบบ พัฒนา และทดสอบระบบจนเสร็จสมบูรณ์ ซึ่งสามารถสรุปผลการดำเนินงาน ปัญหาที่พบ และข้อเสนอแนะสำหรับการพัฒนาต่อยอดได้ดังนี้

6.1 สรุปความสามารถของระบบ

ระบบ BattleHub สามารถทำงานได้ตามขอบเขตที่กำหนดไว้ครบถ้วน โดยมีความสามารถหลักแบ่งตามกลุ่มผู้ใช้งาน ดังนี้:

1. สำหรับผู้ใช้งานทั่วไป (General User):

- สามารถสมัครสมาชิก (Sign Up) และเข้าสู่ระบบ (Login) ได้อย่างปลอดภัย
- สามารถค้นหาและเรียกดูทัวร์นาเมนต์ที่เปิดรับสมัครหรือกำลังแข่งขันได้
- มีหน้าโปรไฟล์ส่วนตัวที่แสดงรูป Avatar และ Bio ได้

2. สำหรับผู้จัดทัวร์นาเมนต์ (Host):

- สามารถสร้างทัวร์นาเมนต์ใหม่ กำหนดชื่อ รายละเอียด หมวดหมู่ และรูปภาพปกได้
- ระบบรองรับการสร้าง Bracket แบบ Single Elimination อัตโนมัติ รองรับผู้เข้าแข่งขัน 2, 4, 8, 16, 32 คน
- มีระบบ Waiting Lobby สำหรับเปิดให้ผู้เข้าแข่งขัน Join ผ่าน PIN Code แบบ Real-time
- สามารถควบคุมการเริ่มและจบการแข่งขัน รวมถึงการจัดการ Comment ที่ไม่เหมาะสมได้

3. สำหรับผู้ดูแลระบบ (Administrator):

- มี Custom Admin Dashboard ที่แสดงสถิติภาพรวม เช่น จำนวนผู้ใช้ทั้งหมด, ทัวร์นาเมนต์ที่ Active, และกราฟการเติบโต
- มีเครื่องมือจัดการขั้นสูง เช่น การแบนผู้ใช้งาน (User Ban) และการลบทัวร์นาเมนต์ที่ผิดกฎ (Force Delete)
- ระบบบันทึก Audit Logs เพื่อตรวจสอบกิจกรรมย้อนหลังได้

4. ระบบ Real-time (Kahoot-style):

- ระบบโหวตที่แสดงผลคะแนนแบบสด (Live Update) โดยผู้ชมไม่ต้องรีเฟรชหน้าจอ
- ระบบจับเวลา (Server Timer) ที่นับถอยหลังพร้อมกันทุกเครื่องและเปลี่ยนหน้าอัตโนมัติเมื่อหมดเวลา

6.2 ปัญหาและอุปสรรคในการพัฒนา

ในระหว่างการพัฒนาโปรแกรม พบปัญหาและอุปสรรคทางเทคนิคบางประการ ดังนี้:

1. ความซับซ้อนของการจัดการ State แบบ Real-time:

- การทำให้หน้าจอของผู้ใช้ทุกคน (Host และ Participants) แสดงผลตรงกันในเวลาเดียวกัน (Synchronization) มีความซับซ้อน โดยเฉพาะในกรณีที่มีผู้ใช้งานจำนวนมากพร้อมกัน
- *การแก้ไข:* ใช้เทคนิค AJAX Polling ทุก 2-5 วินาที เพื่อดึงข้อมูลสถานะล่าสุดจาก Server อย่างสม่ำเสมอ

2. การแสดงผลบนอุปกรณ์ที่ต่างกัน (Responsive Design):

- การแสดงตารางสายการแข่งขัน (Tournament Bracket) บนหน้าจอมือถือที่มีขนาดเล็กทำได้ยากและมักเกิดการแสดงผลผิดเพี้ยน
- *การแก้ไข:* ใช้ CSS Framework (Tailwind CSS) และออกแบบ Layout ให้สามารถ Scroll แนวนอนได้ในส่วนของตาราง

3. ปัญหาเรื่อง HTTPS ในสภาพแวดล้อม Local:

- พีเจียร์บางอย่าง เช่น การเข้าถึงกล่องหรือ Clipboard API ต้องการ HTTPS ซึ่งการรันบน Localhost ปกติไม่รองรับ
- *การแก้ไข:* ใช้เครื่องมือ Ngrok ในการทำ Tunneling เพื่อจำลอง Public HTTPS URL สำหรับการทดสอบจริง

4. การจัดการรูปภาพจำนวนมาก:

- เมื่อมีการลบทัวร์นาเมนต์หรือผู้ใช้ รูปภาพที่อัปโหลดไว้ไม่ได้ถูกลบออกจาก Server โดยอัตโนมัติ ทำให้เปลืองพื้นที่จัดเก็บ
- *การแก้ไข:* ติดตั้ง Library `django-cleanup` เพื่อช่วยลบไฟล์รูปภาพที่ไม่ได้ใช้งานออกจากระบบโดยอัตโนมัติ

6.3 แนวทางในการพัฒนาต่อ

เพื่อเพิ่มประสิทธิภาพและขยายขีดความสามารถของระบบให้ดียิ่งขึ้น คณะผู้จัดทำมีข้อเสนอแนะสำหรับการพัฒนาต่อยอดในอนาคต ดังนี้:

1. เปลี่ยนไปใช้ WebSockets (Django Channels):
 - ปัจจุบันระบบใช้ AJAX Polling ซึ่งอาจสร้างภาระให้กับ Server หากมีผู้ใช้งานจำนวนมาก การเปลี่ยนไปใช้ WebSockets จะช่วยให้การส่งข้อมูล Real-time รวดเร็วขึ้นและใช้ทรัพยากรน้อยลง
2. เพิ่มรูปแบบการแข่งขัน (Tournament Formats):
 - พัฒนาให้รองรับรูปแบบอื่นๆ นอกเหนือจาก Single Elimination เช่น Round Robin (พบกันหมด) หรือ Double Elimination (แพ้คัดออกสองครั้ง) เพื่อความหลากหลาย
3. ระบบล็อกอินผ่าน Social Media:
 - เพิ่มความสะดวกให้ผู้ใช้งานด้วยการล็อกอินผ่าน Google, Facebook หรือ Line (OAuth2)
4. การ Deploy ขึ้น Cloud Server จริง:
 - นำระบบขึ้นใช้งานบน Cloud Provider เช่น AWS, DigitalOcean หรือ Google Cloud Platform แทนการใช้ Ngrok เพื่อความเสถียรและรองรับผู้ใช้งานได้จริงในวงกว้าง
5. ระบบ Payment Gateway:
 - พัฒนาระบบชำระเงินเพื่อรองรับการจัดทัวร์นาเมนต์แบบมีเงินรางวัล (Prize Pool) หรือค่าสมัครเข้าร่วมแข่งขัน

บรรณานุกรม

1) เอกสารคู่มือและเอกสารอ้างอิงของเทคโนโลยี (Official Documentation)

- [1] Python Software Foundation. (n.d.). Python 3 Documentation.
<https://docs.python.org/3/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [2] Django Software Foundation. (n.d.). Django Documentation.
<https://docs.djangoproject.com/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [3] Tailwind Labs Inc. (n.d.). Tailwind CSS Documentation. <https://tailwindcss.com/docs/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [4] Fonticons, Inc. (n.d.). Font Awesome Documentation. <https://fontawesome.com/docs> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [5] Docker Inc. (n.d.). Docker Documentation. <https://docs.docker.com/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [6] PostgreSQL Global Development Group. (n.d.). PostgreSQL Documentation.
<https://www.postgresql.org/docs/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [7] Ngrok. (n.d.). Ngrok Documentation. <https://ngrok.com/docs/> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [8] MDN Web Docs. (n.d.). HTML: HyperText Markup Language.
<https://developer.mozilla.org/en-US/docs/Web/HTML> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [9] MDN Web Docs. (n.d.). CSS: Cascading Style Sheets.
<https://developer.mozilla.org/en-US/docs/Web/CSS> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)
- [10] MDN Web Docs. (n.d.). JavaScript.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript> (สืบค้นเมื่อ 14 กุมภาพันธ์ 2569)

ภาคผนวก

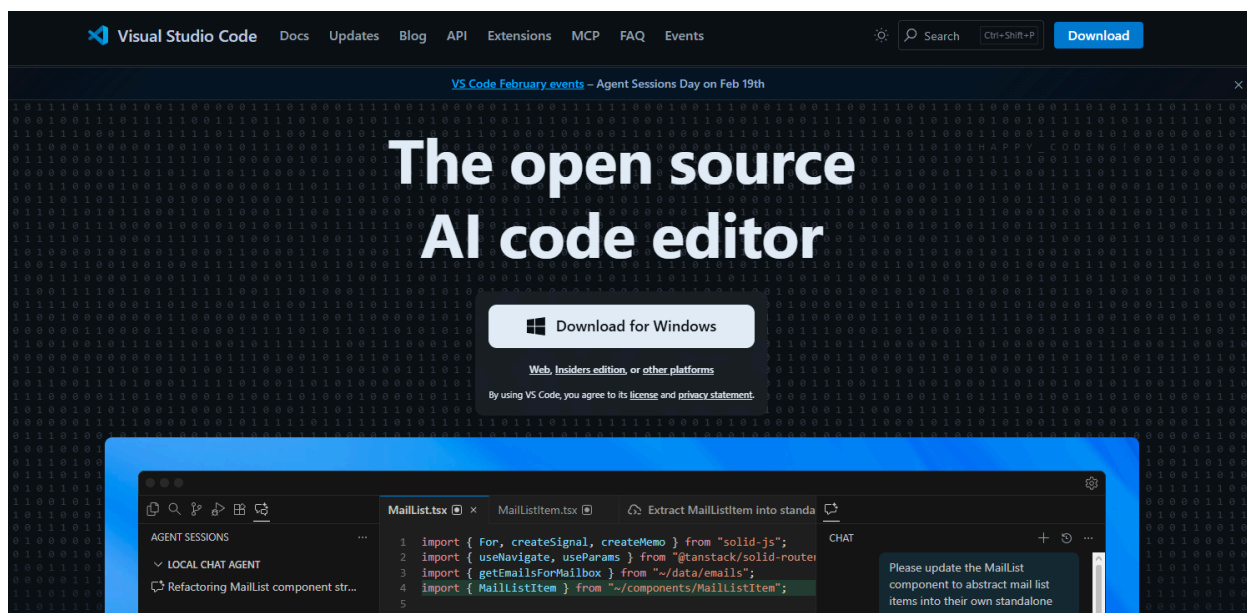
ภาคผนวก ก

การติดตั้งเครื่องมือที่ใช้พัฒนาโปรแกรม

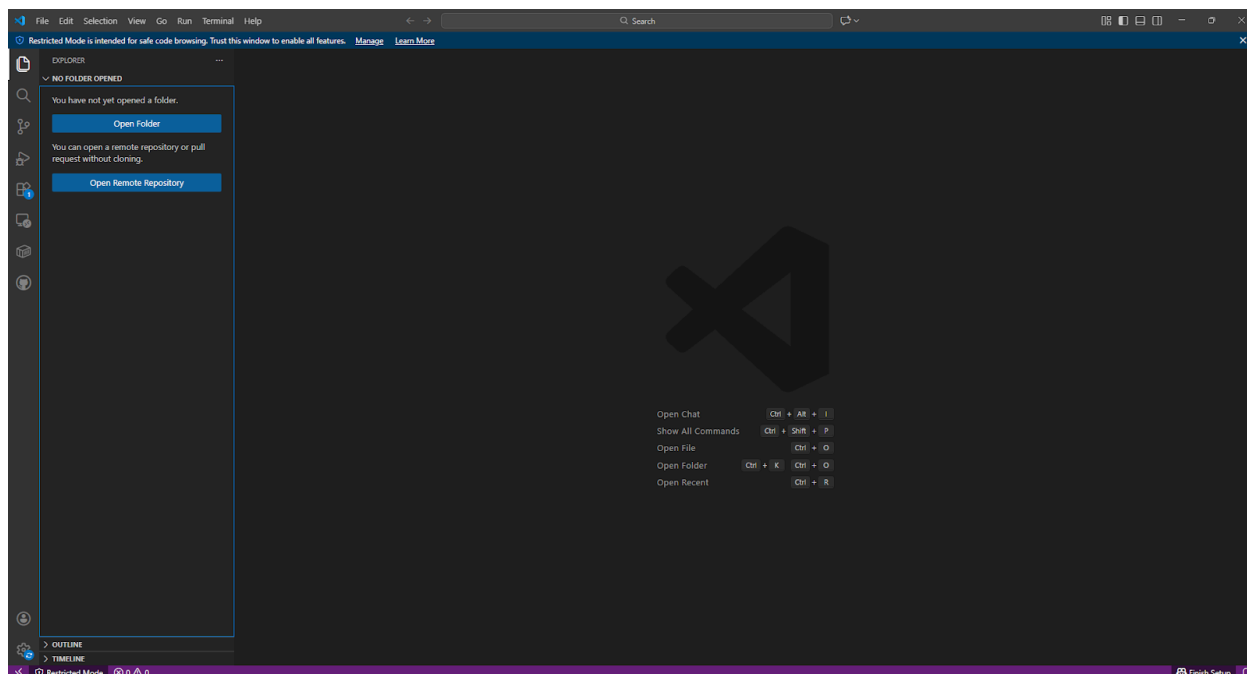
การติดตั้งเครื่องมือที่ใช้ในการพัฒนาเว็บแอปพลิเคชันระบบจัดการการแข่งขันและโหวตแบบเรียลไทม์ (BattleHub) มีโปรแกรมที่จำเป็นในการพัฒนาระบบดังต่อไปนี้

ก.1 การติดตั้ง Visual Studio Code

1. ดาวน์โหลดโปรแกรม Visual Studio Code จากเว็บไซต์ <https://code.visualstudio.com/>
2. ทำการติดตั้งโปรแกรมตามขั้นตอนมาตรฐาน (Default Installation)
3. เปิดโปรแกรม Visual Studio Code และติดตั้ง Extensions ที่จำเป็น เช่น Python, Django, Docker



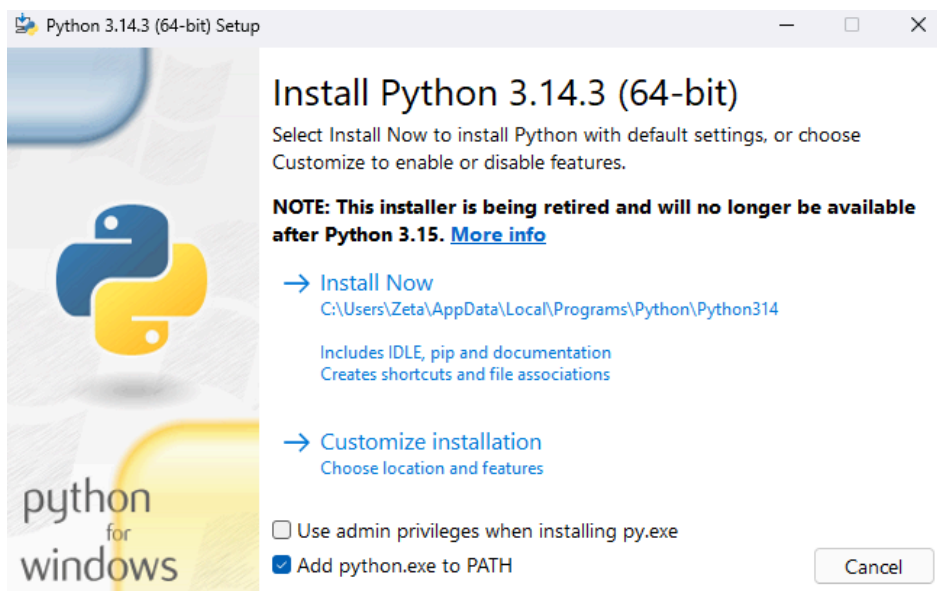
(รูปภาพที่ ก.1 หน้าต่างเว็บไซต์สำหรับดาวน์โหลด Visual Studio Code)



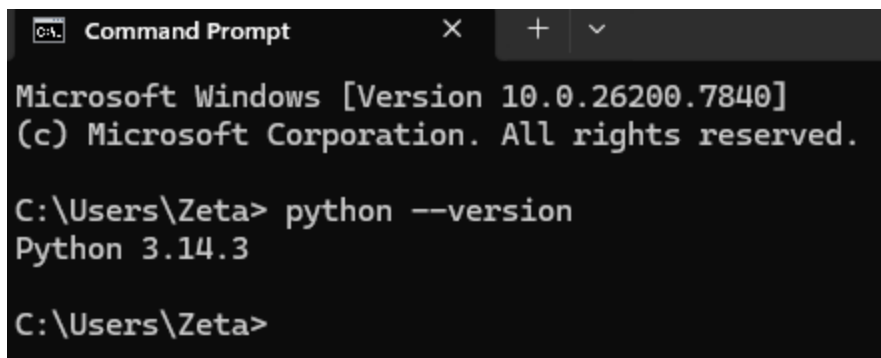
(รูปภาพที่ ก.2 หน้าต่างโปรแกรม Visual Studio Code เมื่อติดตั้งเสร็จสมบูรณ์)

ก.2 การติดตั้ง Python

1. ดาวน์โหลด Python เวอร์ชัน 3.11 ขึ้นไปจากเว็บไซต์ <https://www.python.org/downloads/>
2. รันไฟล์ติดตั้ง และต้องเลือกติ๊กถูกที่ช่อง "Add Python to PATH" ก่อนกด Install
3. ตรวจสอบการติดตั้งโดยเปิด Command Line แล้วพิมพ์คำสั่ง `python --version`



(รูปที่ ก.3 หน้าต่างตัวเลือกการติดตั้ง Python (Add Python to PATH))



```

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Zeta> python --version
Python 3.14.3

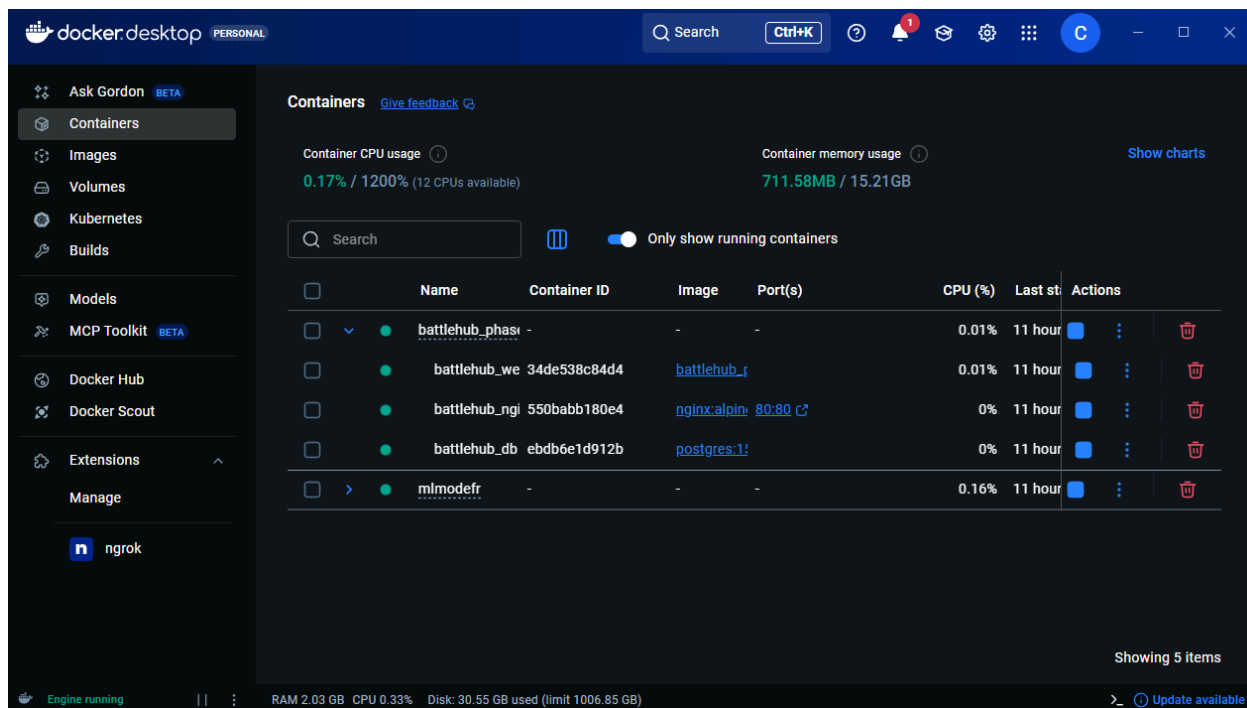
C:\Users\Zeta>

```

(รูปที่ ก.4 หน้าต่าง Command Line แสดงเวอร์ชันของ Python ที่ติดตั้งแล้ว)

ก.3 การติดตั้ง Docker

1. ดาวน์โหลด Docker Desktop จากเว็บไซต์ <https://www.docker.com/products/docker-desktop/>
2. ทำการติดตั้งและ Restart เครื่องคอมพิวเตอร์ 1 ครั้ง
3. เปิดโปรแกรม Docker Desktop และรอจนกว่าสถานะจะขึ้นเป็น "Engine Running" (สีเขียว)



(รูปที่ ก.5 หน้าต่างโปรแกรม Docker Desktop แสดงสถานะ Engine Running)

ภาคผนวก ข

คู่มือการติดตั้งระบบ

เพื่อให้สามารถนำระบบ BattleHub ไปใช้งานหรือพัฒนาต่อยอดได้ผู้ติดตั้งควรปฏิบัติตามขั้นตอนดังนี้

ข.1 การเตรียมสภาพแวดล้อม (Prerequisites)

ก่อนเริ่มการติดตั้ง โปรดตรวจสอบว่าเครื่องคอมพิวเตอร์มีโปรแกรมพื้นฐานดังนี้:

1. Git: สำหรับดาวน์โหลด Source Code
2. Docker Desktop: สำหรับจำลอง Server (แนะนำเวอร์ชันล่าสุด)
3. Command Line / Terminal: (PowerShell หรือ Docker CLI)

ข.2 การติดตั้งด้วย Docker Compose (แนะนำ)

วิธีนี้ง่ายที่สุดและลดปัญหาความเข้ากันได้ของ Environment

ขั้นตอนที่ 1: ดาวน์โหลด Source Code เปิด Terminal และรันคำสั่ง:

Shell

```
git clone https://github.com/username/battlehub.git  
cd battlehub
```

ขั้นตอนที่ 2: ตั้งค่า Environment Variables สร้างไฟล์ `.env` ที่โฟลเดอร์หลัก (Root Directory) และกำหนดค่าดังนี้:

None

```
DEBUG=True  
SECRET_KEY=your-secret-key-change-me-in-production
```

```

ALLOWED_HOSTS=localhost,127.0.0.1,.ngrok-free.app
DATABASE_NAME=battlehub
DATABASE_USER=postgres
DATABASE_PASSWORD=postgres
DATABASE_HOST=db
DATABASE_PORT=5432

```

ขั้นตอนที่ 3: สั่งรันระบบ ใช้คำสั่ง Docker Compose เพื่อสร้างและรัน Container:

Shell

```
docker-compose up --build -d
```

รอกันกว่าระบบจะดาวน์โหลดและติดตั้งเสร็จสิ้น (อาจใช้เวลา 5-10 นาทีในครั้งแรก)

ขั้นตอนที่ 4: สร้างผู้ดูแลระบบ (Superuser)

Shell

```
docker-compose exec web python manage.py createsuperuser
```

(ทำตามขั้นตอนที่ปรากฏบนหน้าจอเพื่อกำหนด Username และ Password)

ข.3 การเข้าใช้งานระบบผ่าน Ngrok (Public Access)

เนื่องจากระบบต้องใช้ HTTPS สำหรับฟีเจอร์บางอย่าง การทดสอบผ่าน Ngrok จึงเป็นวิธีที่เหมาะสม

1. เปิดโปรแกรม Ngrok
2. รันคำสั่งเพื่อ Forward Port 8000:

Shell

```
ngrok http 8000
```

3. คัดลอก Forwarding URL ที่ปรากฏ (เช่น <https://xxxx-xxxx.ngrok-free.app>)
4. นำ URL ไปวางใน Browser เพื่อเริ่มใช้งาน

หมายเหตุ: URL ของ Ngrok จะเปลี่ยนแปลงทุกครั้งที่มีการรันโปรแกรมใหม่ ให้ตรวจสอบ URL
ล่าสุดเสมอ

ข.4 การแก้ไขปัญหาเบื้องต้น (Troubleshooting)

- หากเข้า localhost:8000 ไม่ได้: ตรวจสอบสถานะ Container ด้วย docker-compose ps
- หากรูปภาพไม่แสดง: ตรวจสอบการตั้งค่า MEDIA_URL และ MEDIA_ROOT ใน settings.py หรือลอง
รันคำสั่ง docker-compose exec web python manage.py collectstatic

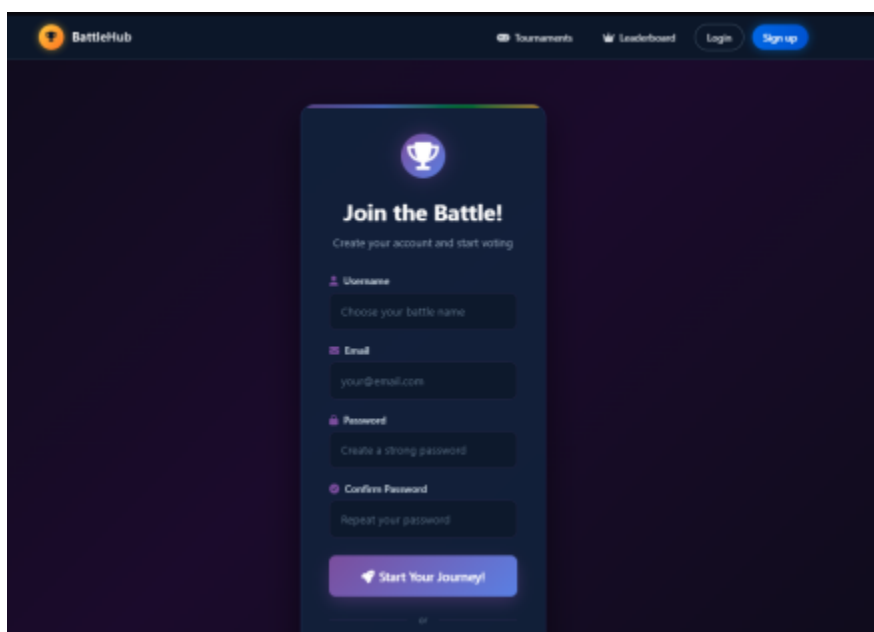
ภาคผนวก ค

คู่มือการใช้งานของระบบ

ค.1 สำหรับผู้ใช้ทั่วไป

ค.1.1 การสมัครสมาชิก

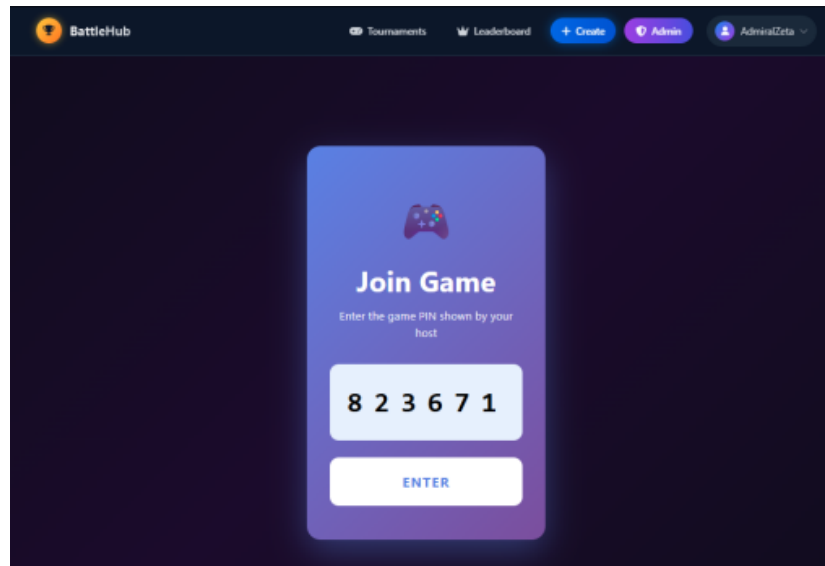
- 1) เข้าสู่หน้าหลักของระบบ
- 2) คลิกปุ่ม "สมัครสมาชิก" หรือ "Register"
- 3) กรอกข้อมูล: ชื่อผู้ใช้, อีเมล, รหัสผ่าน
- 4) คลิก "ยืนยัน" เพื่อสมัครสมาชิก



(รูปที่ ค.1 หน้าสมัครสมาชิก)

ค.1.2 การเข้าร่วมทัวร์นาเมนต์ด้วย PIN

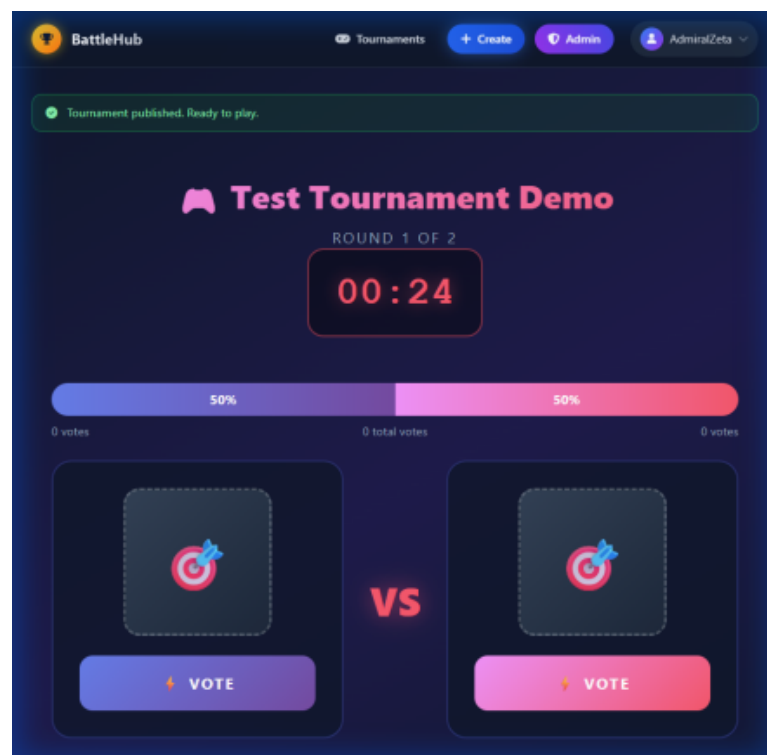
- 1) ได้รับ PIN 6 หลักจากผู้จัดทัวร์นาเมนต์
- 2) คลิก "Join via PIN" ที่หน้าหลัก
- 3) กรอก PIN และ Nickname
- 4) รอในห้อง Lobby จนกว่าเกมจะเริ่ม



(รูปที่ ค.2 หน้า Join via PIN)

ค.1.3 การโหวต

- 1) เมื่อแมตช์เริ่ม จะเห็นผู้เข้าแข่งขัน 2 ฝ่าย
- 2) คลิกปุ่มโหวตได้รูปผู้เข้าแข่งขันที่ต้องการเลือก
- 3) รอนจนหมดเวลา ระบบจะประกาศผู้ชนะอัตโนมัติ



(รูปที่ ค.3 หน้าโหวต)

ค.2 สำหรับผู้จัดทัวร์นาเมนต์

ค.2.1 การสร้างทัวร์นาเมนต์

- 1) เข้าสู่ระบบ
- 2) คลิก "สร้างทัวร์นาเมนต์ใหม่"
- 3) กรอกข้อมูล: ชื่อ, คำอธิบาย, เวลาโหวต
- 4) คลิก "บันทึก"

BattleHub Tournaments Leaderboard + Create Admin AdminZeta

Create tournament

Define basic info and settings. Bracket & images will be handled after this.

Name: Enter tournament name

Bracket size: 4

Description: Describe your tournament...

Waiting duration (seconds): 60

Thumbnail (optional): Choose File No file chosen

Category: other

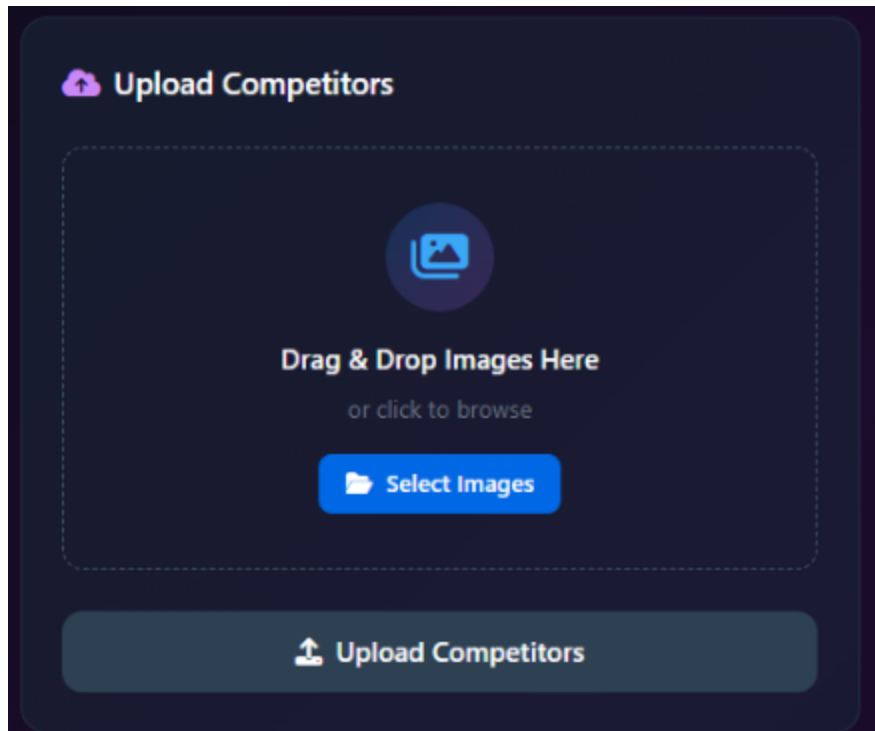
Language: e.g. English, Thai

Cancel Save

(รูปที่ ค.4 หน้าสร้างทัวร์นาเมนต์)

ค.2.2 การอัปโหลดผู้เข้าแข่งขัน

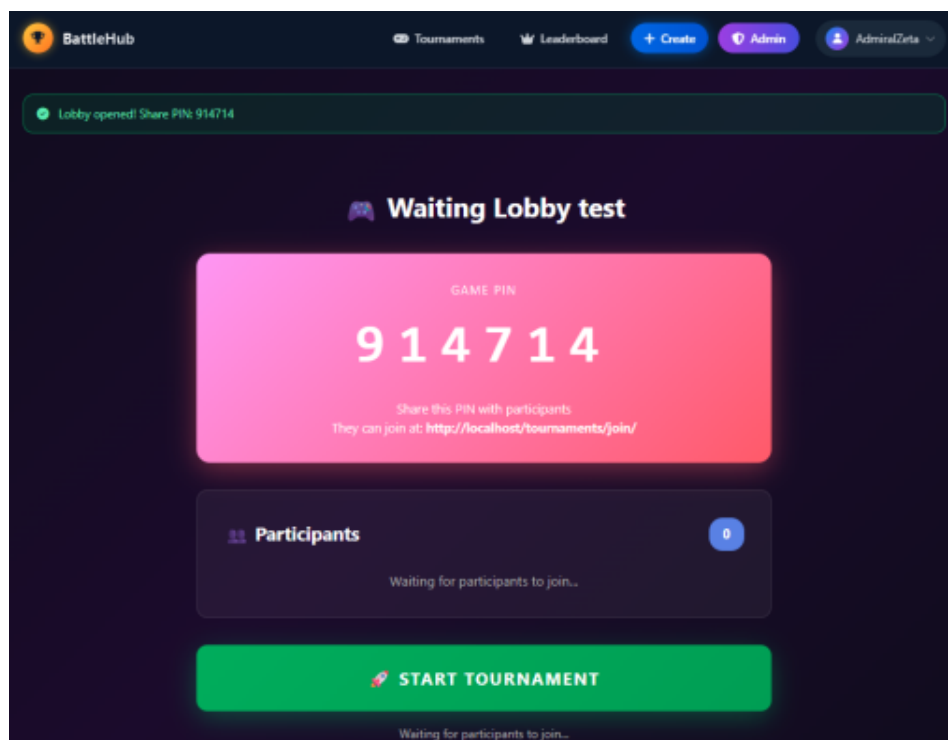
- 1) เข้าหน้ารายละเอียดทัวร์นาเมนต์
- 2) คลิก "Bulk Upload"
- 3) เลือกรูปภาพหลายไฟล์พร้อมกัน
- 4) ระบบจะสร้างผู้เข้าแข่งขันจากชื่อไฟล์
- 5) เมื่ออัปโหลดครบตามจำนวน คลิกปุ่ม 'Open Waiting Lobby' เพื่อเข้าสู่หน้าห้องพักรอ



(รูปที่ ค.5 หน้า Bulk Upload)

ค.2.3 การเริ่มทัวร์นาเมนต์

- 1) เข้าหน้า Waiting Lobby
- 2) รอจนมีผู้เข้าร่วมครบ
- 3) คลิก "Start Tournament"
- 4) ระบบจะสุ่มจับคู่และเริ่มแมตช์แรก

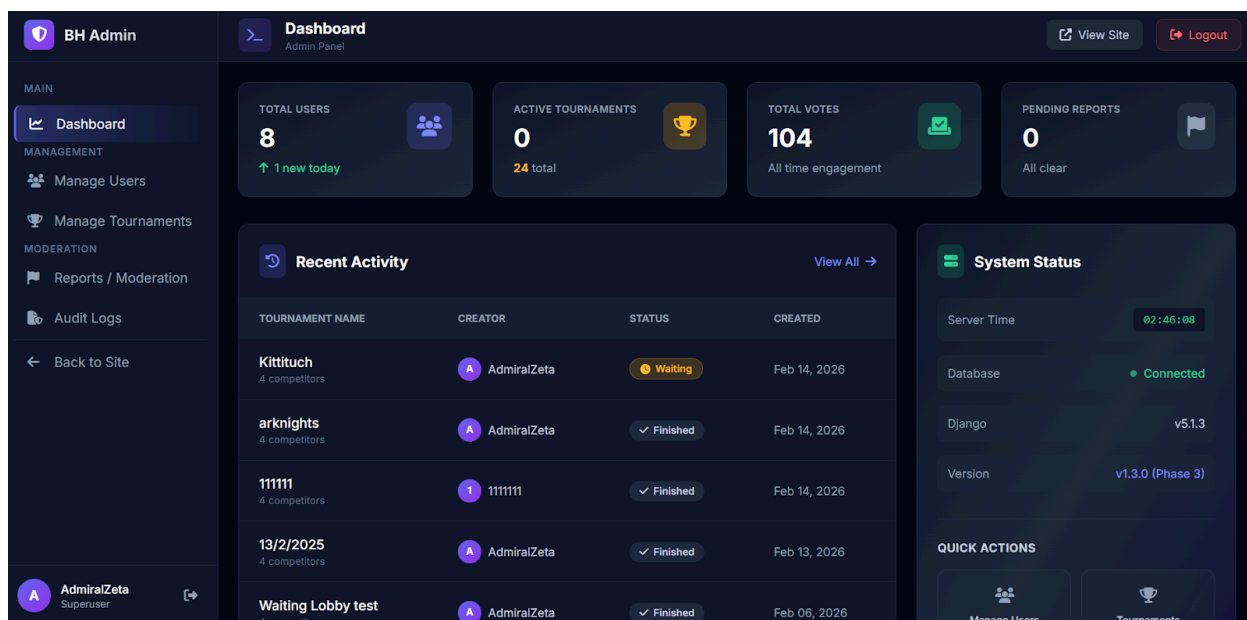


(รูปที่ ค.6 หน้า Waiting Lobby)

ค.3 สำหรับผู้ดูแลระบบ

ค.3.1 การเข้า Admin Dashboard

- 1) เข้าสู่ระบบด้วยบัญชี Admin
- 2) ไปที่ /admin-panel/
- 3) คู่มือการใช้งาน: จำนวนผู้ใช้, ทัวร์นาเมนต์, โหวต



(รูปที่ ค.7 หน้า Admin Dashboard)

ค.3.2 การจัดการ Reports

- 1) เข้าหน้า Reports ผ่าน Admin Dashboard
- 2) ดูรายการ Report ที่ถูกส่งเข้ามา
- 3) คลิก "Resolve" เมื่อดำเนินการเสร็จ หรือ "Dismiss" เพื่อยกเลิก